

從 HDL 到 CMOS

2021 年 11 月 2 日

如果

- 你想要從上到下，了解整台電腦的運作原理，那我會推薦你三門課
 - 1. nand2tetris <https://nand2tetris.org/>
 - 2. xv6 <https://pdos.csail.mit.edu/6.828/2021/xv6.html>
 - 3. ece5745 <https://www.csl.cornell.edu/courses/ece5745/>

nand2tetris

- 教你怎麼設計一台電腦，從硬體到軟體
- 其中的 1-5 章是硬體部分，簡單又好學
- 6-12 章是軟體部分，就會比較複雜了！

所以

- 對於軟體部分，我會建議以 xv6 課程為主
- Xv6 是 MIT 的教學型作業系統，改寫自 UNIX 第六版 (UNIX v6)
- 2018 年以前，xv6 是建構在 x86 處理器上的
但 2019 年開始，則移植到了 RISC-V 處理器上

但是

- 直接學 xv6 有點困難，所以我寫了一本電子書，還有 mini-riscv-os 來幫助大家入門

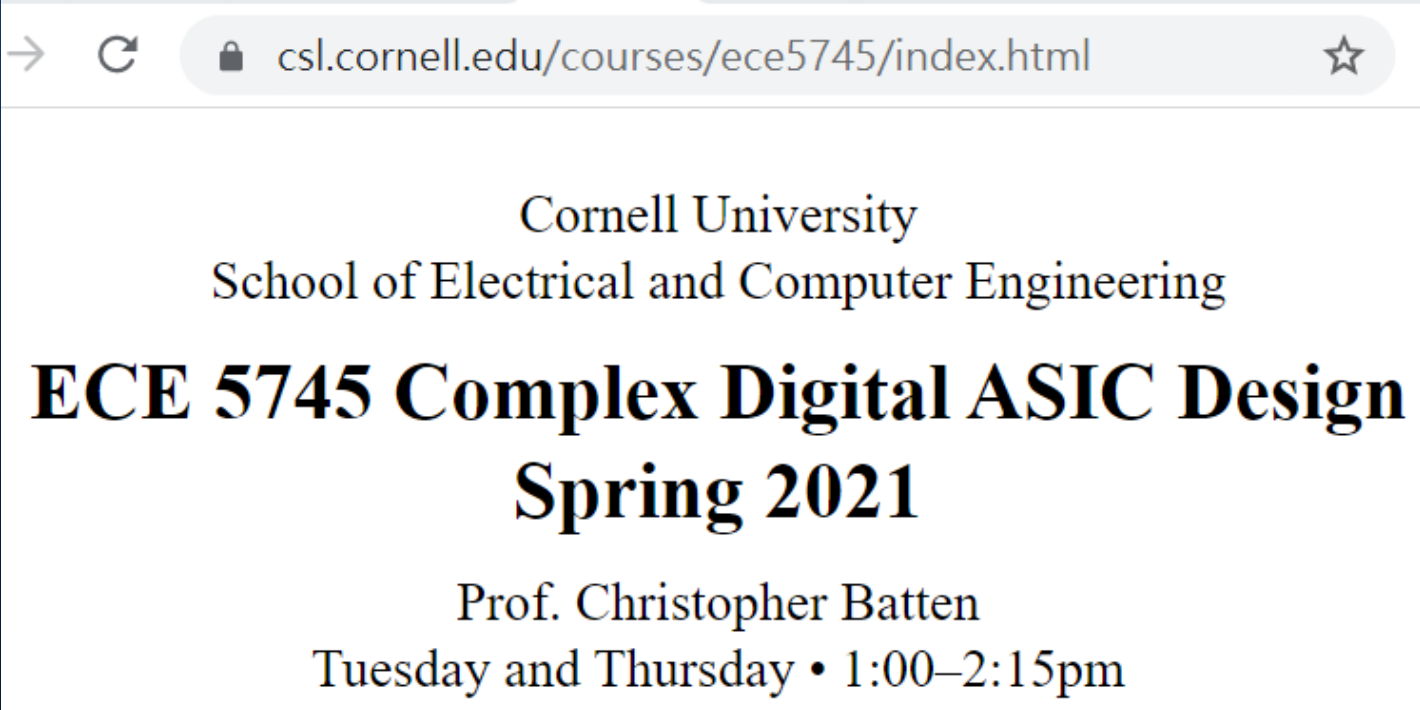


透過這兩門課

- 你可以學會電腦的軟硬體設計
- 但是對於數位邏輯以下的世界，仍然會有所迷惑

因此，我們要推薦第三門課

- 康乃爾大學的 ASIC 課程



A screenshot of a web browser displaying the Cornell University course page for ECE 5745. The browser's address bar shows the URL [csl.cornell.edu/courses/ece5745/index.html](https://www.csl.cornell.edu/courses/ece5745/index.html). The page content is centered and includes the following text: "Cornell University", "School of Electrical and Computer Engineering", "ECE 5745 Complex Digital ASIC Design", "Spring 2021", "Prof. Christopher Batten", and "Tuesday and Thursday • 1:00–2:15pm".

→ ↻ [csl.cornell.edu/courses/ece5745/index.html](https://www.csl.cornell.edu/courses/ece5745/index.html) ☆

Cornell University
School of Electrical and Computer Engineering
ECE 5745 Complex Digital ASIC Design
Spring 2021
Prof. Christopher Batten
Tuesday and Thursday • 1:00–2:15pm

這門課的投影片

- 把 CMOS 到 ASIC 這段講得既簡單又清楚

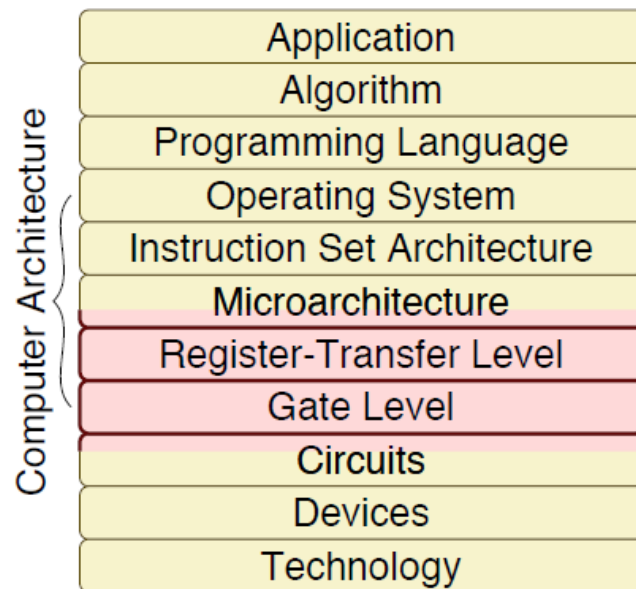
🔒 [csl.cornell.edu/courses/ece5745/handouts.html](https://www.csl.cornell.edu/courses/ece5745/handouts.html)

Lecture Handouts

- [Course Overview \(notes\)](#)
- [Topic 1: Hardware Description Languages \(notes\)](#)
- [Topic 2: CMOS Devices \(notes\)](#)
- [Topic 3: CMOS Circuits \(notes\)](#)
- [Topic 4: Full-Custom Design Methodology \(activity, notes\)](#)
- [Topic 5: Automated Design Methodologies \(activity1, activity2, activity3, notes\)](#)
- [Topic 6: Closing the Gap Between ASIC and Custom \(notes\)](#)
- [Topic 7: Packaging, Power Distribution, Clocking, and I/O](#)
- [Topic 8: Testing and Verification](#)
- [Topic 9: CMOS Combinational Logic \(notes\)](#)
- [Topic 10: CMOS Sequential State \(notes\)](#)
- [Topic 11: CMOS Interconnect \(notes, video\)](#)
- [Topic 12: Synthesis Algorithms \(notes, activity1, activity2\)](#)
- [Topic 13: Physical Design Automation Algorithms \(notes\)](#)

課程的焦點是 ASIC 設計

What is Complex Digital ASIC Design?



Complex digital ASIC design is the process of

quantitatively exploring the area, cycle time, execution time, and energy trade-offs

of various

application-specific accelerators (and general-purpose proc+mem+net)

using

automated standard-cell CAD tools

and then to transform the most promising design to

layout ready for fabrication

但是卻涵蓋了

- 從 HDL 到 CMOS
- 從 EDA 到晶圓代工

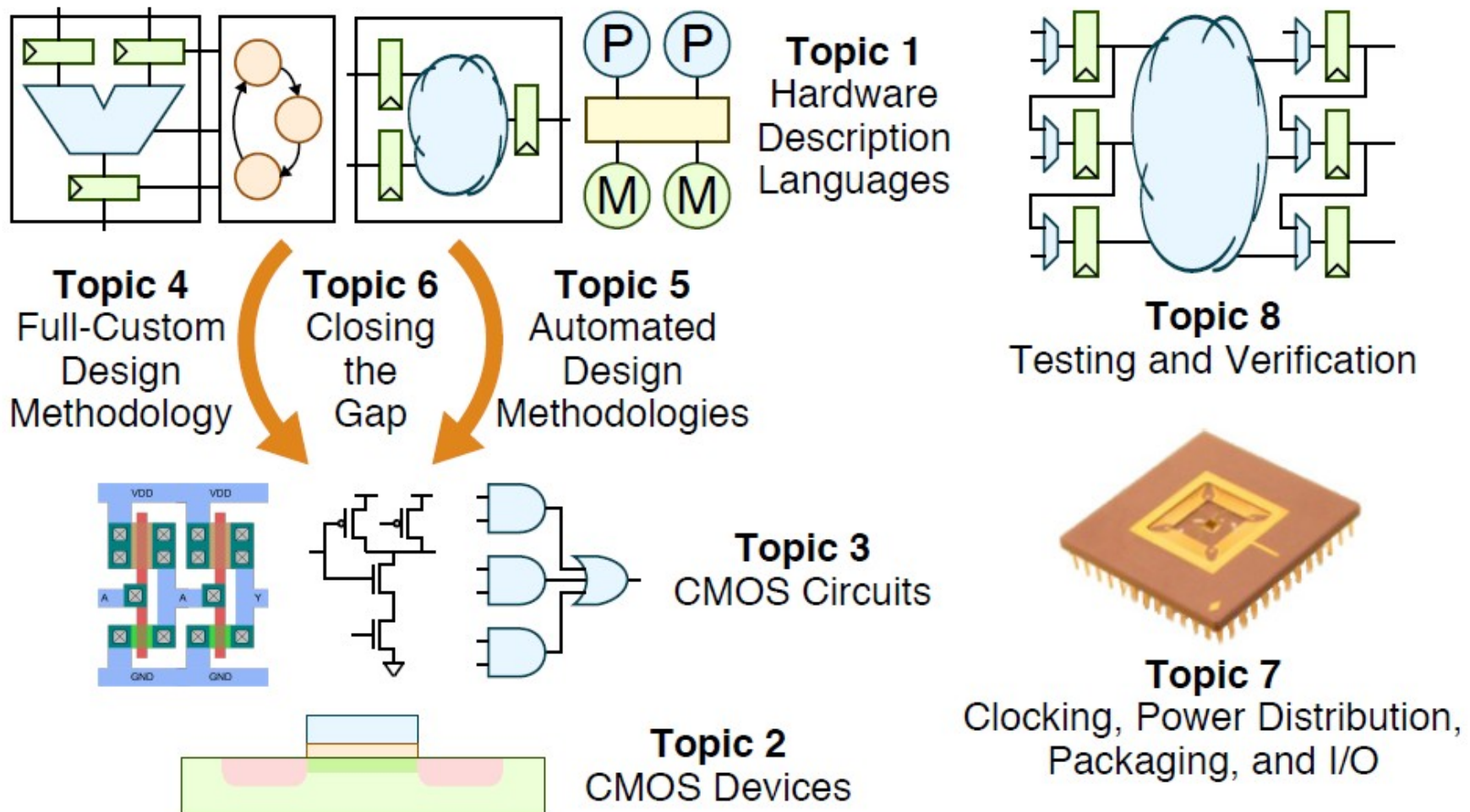
的那些知識 ...

該課程分成三部分

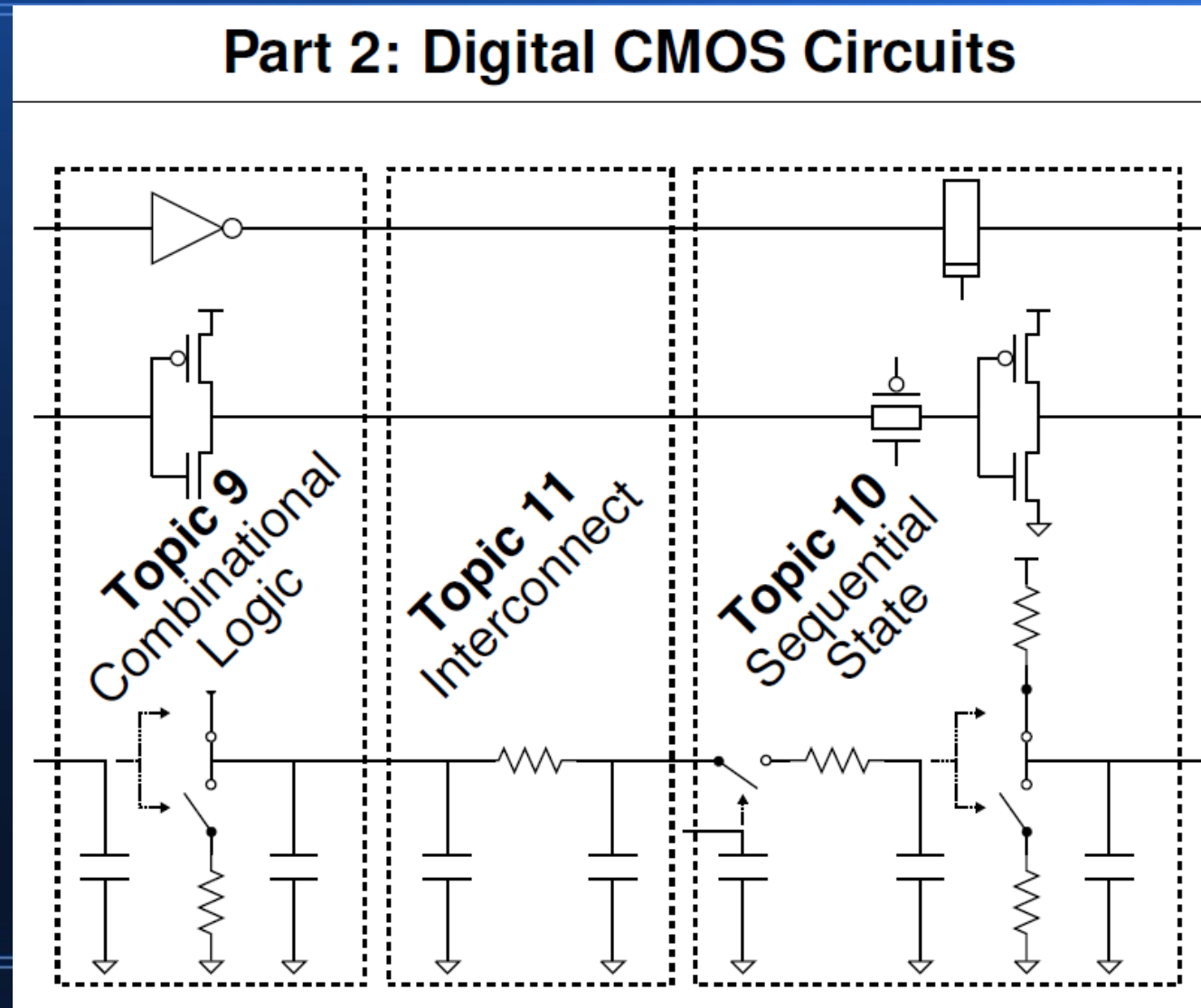
- Part 1: ASIC Design Overview
- Part 2: Digital CMOS Circuits
- Part 3: CAD Algorithms

第一部分的主題如下

Part 1: ASIC Design Overview



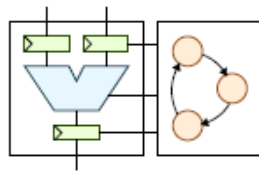
第二部分的主題如下



第三部分的主題如下

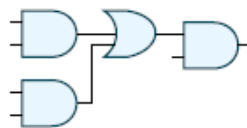
Part 3: CAD Algorithms

Topic 12 Synthesis Algorithms



$$x = a'bc + a'bc'$$
$$y = b'c' + ab' + ac$$

$$x = a'b$$
$$y = b'c' + ac$$



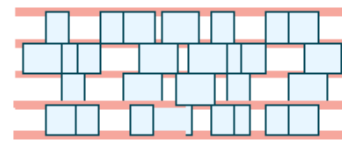
*RTL to Logic
Synthesis*

*Technology
Independent
Synthesis*

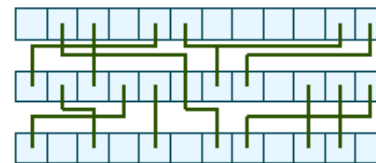
*Technology
Dependent
Synthesis*

Topic 13 Physical Design Automation

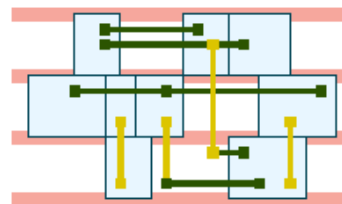
Placement



*Global
Routing*



*Detailed
Routing*



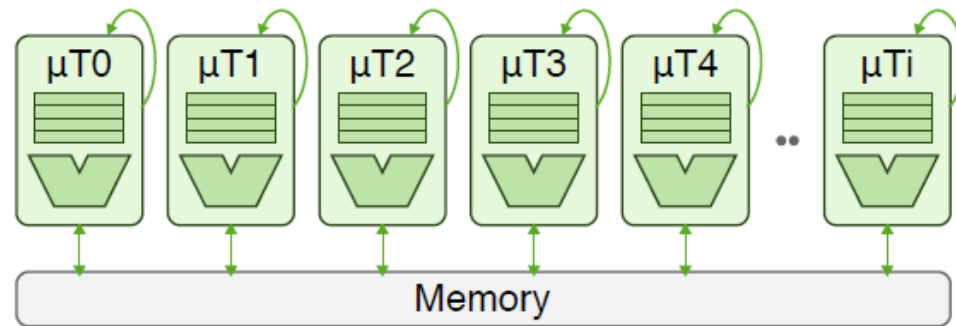
這個課程

- 不只是一門課
- 而且是一份課程地圖

像是計算機結構的觀念

Scalar Processors with Multithreading

Programmer's
Logical
View



Vector-Vector Add

```
loop:
  load  a, a_ptr
  load  b, b_ptr
  add   c, a, b
  store c, c_ptr
  a_ptr++
  b_ptr++
  c_ptr++
  branch
```

Masked Filter

```
loop:
  load a, a_ptr
  a_ptr++
  branch a = 0
  op0
  op1
  ...
  branch
```

硬體描述語言的觀念

SystemVerilog: Typed Ports and Type Parameters

```
// Structs, unions, tagged unions    // Type parameter allows more
// can be used as ports              // expressive polymorphism

module InstrDecodeTable
(
    input  Instr      instr,
    output ControlSigs cs
)

    // Use structure selectors
    // to access instruction
    // and control signal fields

endmodule

module Queue
#(
    parameter type ItemType
) (
    input  clk, reset

    input          enq_val,
    output          enq_rdy,
    input  ItemType enq_item,

    output          deq_val,
    input           deq_rdy,
    output  ItemType deq_bits,
)

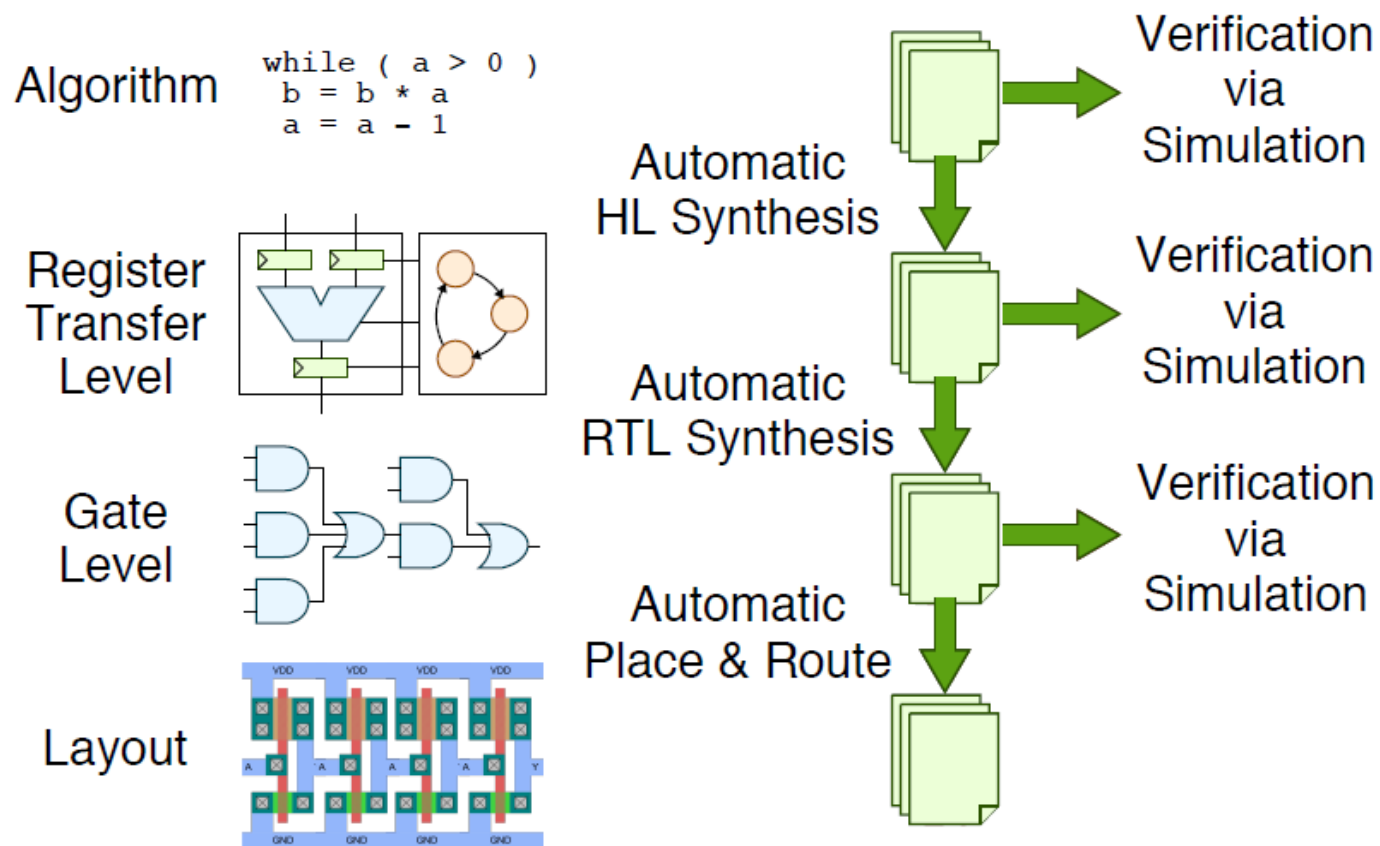
    // Use \ $bits for size of item

endmodule

// Instantiate polymorphic queue
Queue#(Instr) queue( ... )
```

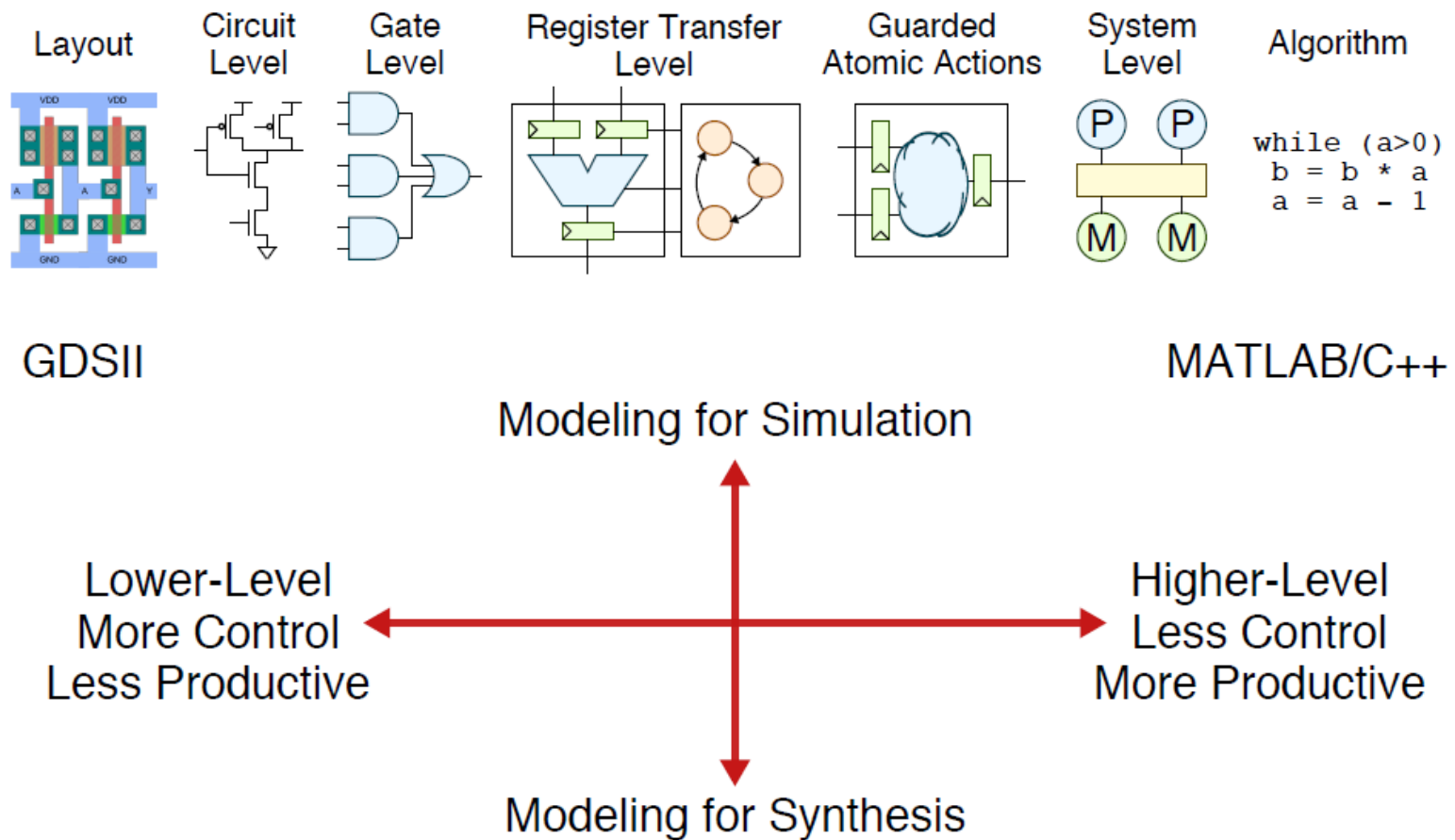
還有從演算法貫穿到半導體製程等等觀念

**Once designs were written in HDLs tools
could be used for automatic translation**



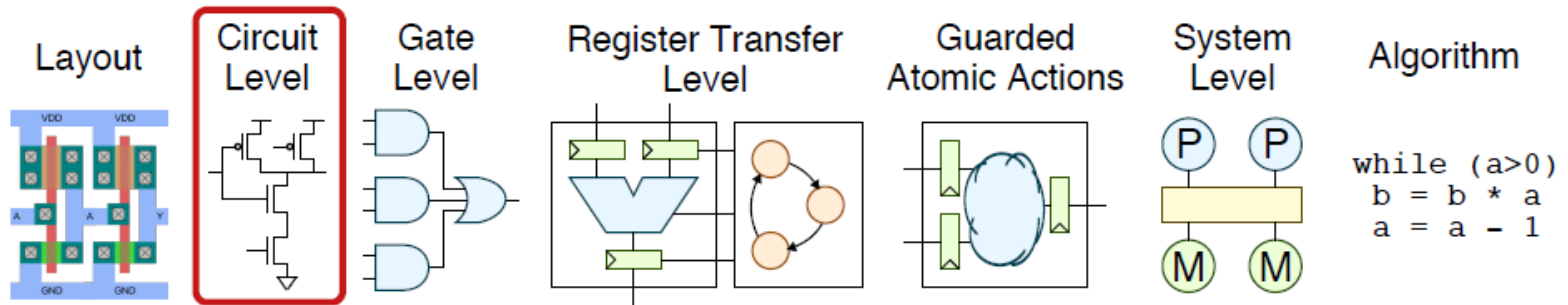
都可以從中學學習到

HDLs Across The Computer Engineering Stack



從 Spice 類比電路

Circuit-Level Modeling with Spice



* CMOS NAND gate

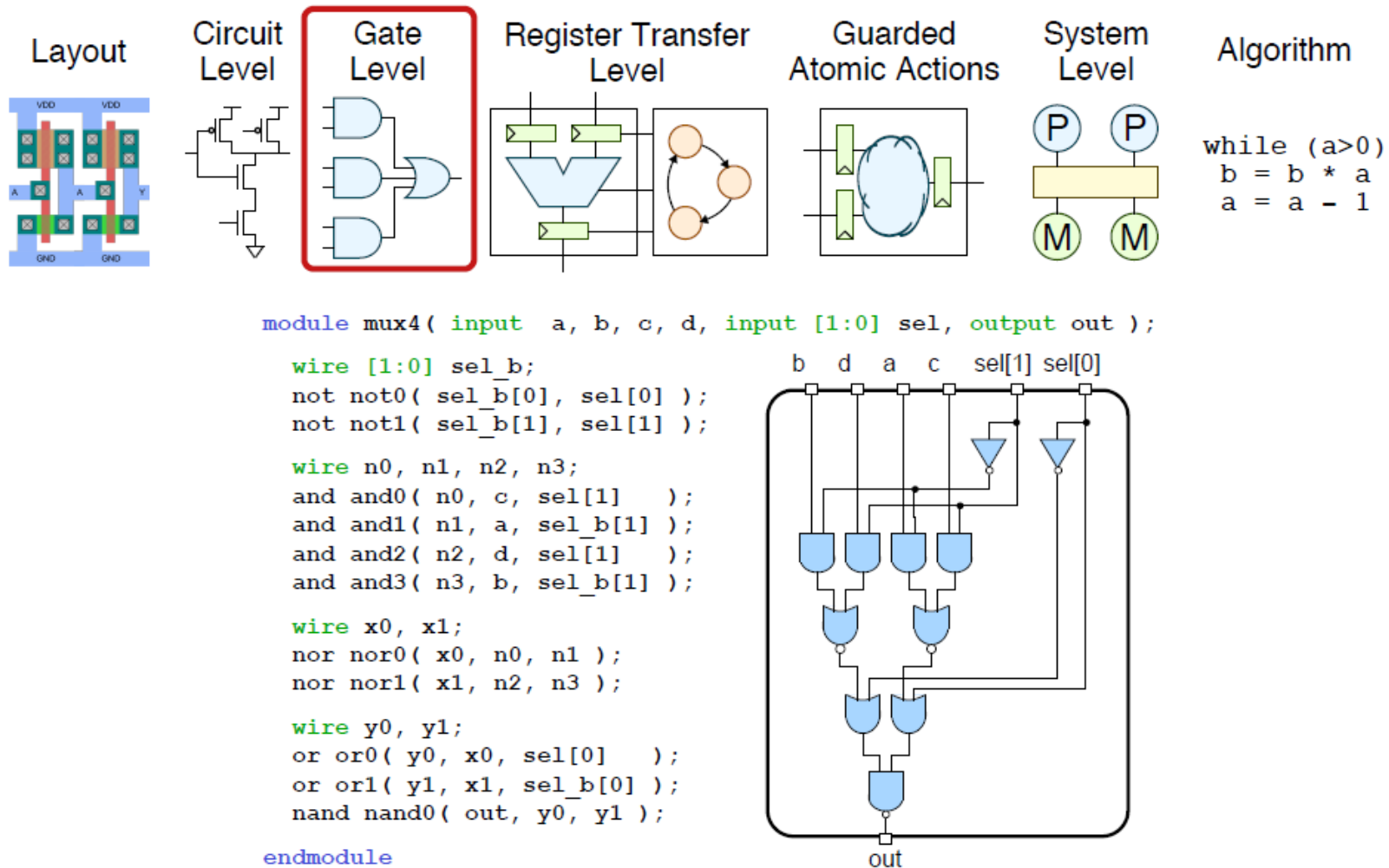
```
MP1 4 1 3 3 CMOSP W=28.0U L=2.0U AS=252P AD=252P
MP2 4 2 3 3 CMOSP W=28.0U L=2.0U AS=252P AD=252P
MN1 4 1 5 0 CMOSN W=10.0U L=2.0U AS=90P AD=90P
MN2 5 2 0 0 CMOSN W=10.0U L=2.0U AS=90P AD=90P
```

* Input stimulus

```
VINA 2 0 PULSE(0 5 100ns 5ns 5ns 100n 200ns)
VINB 1 0 PULSE(0 5 205ns 5ns 5ns 200n 400ns)
VDD 3 0 DC 5.0
```


到 Verilog 數位電路

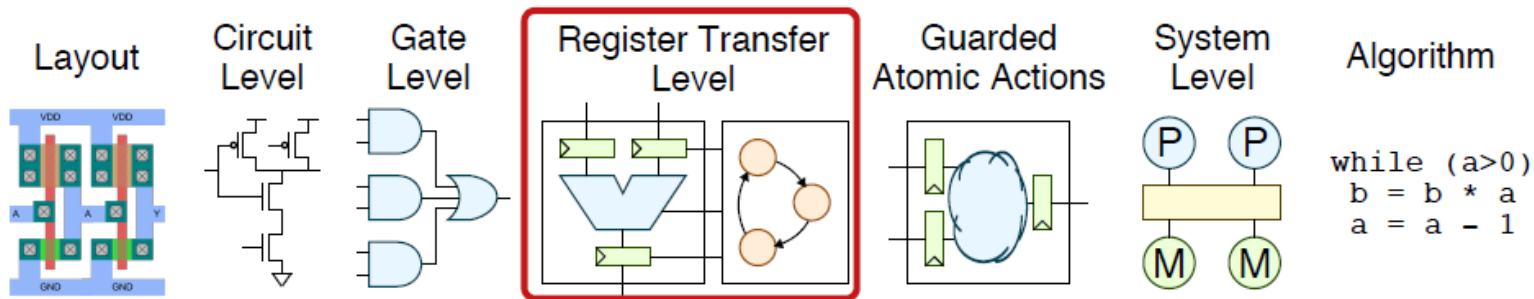
Gate-Level Modeling with Verilog



作者 Christopher Batten

- 都用幾張投影片，輕鬆地勾勒出技術的樣貌

“Low-Level” RTL Modeling with Verilog



// Combinational Logic: Operand Muxes

```
wire [63:0] a_mux_out
= ( a_mux_sel == op_load ) ? { 32'b0, unsigned_a }
: ( a_mux_sel == op_next ) ? a_shift_out
: 64'bx;
```

```
wire [31:0] b_mux_out
= ( b_mux_sel == op_load ) ? unsigned_b
: ( b_mux_sel == op_next ) ? b_shift_out
: 32'bx;
```

```
reg [4:0] counter_reg;
reg      sign_reg;
reg [63:0] a_reg;
reg [31:0] b_reg;
reg [63:0] result_reg;
```

// Sequential State

```
always @ ( posedge clk ) begin
  if ( sign_en ) begin
    sign_reg <= sign_next;
  end

  if ( result_en ) begin
    result_reg <= result_mux_out;
  end

  counter_reg <= counter_mux_out;
  a_reg <= a_mux_out;
  b_reg <= b_mux_out;
end
```

從演算法

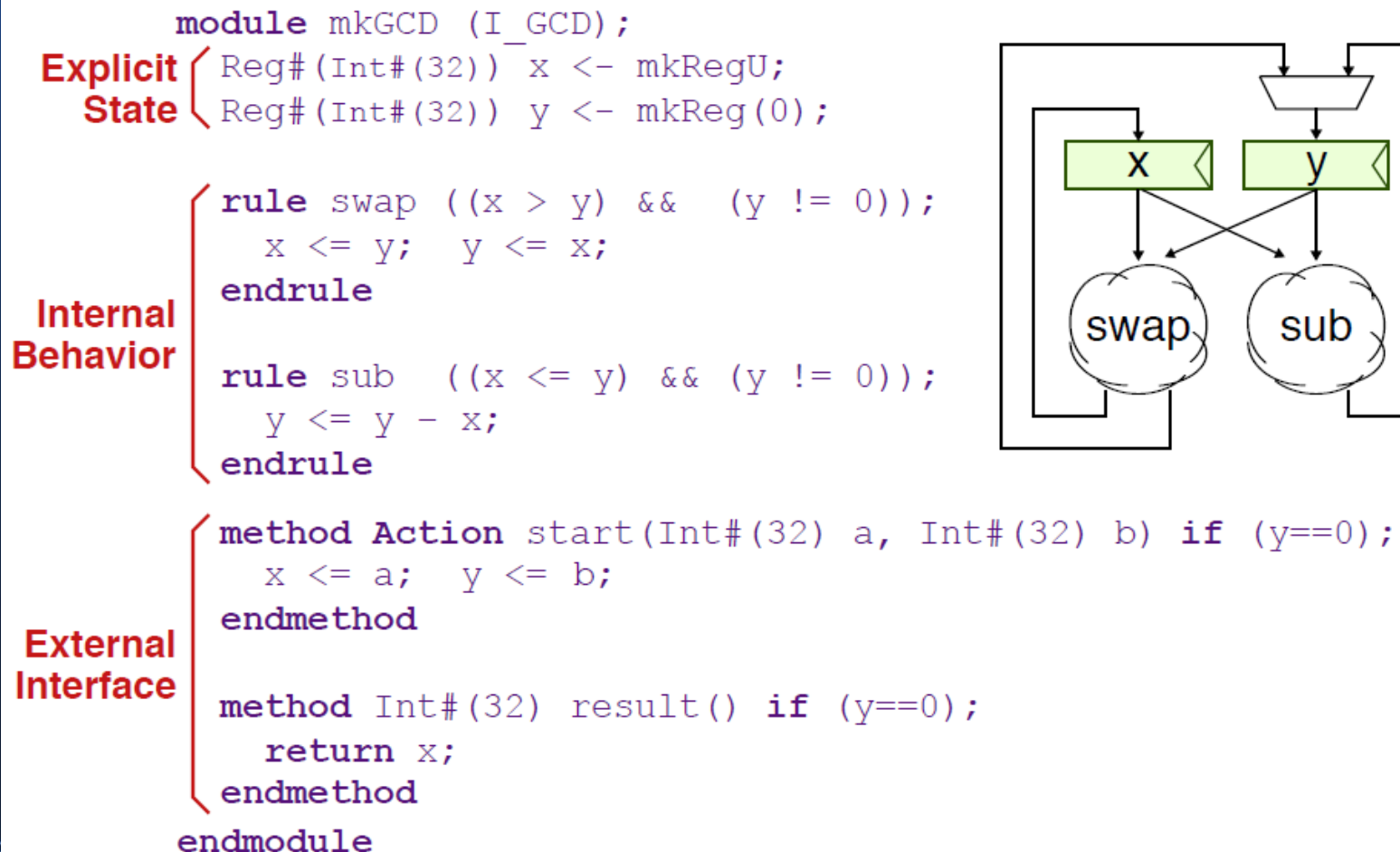
GCD Using Euclid's Algorithm

```
def gcd( x, y ):  
    while True:  
        if x > y:  
            x,y = y,x  
        elif y != 0:  
            y = y - x  
        else:  
            return x
```

	x	y	op
1.	25	15	swap
2.	15	25	sub
3.	15	10	swap
4.	10	15	sub
5.	10	5	swap
6.	5	10	sub
7.	5	5	sub
8.	5	0	return x

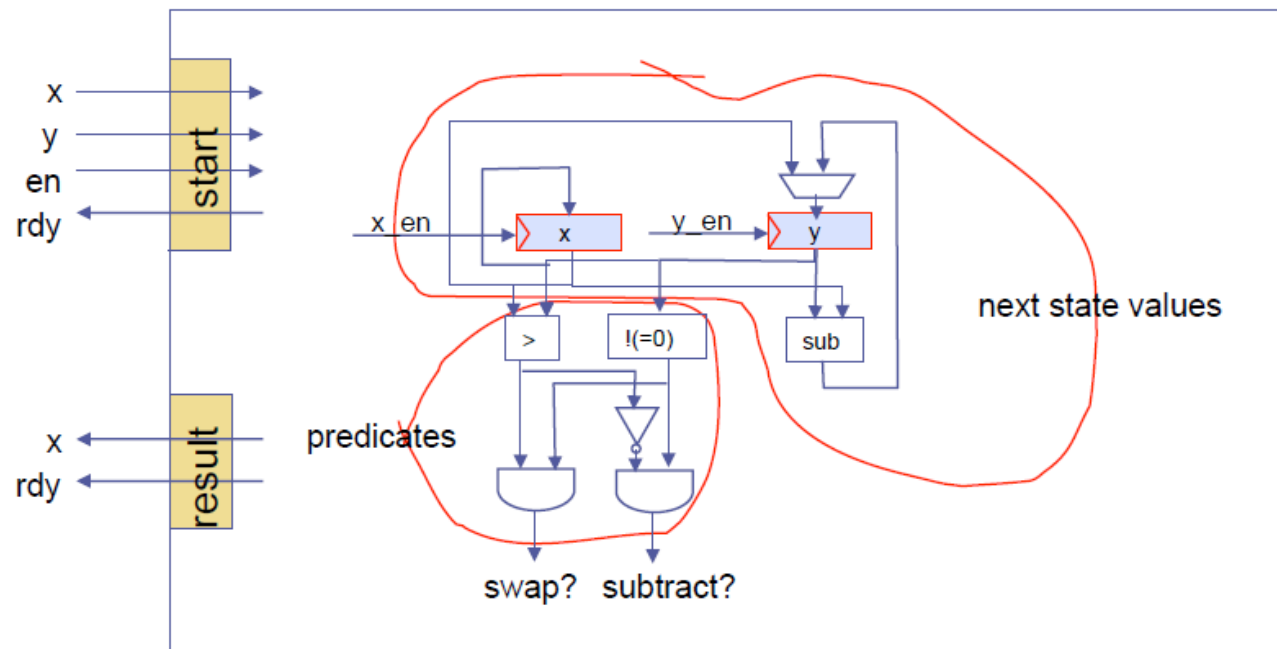
到硬體描述語言

GCD Implementation in Bluespec



然後到高階電路

Generated GCD Hardware: Rules



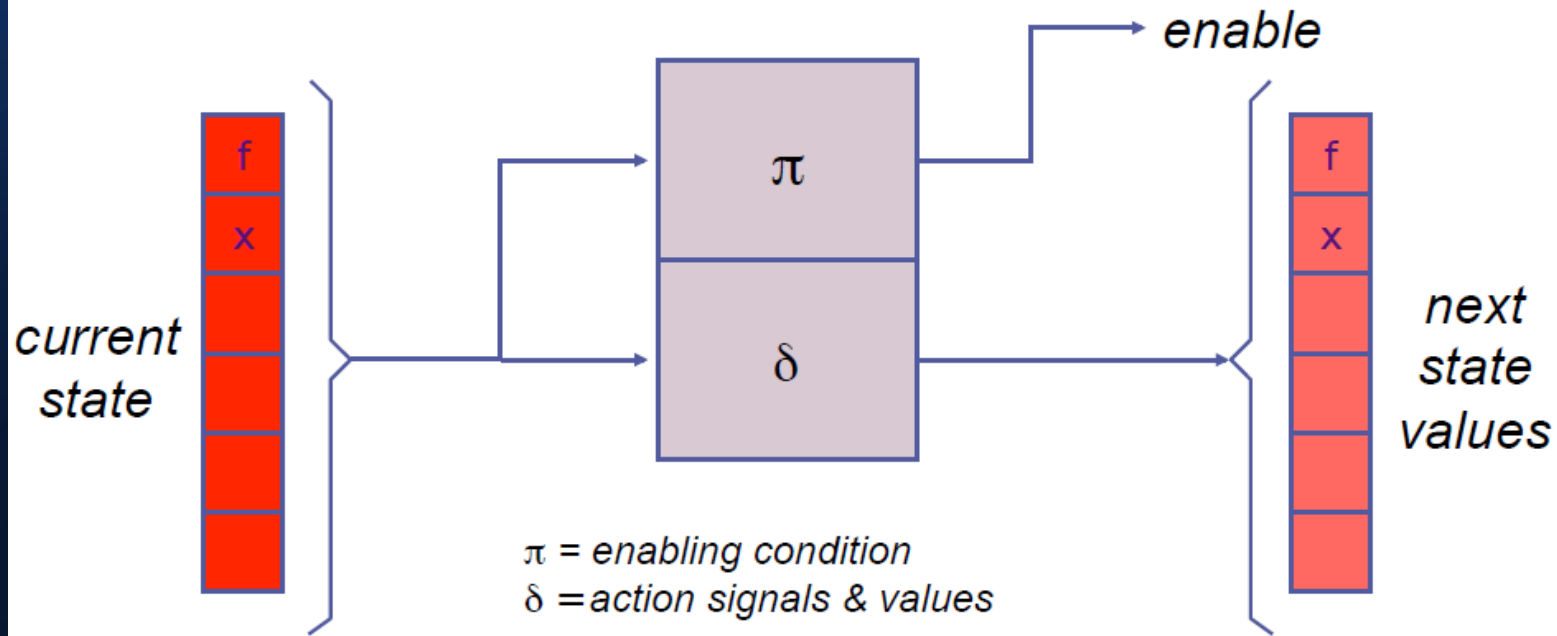
```
rule swap ((x>y) && (y!=0));  
    x <= y; y <= x; endrule  
rule subtract ((x<=y) && (y!=0));  
    y <= y - x; endrule
```

$x_en = \text{swap?}$
 $y_en = \text{swap? OR subtract?}$

再到元件的數學描述

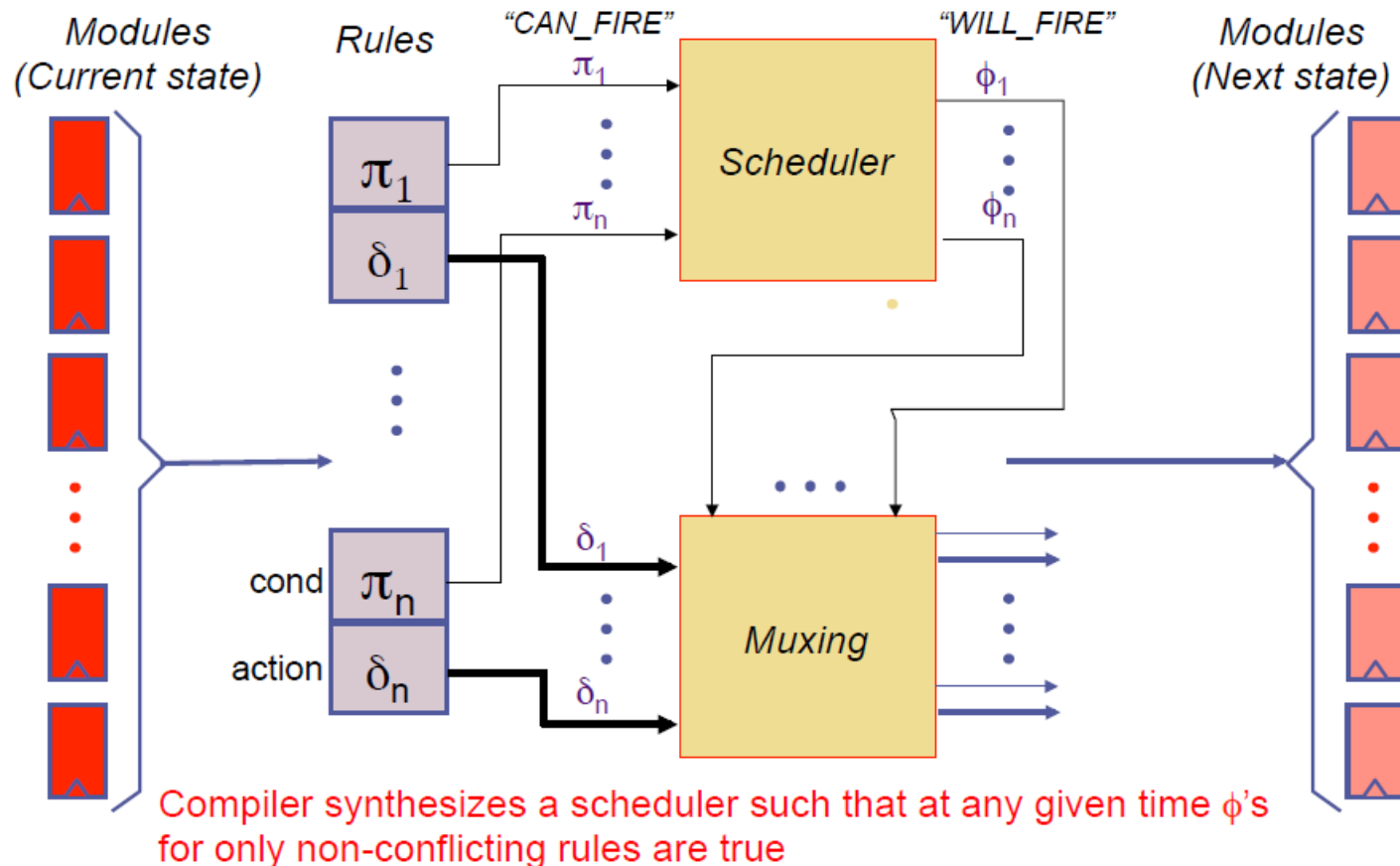
Compiling a Rule

```
rule r (f.first() > 0) ;  
    x <= x + 1 ; f.deq () ;  
endrule
```



組合出一幅幅美麗的技术圖像

Scheduling and Control Logic



軟體與硬體

- 交錯融合於其中

SystemC Producer-Consumer Example

```
struct Producer : public sc_module
{
    // Ports
    sc_out<bool> clk;
    sc_out<int> value;

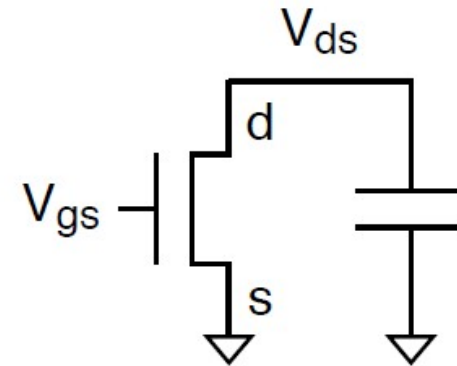
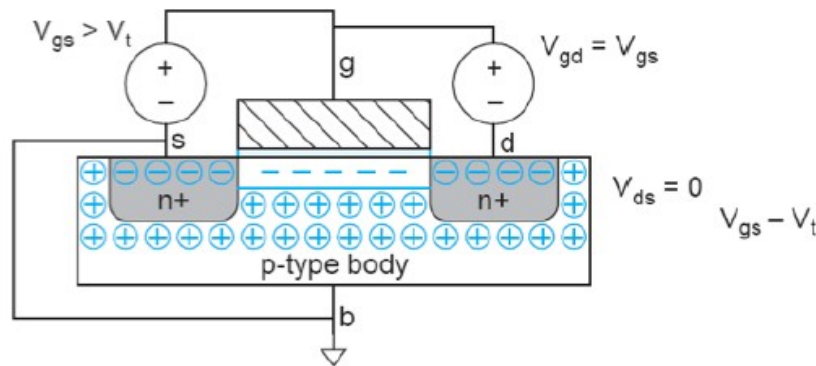
    SC_HAS_PROCESS(Producer);
    Producer( sc_module_name name ) : sc_module(name)
    {
        // Declares compute as a thread
        SC_THREAD(compute);
    }

    void compute()
    {
        for ( int i = 0; i < 10; ++i ) {
            clk.write(false);    // toggle clk
            wait( 5, SC_NS );    // wait for 5 nanoseconds
            clk.write(true);     // toggle clk
            value.write(i);      // write value to channel
            wait( 5, SC_NS );    // wait for 5 nanoseconds
        }
    }
};
```

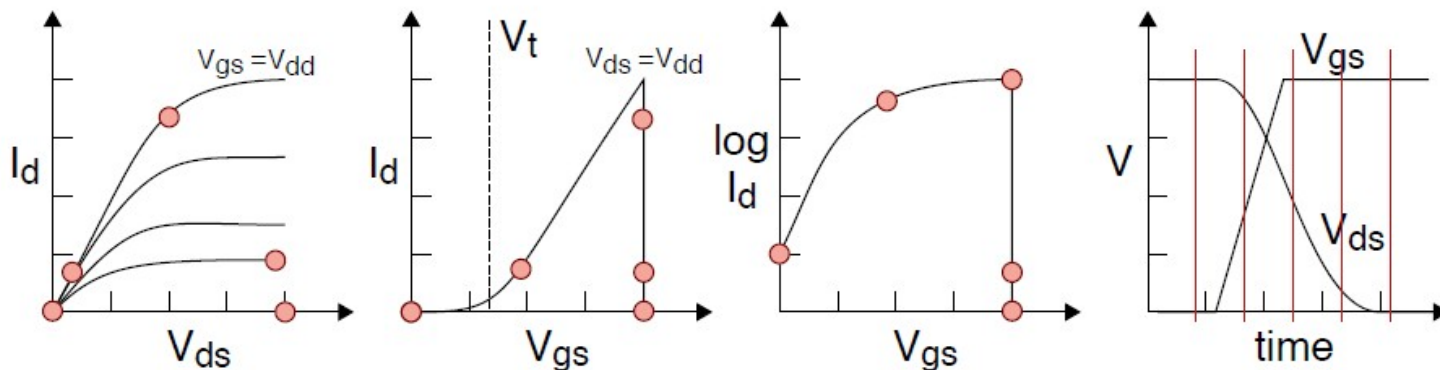

底層的 CMOS

- 是台積電專長的半導體領域

Simple NMOS Circuit



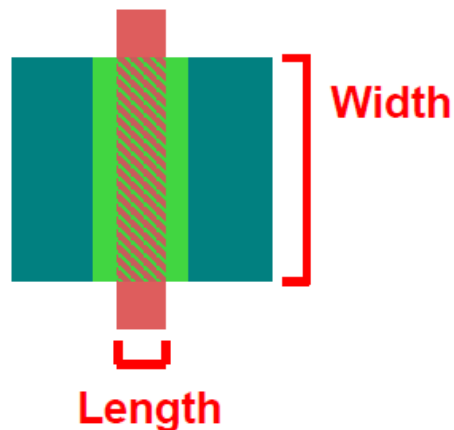
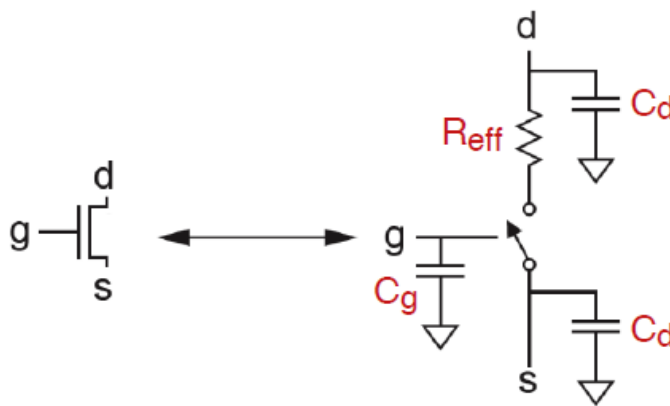
Linear: Channel Formed
 I_{ds} increases with V_{ds}



其中的電路原理

- 是電子學課程的核心

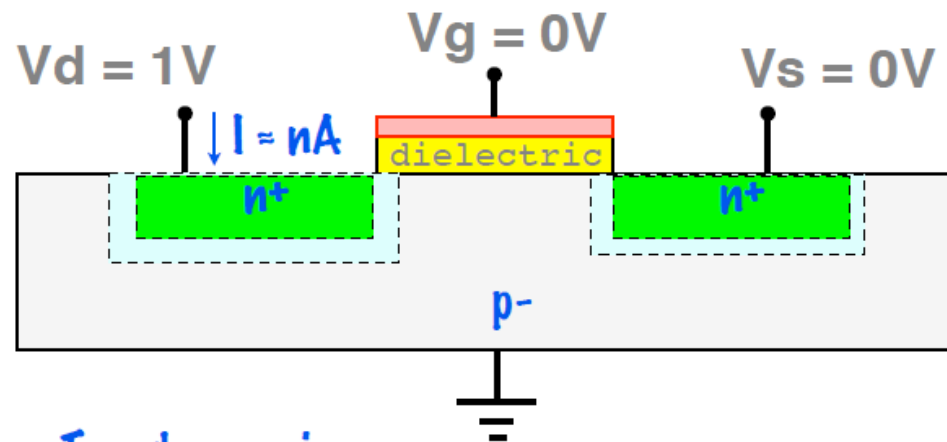
Key Qualitative Characteristics of MOSFET Transistors



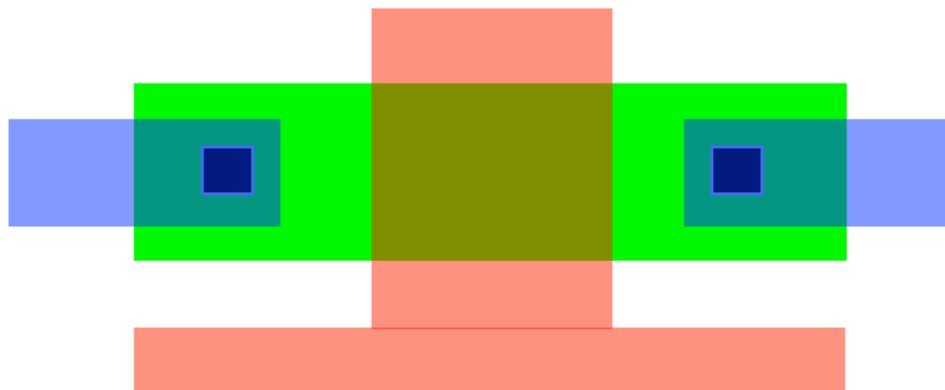
- ▶ V_t sets when transistor turns on, impacts leakage current
- ▶ $I_d \propto \mu \times (W/L)$
- ▶ $\mu_n > \mu_p \implies R_{N,eff} < R_{P,eff}$
- ▶ $C_g \propto (W \times L)$
- ▶ $C_d \propto W$
- ▶ $\uparrow W = \downarrow R_{eff} = \uparrow I_d = \uparrow C_d, C_g$
- ▶ $\uparrow L = \uparrow R_{eff} = \downarrow I_d = \uparrow C_g$

台灣的電子產業

Mask Set for NMOS Transistor (circa 1986)



Top-down view:



Masks

- #1: n^+ diffusion
- #2: poly (gate)
- #3: diff contact
- #4: metal

Layers to do
p-Fet not shown.

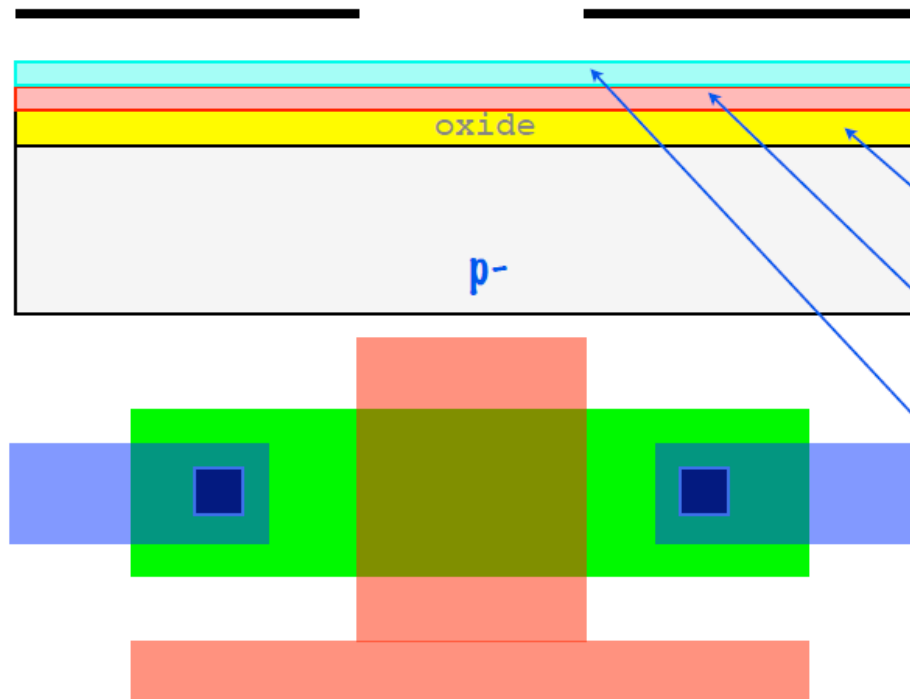
Modern
processes have 6
to 10 metal
layers (or more)
(in 1986: 2).

和這些圖有密切關聯

Start With an Un-Doped Wafer



UV hardens exposed resist. A wafer wash leaves only hard resist.

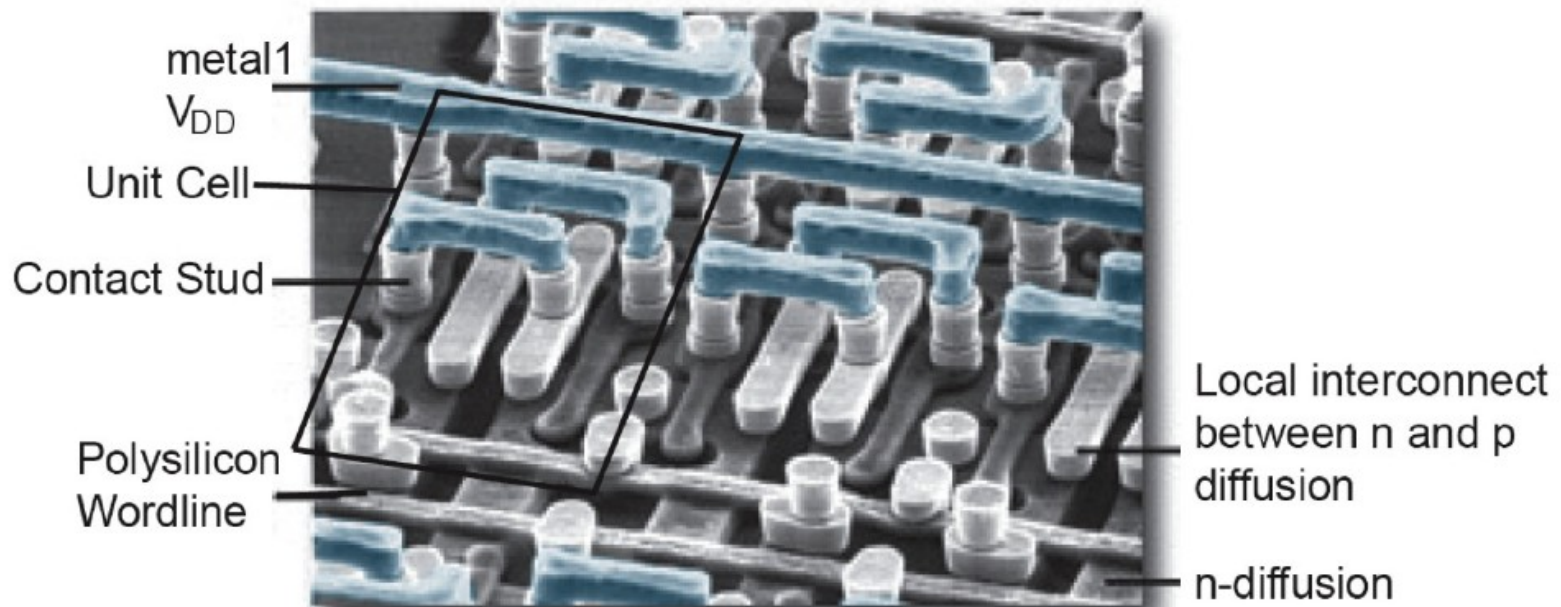


Steps

- #1: dope wafer p-
- #2: grow gate oxide
- #3: deposit undoped polysilicon
- #4: spin on photoresist
- #5: place positive poly mask and expose with UV.

顯微鏡底下的奈米世界

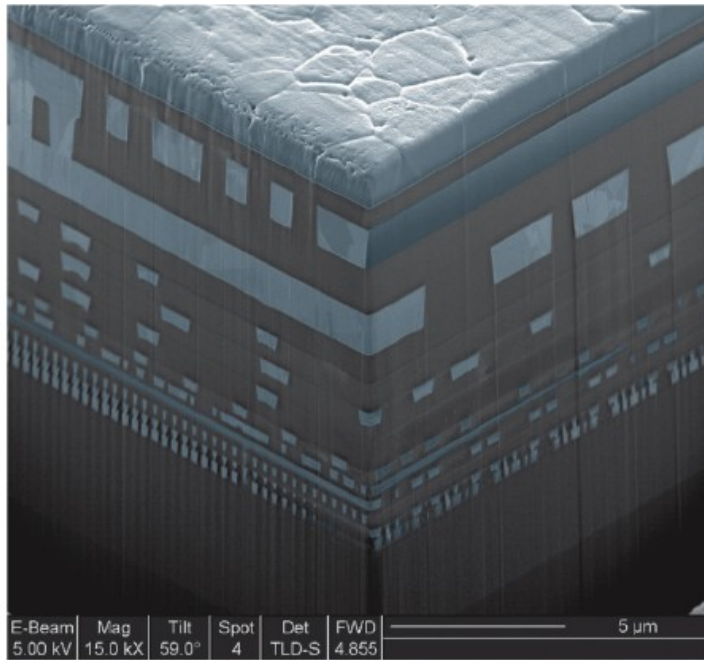
Local Interconnect



IBM 6-Transistor SRAM Cell

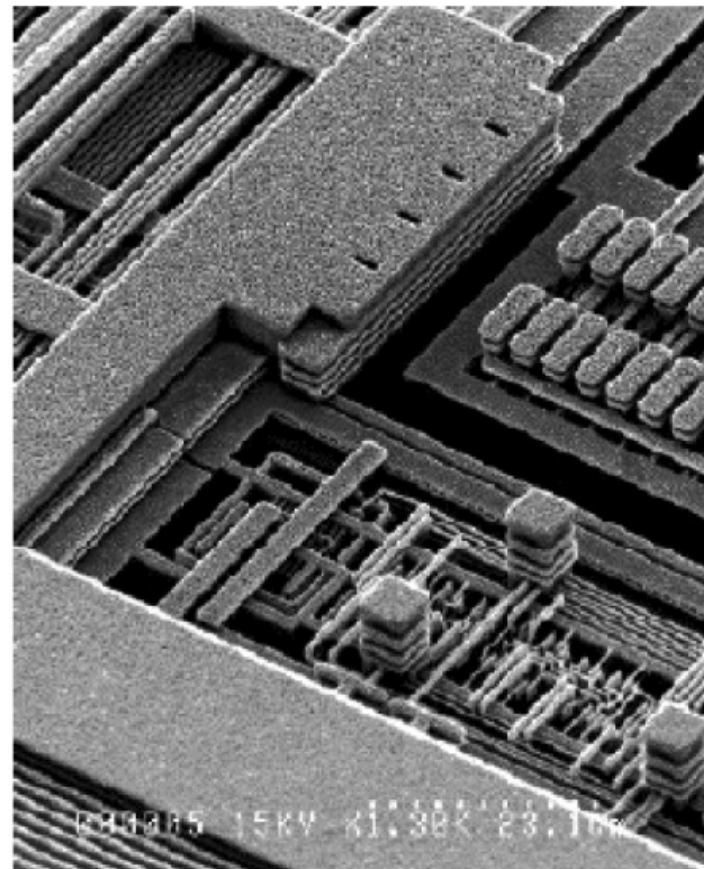
透露出整個產業的秘密

IBM Metal Stacks



IBM 11-layer Copper Metal Stack

IBM 6-layer Copper Metal Stack

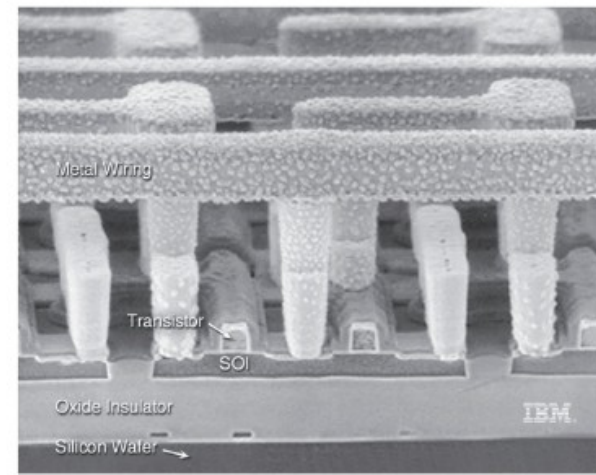
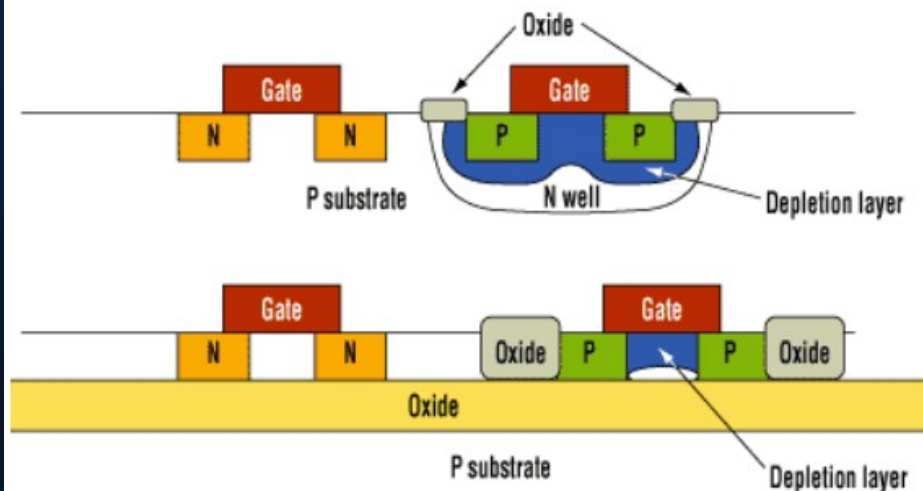


掌握這些知識

- 對理解 IC 產業有很大的幫助

Bulk vs. Silicon-on-Insulator Processing

- ▶ Eliminates parasitic capacitance between source/drain and the body → lower energy, higher performance
- ▶ Lower subthreshold leakage, but threshold voltage varies over time
- ▶ 10–15% increase in total manufacturing cost due to substrate cost



讓我們隨著課程

ECE 5745 Complex Digital ASIC Design **Topic 3: CMOS Circuits**

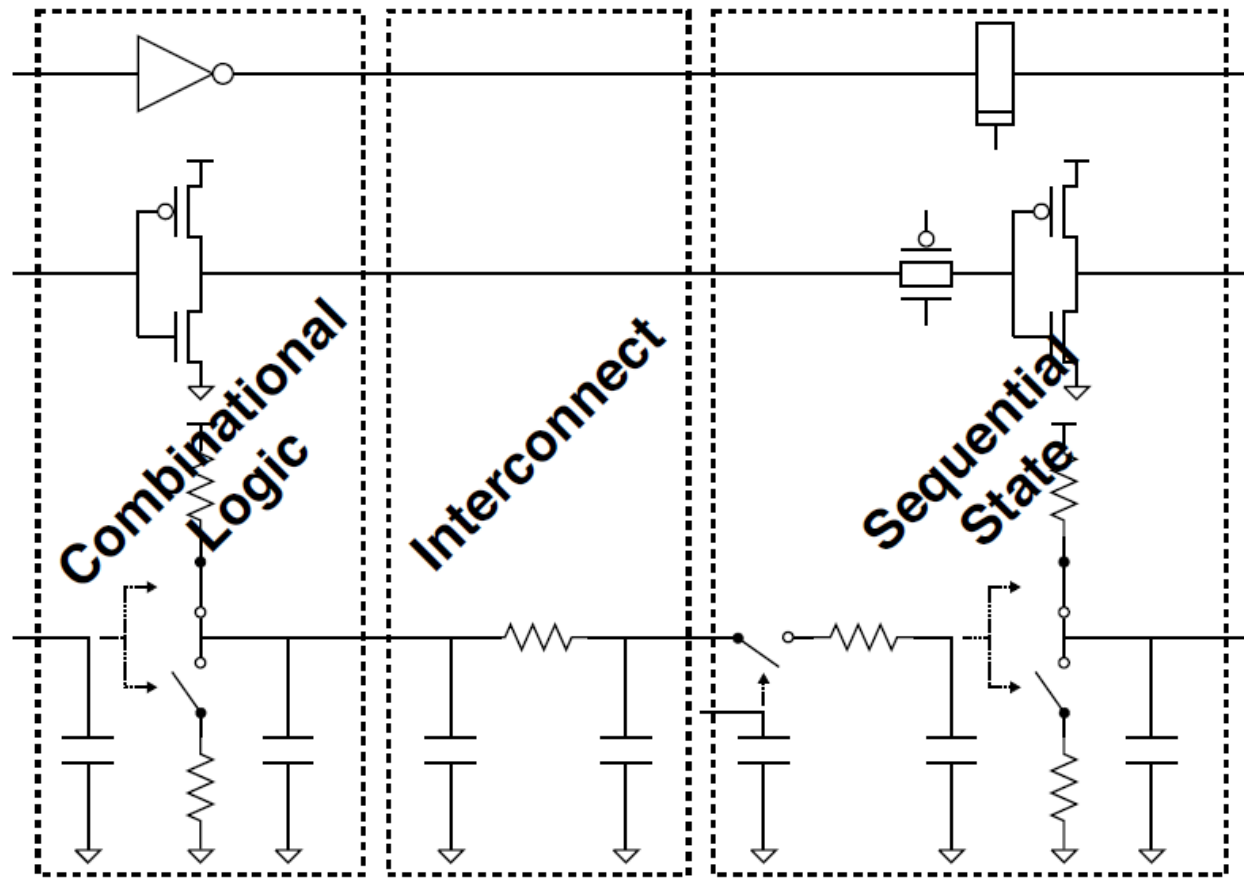
Christopher Batten

School of Electrical and Computer Engineering
Cornell University

<http://www.csl.cornell.edu/courses/ece5745>

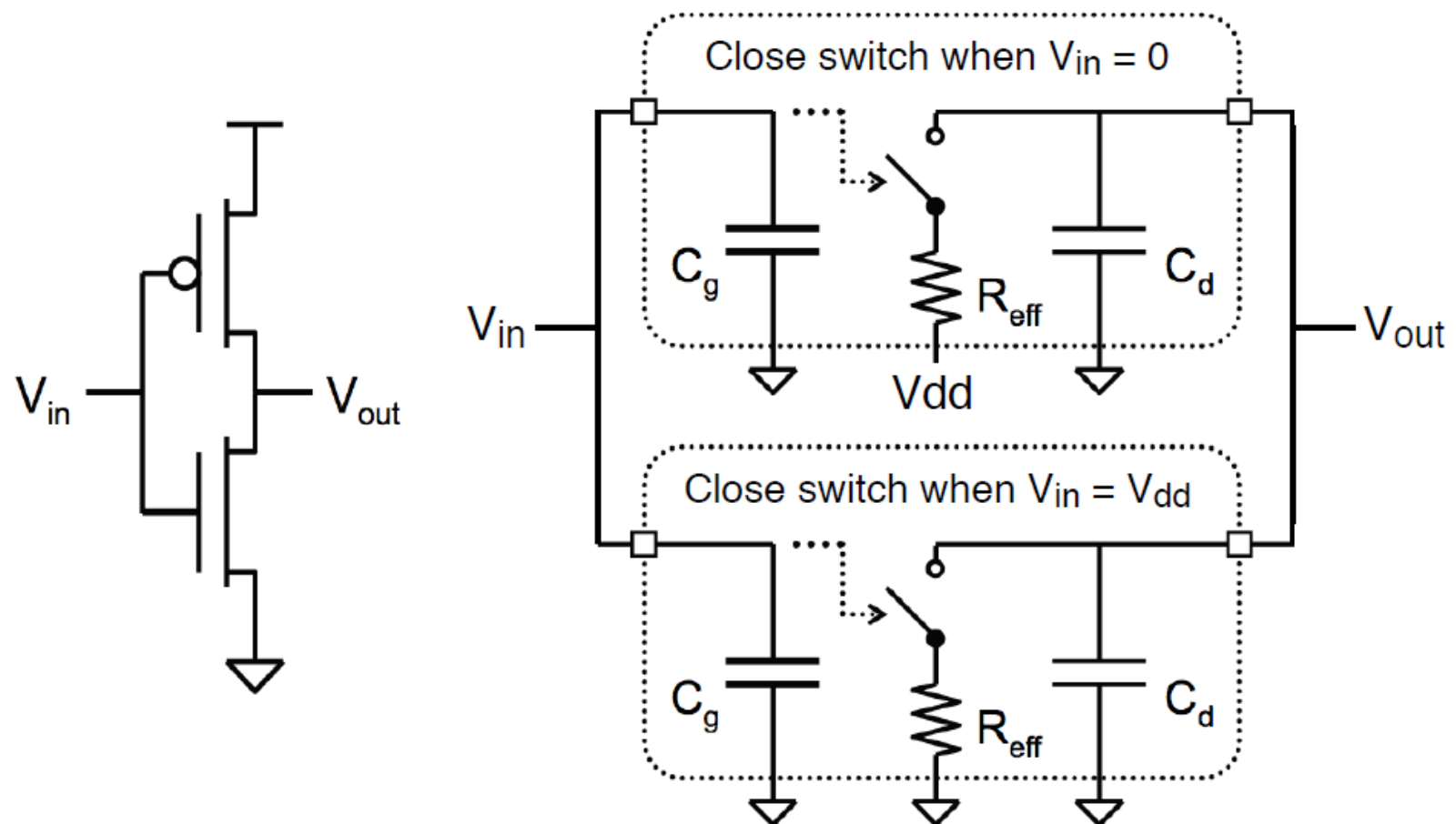
繼續進入 CMOS 的電子世界

CMOS Logic, State, Interconnect



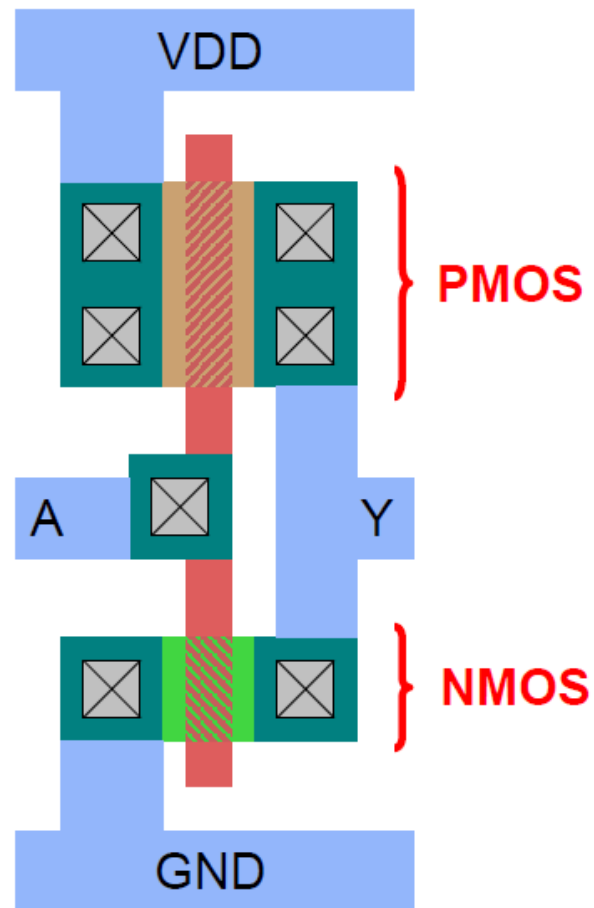
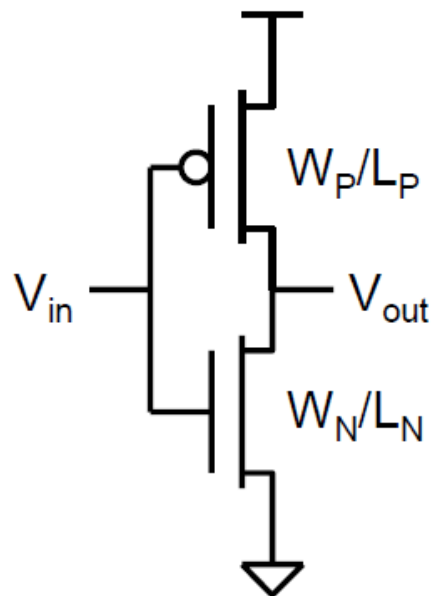
下圖是 CMOS 的 NOT 閘

CMOS Inverter Simple RC Model



NOT 閘在 IC 基板上布局

CMOS Inverter Layout

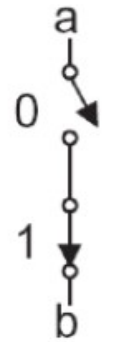


AND 和 NAND 閘

Series Transistors



OFF



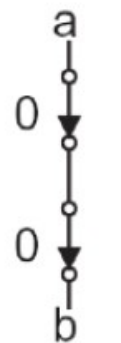
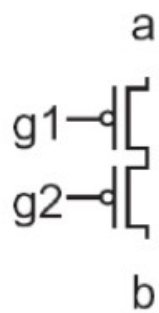
OFF



OFF



ON



ON



OFF



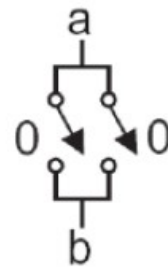
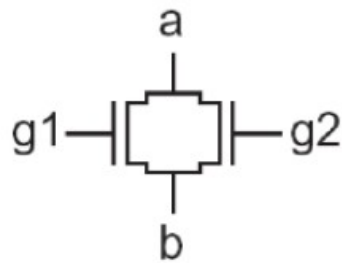
OFF



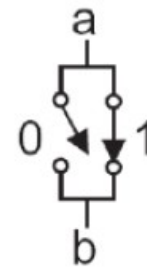
OFF

OR 和 NOR 閘

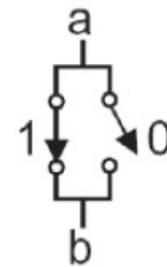
Parallel Transistors



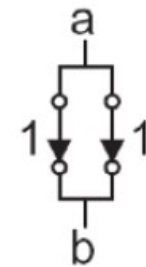
OFF



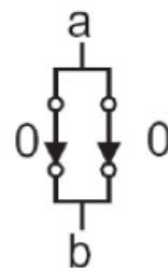
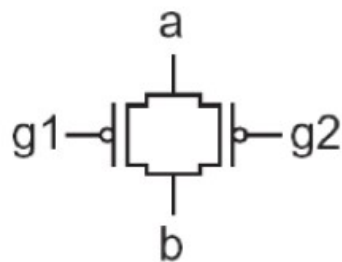
ON



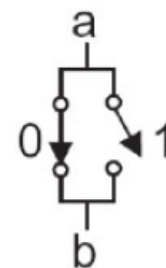
ON



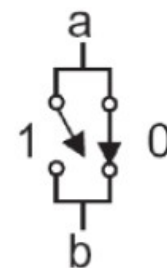
ON



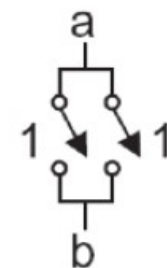
ON



ON



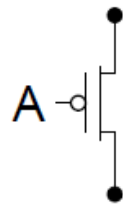
ON



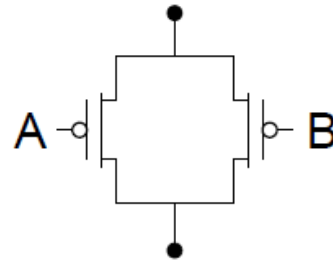
OFF

都可以透過 NMOS 和 PMOS 組合出來

Series/Parallel Transistor Networks are Natural Duals



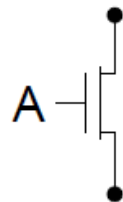
Conducts if $A=0$



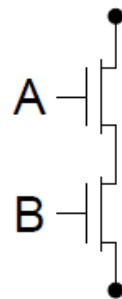
Conducts if $A=0$ **OR** $B=0$



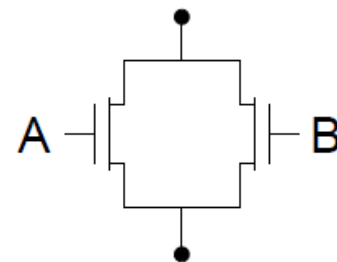
Conducts if $A=0$ **AND** $B=0$



Conducts if $A=1$



Conducts if $A=1$ **AND** $B=1$

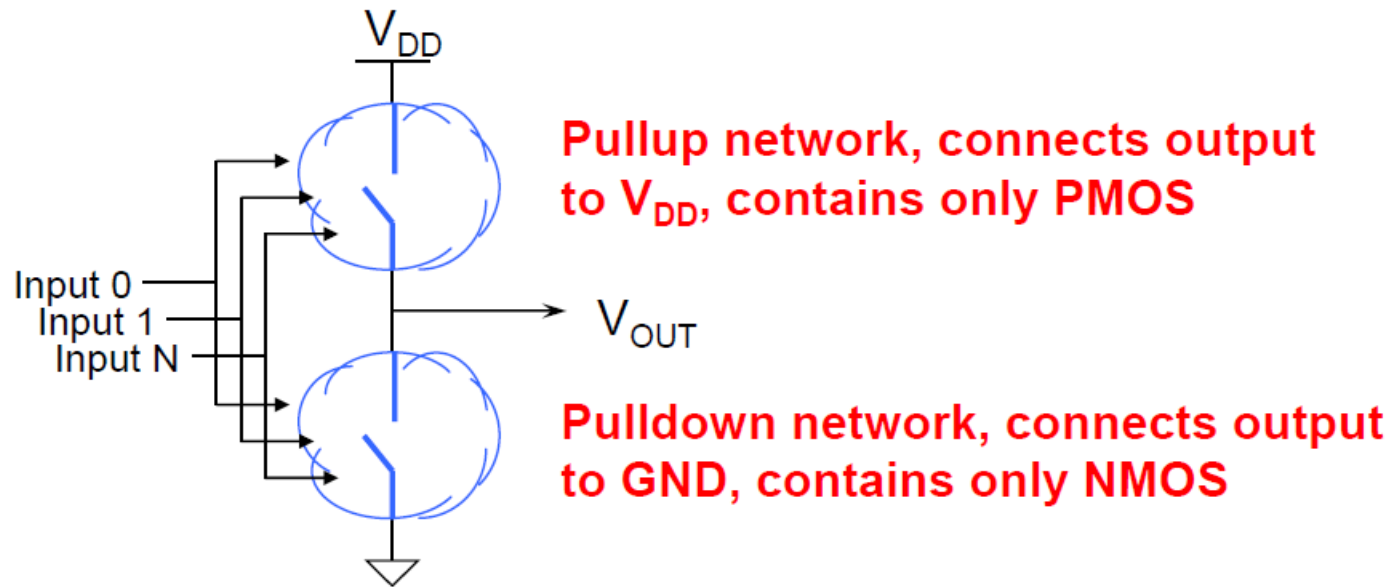


Conducts if $A=1$ **OR** $B=1$

這些 NMOS 和 PMOS

- 就像一堆開關，組成了《上拉與下拉網路》

CMOS Static Logic Style



For every set of input logic values, either pullup or pulldown network makes connection to V_{DD} or GND

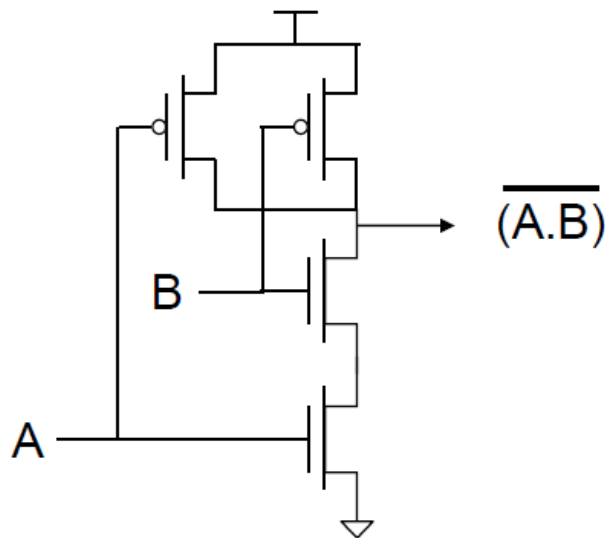
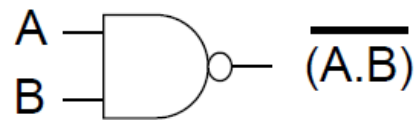
- If both connected, power rails would be shorted together
- If neither connected, output would float (tristate logic)

透過對稱的互補方式

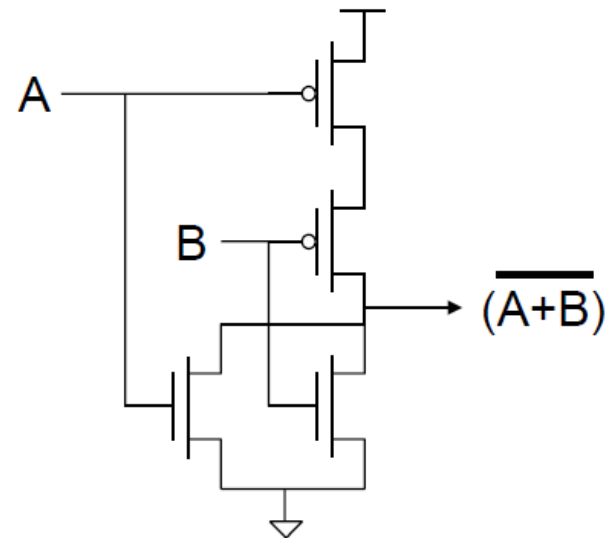
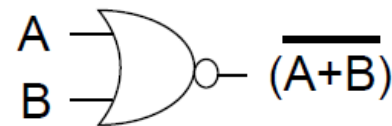
- 形成 NMOS+PMOS=CMOS 的架構，具有省電的特性

NAND/NOR Static CMOS Logic Gates

NAND Gate



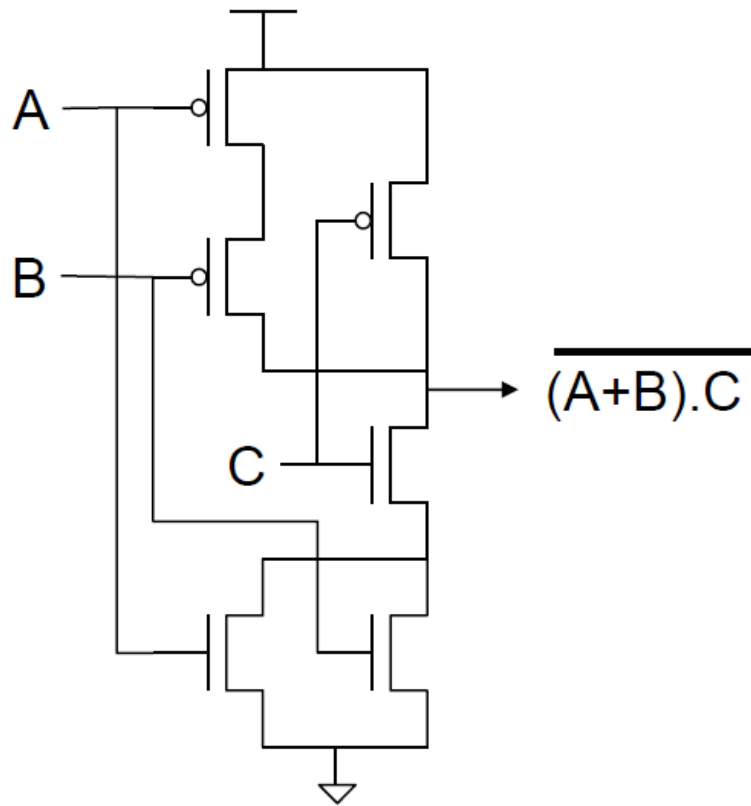
NOR Gate



只要給定邏輯式

- 就能用 EDA 自動排列 CMOS 電路

Complex Logic Gate Example



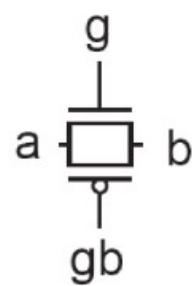
$$f = \overline{(A + B) \cdot C}$$

$$PD = (A + B) \cdot C$$

$$\begin{aligned} PU &= \overline{(A + B) \cdot C} \\ &= \overline{(A + B)} + \overline{C} \\ &= (\overline{A} \cdot \overline{B}) + \overline{C} \end{aligned}$$

多工器可以轉為 CMOS

CMOS Transmission Gate Multiplexer



$g = 0, gb = 1$



$g = 1, gb = 0$



Input

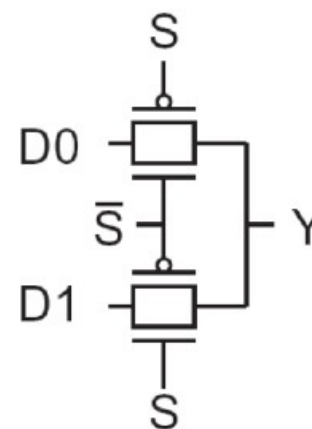
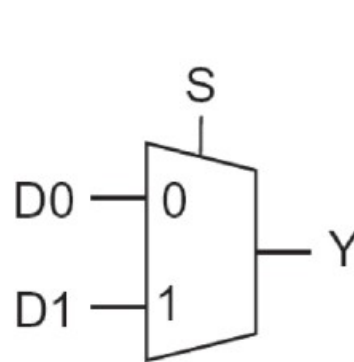
$g = 1, gb = 0$

$0 \rightarrow \text{strong } 0$

$g = 1, gb = 0$

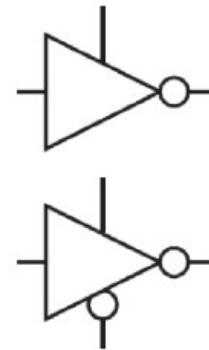
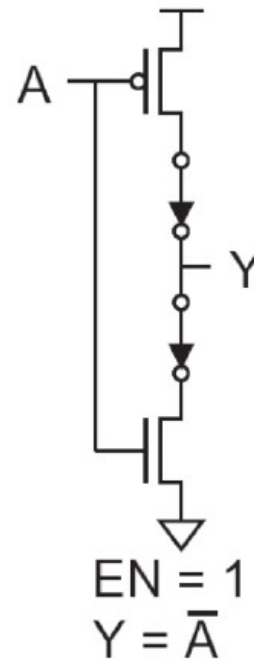
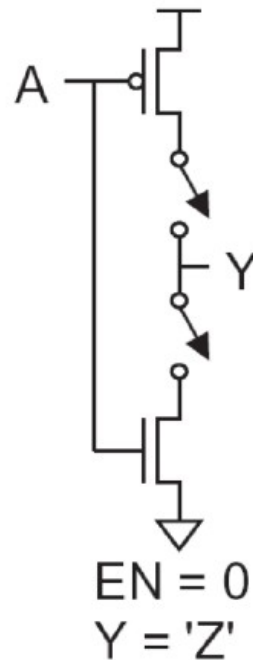
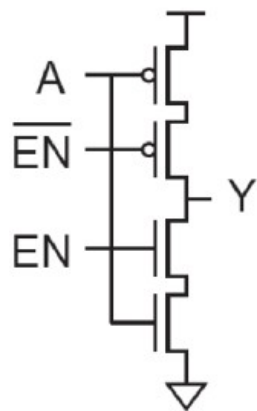
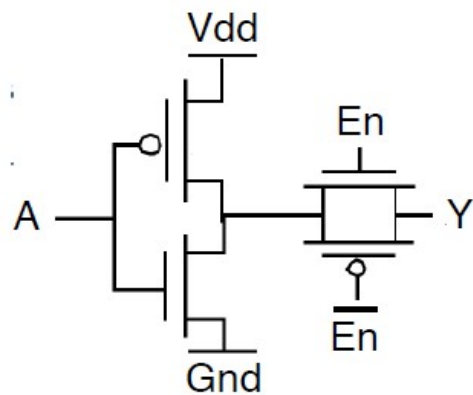
$1 \rightarrow \text{strong } 1$

Output

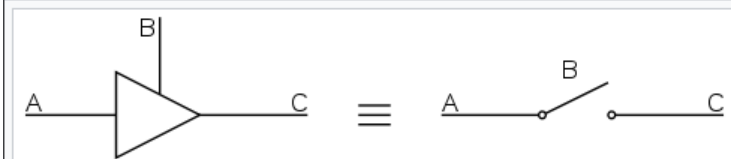


三態開關也很容易表達

CMOS Tri-State Buffers



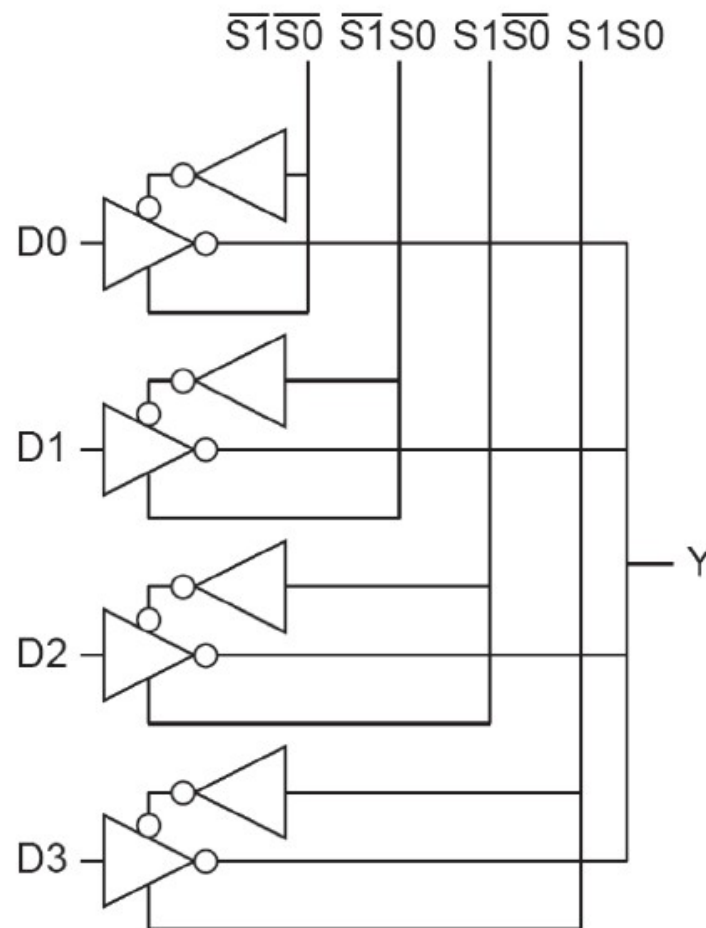
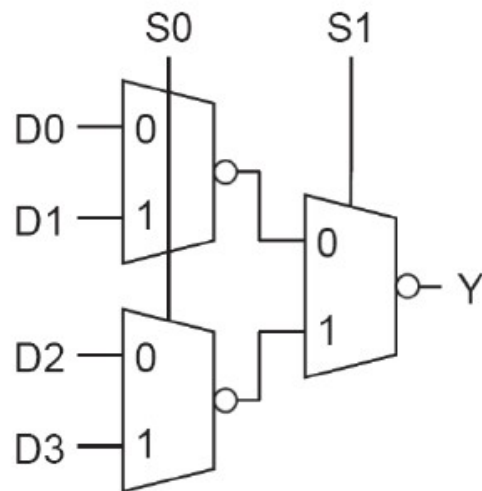
INPUT		OUTPUT
A	B	C
0	0	Z (high impedance)
1		Z (high impedance)
0	1	0
1		1



A tristate buffer can be thought of as a switch. If B is on, the switch is closed. If B is off, the switch is open.

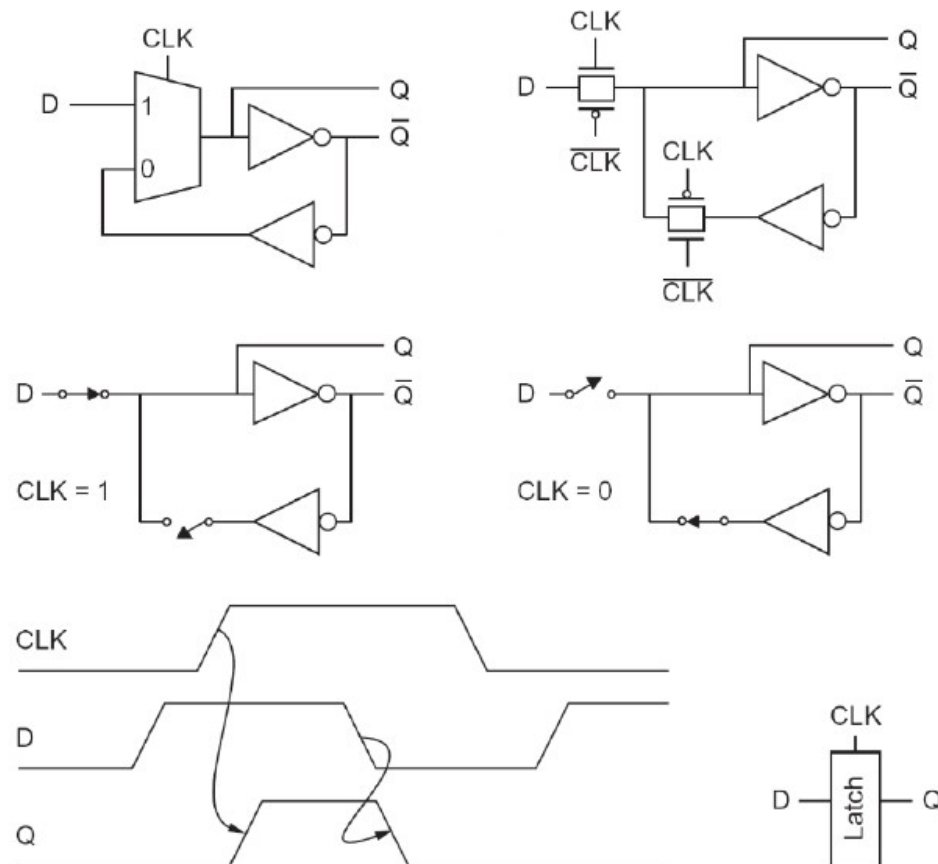
繼續往上組合也沒問題

Larger Tri-State Multiplexers



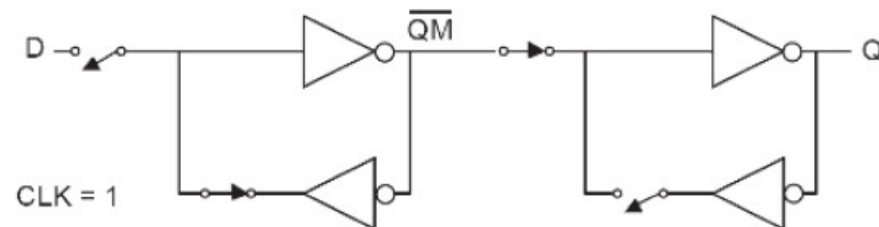
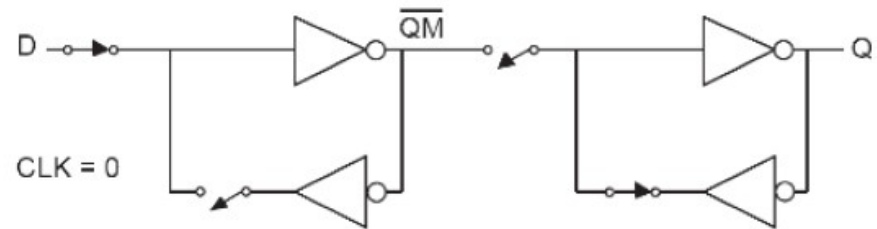
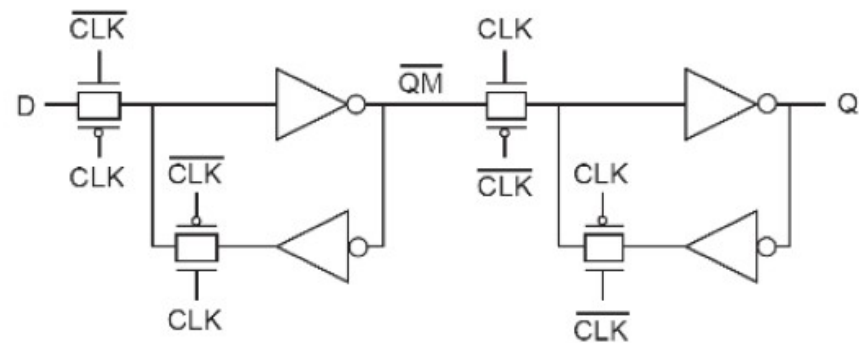
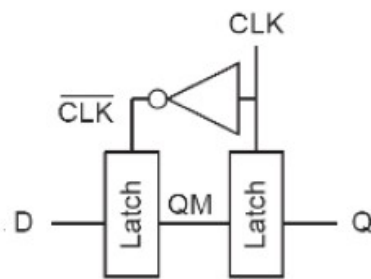
像是栓鎖器 Latch

Level-High Latch



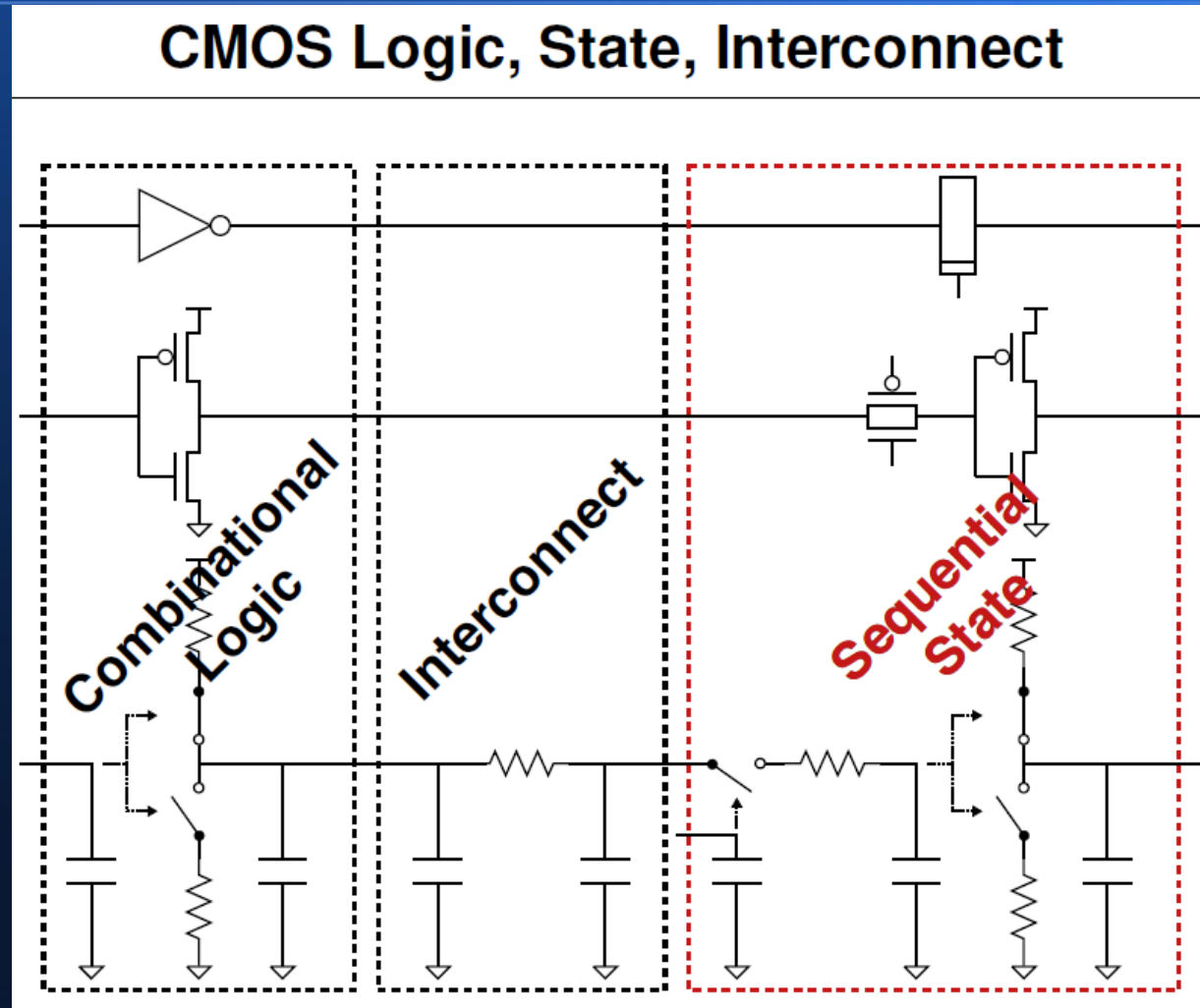
邊緣觸發正反器

Positive-Edge Triggered Flip-Flop



甚至更高階的數位電路

- 都可以透過 CMOS 組合出來

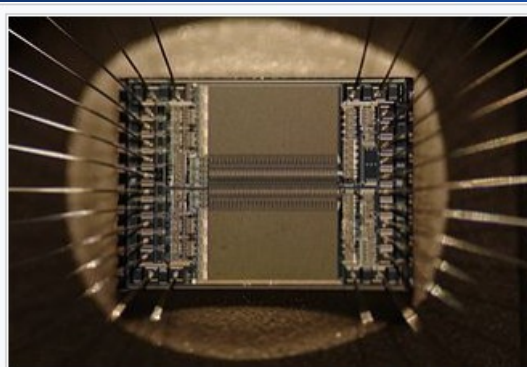


這些 CMOS 形成的積體電路

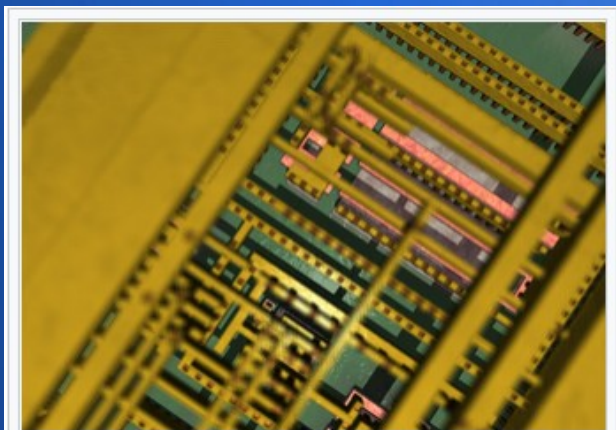
- 就稱為 IC



Erasable programmable read-only memory (EPROM) integrated circuits in dual in-line packages. These packages have a transparent window that shows the die inside. The window is used to erase the memory by exposing the chip to ultraviolet light.



Integrated circuit from an EPROM memory microchip showing the memory blocks, the supporting circuitry and the fine silver wires which connect the integrated circuit die to the legs of the packaging

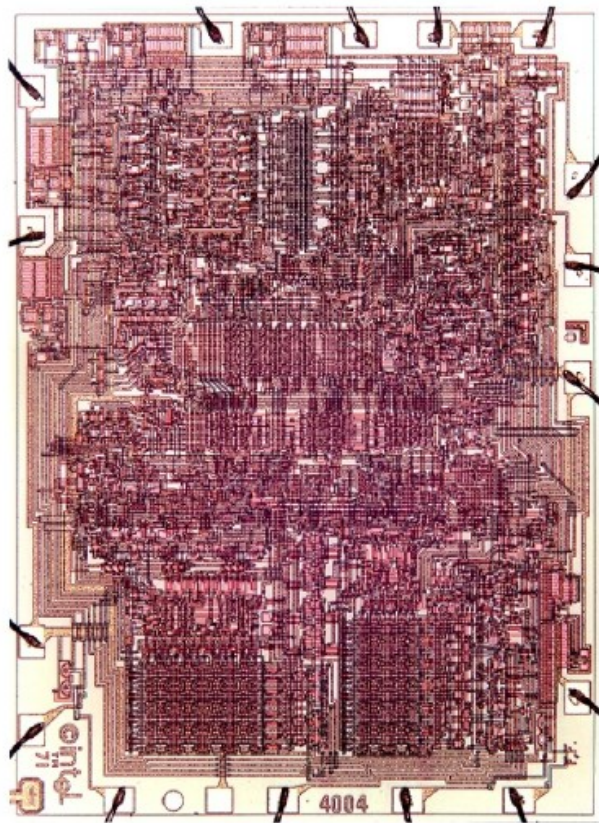


Virtual detail of an integrated circuit through four layers of planarized copper interconnect, down to the polysilicon (pink), wells (greyish), and substrate (green)

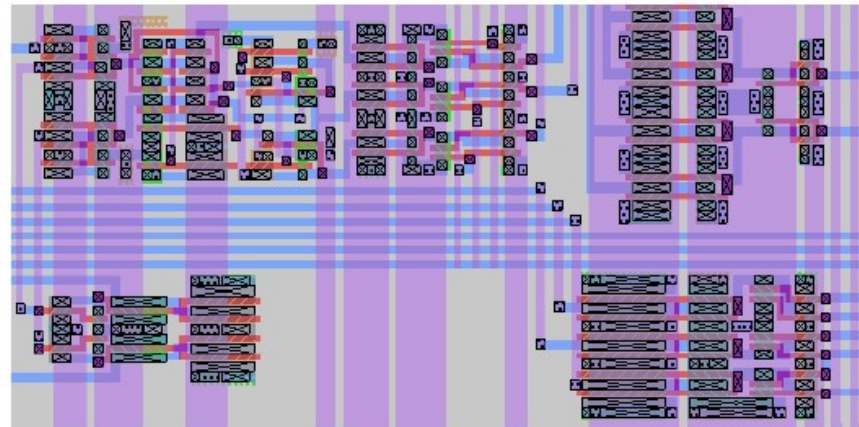
在 1960 年代

- 設計 IC 全靠人腦，你可以想像那有多複雜...

Full Custom Design



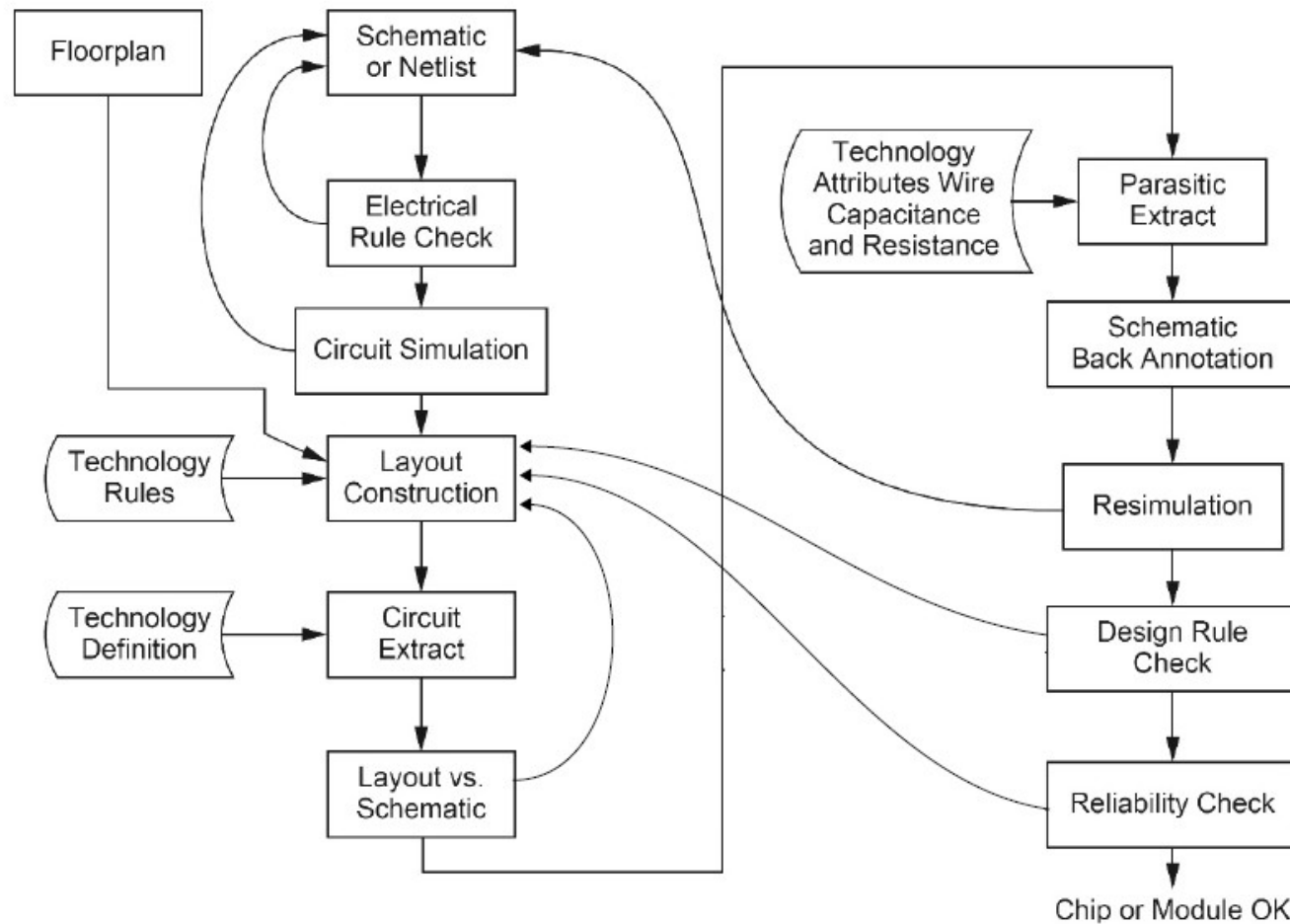
Intel 4004



Key is that all circuits and transistors are optimized for specific context

人腦設計 IC 的過程像這樣

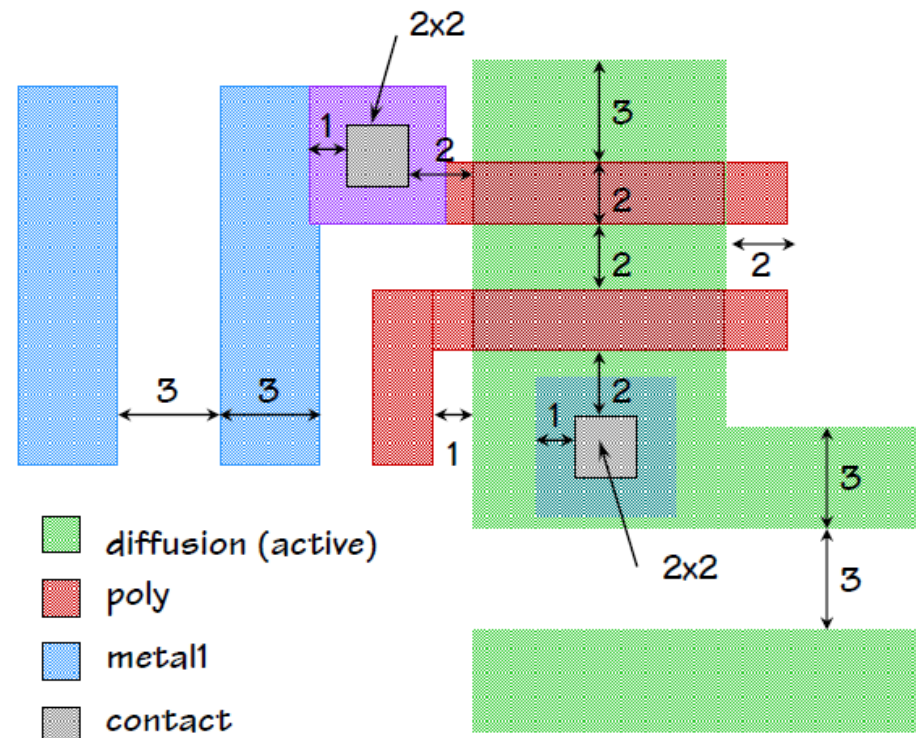
Overview of Full Custom Design Methodology



雖然有些法則可循

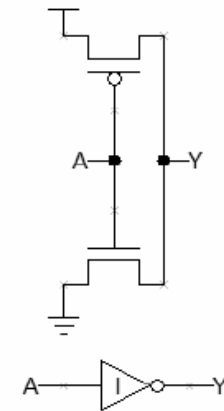
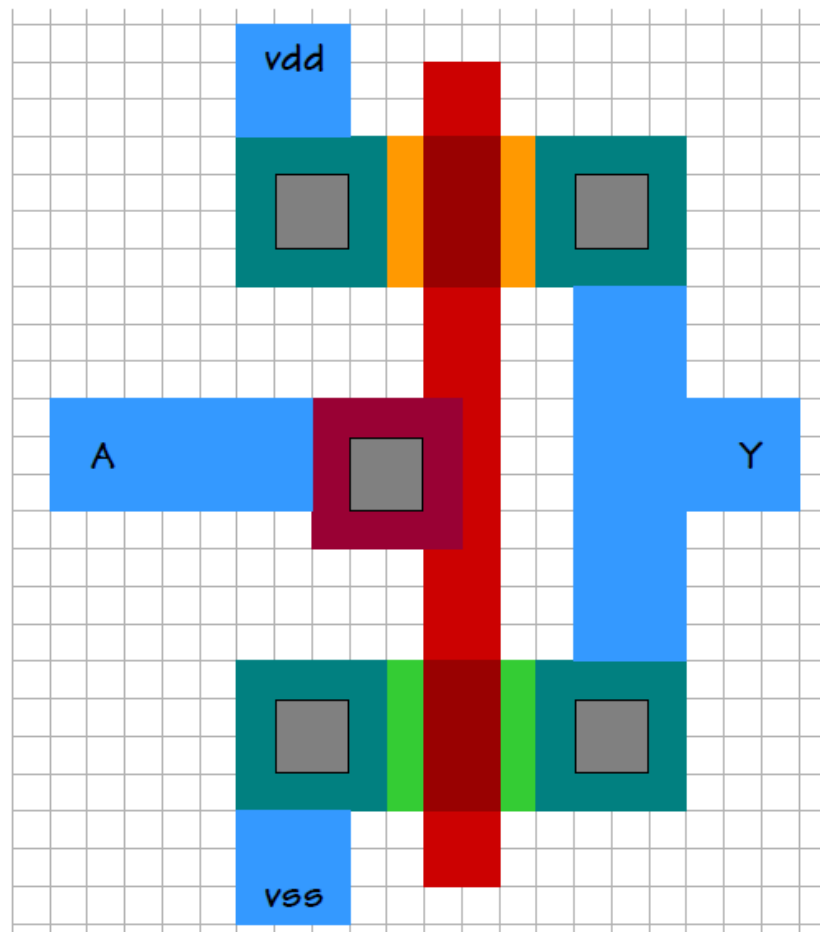
Custom Cells: Lambda-Based Design Rules

One lambda = one half of the “minimum” mask dimension, typically the length of a transistor channel. Usually all edges must be “on grid”, e.g., in the MOSIS scalable rules, all edges must be on a lambda grid.



但實在非常複雜又辛苦

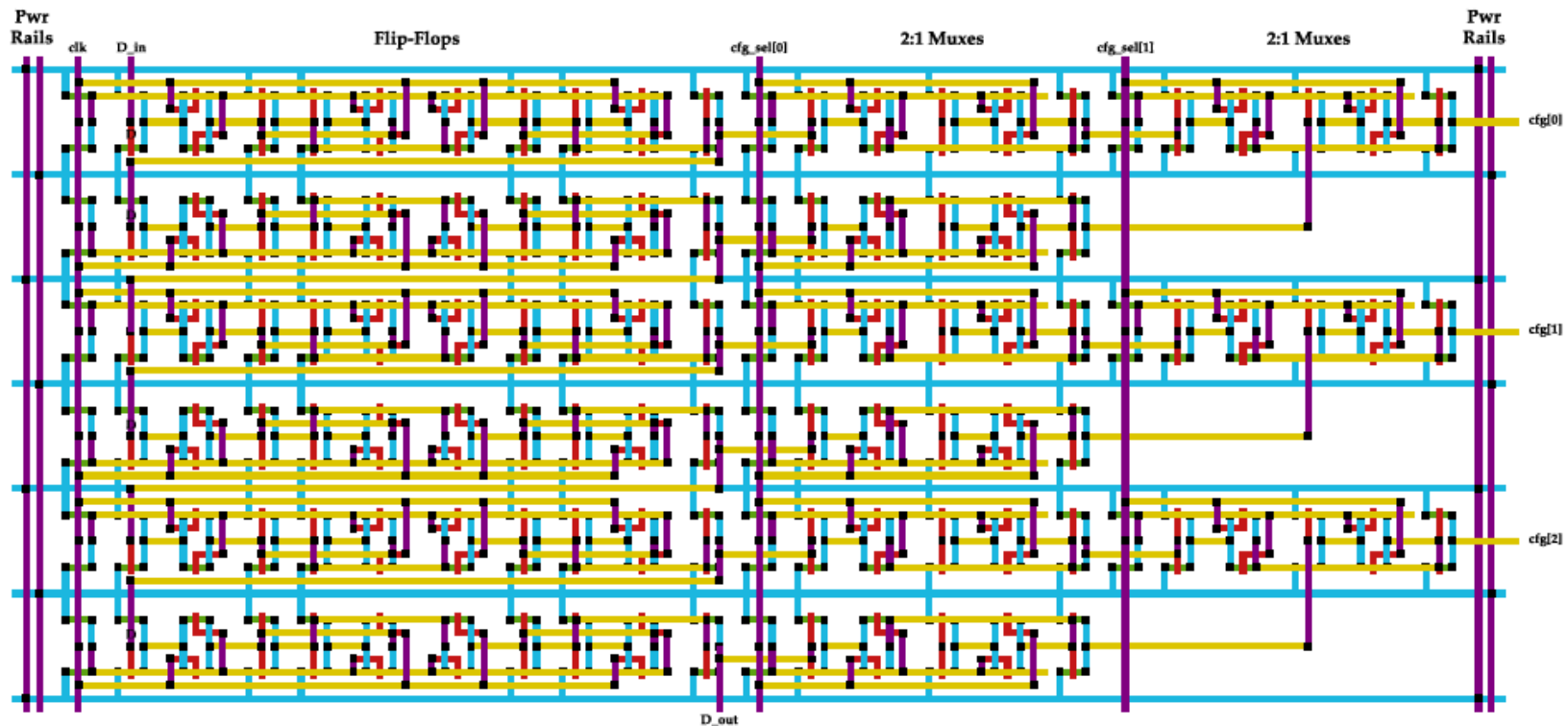
Custom Cells: Sample “Lambda” Layout



像是這樣的圖

- 竟然能手動做得出來

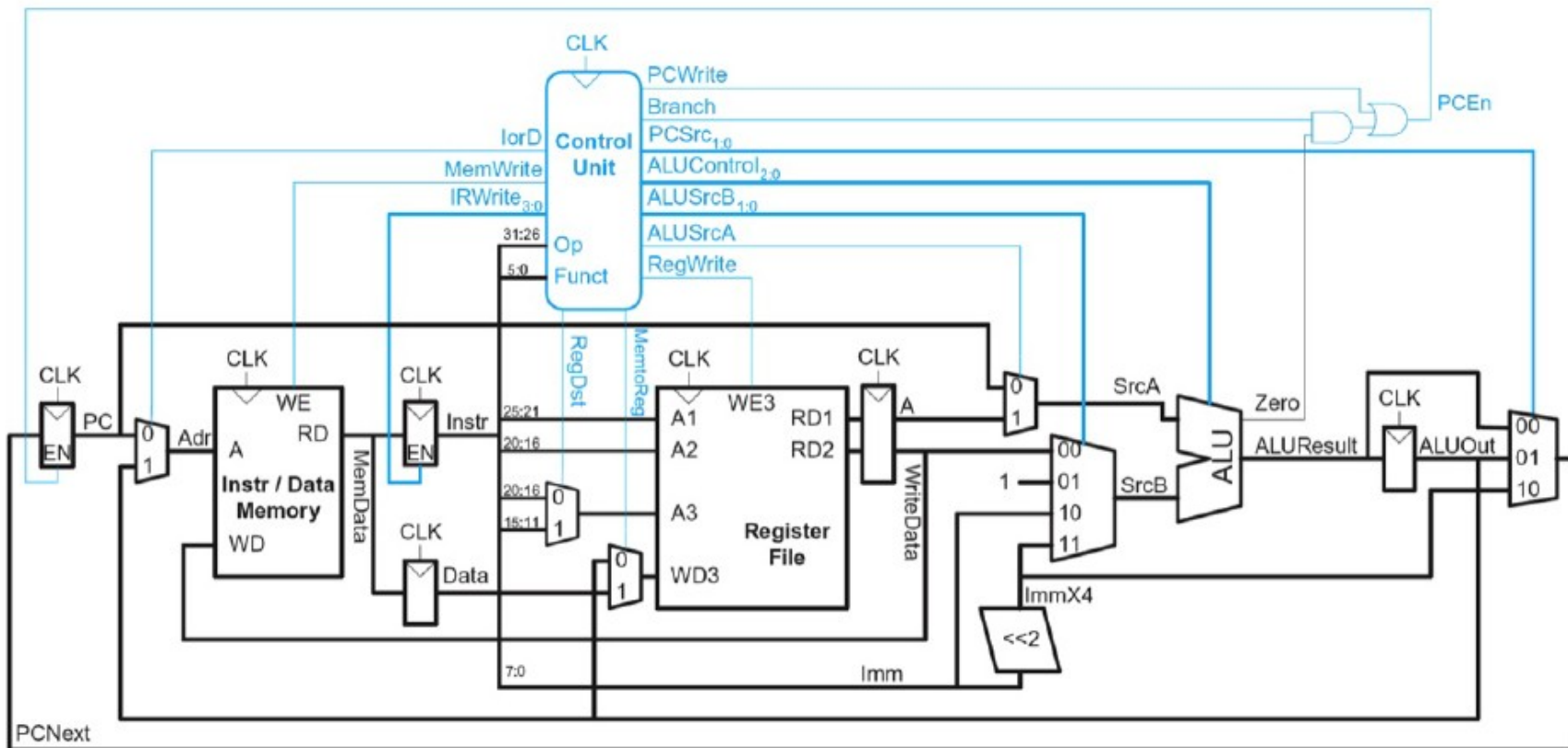
Custom Cells: Example Stick Diagram



如果要做一顆 CPU

- 那不是要畫到瘋掉

Custom Datapaths, Memories, Control

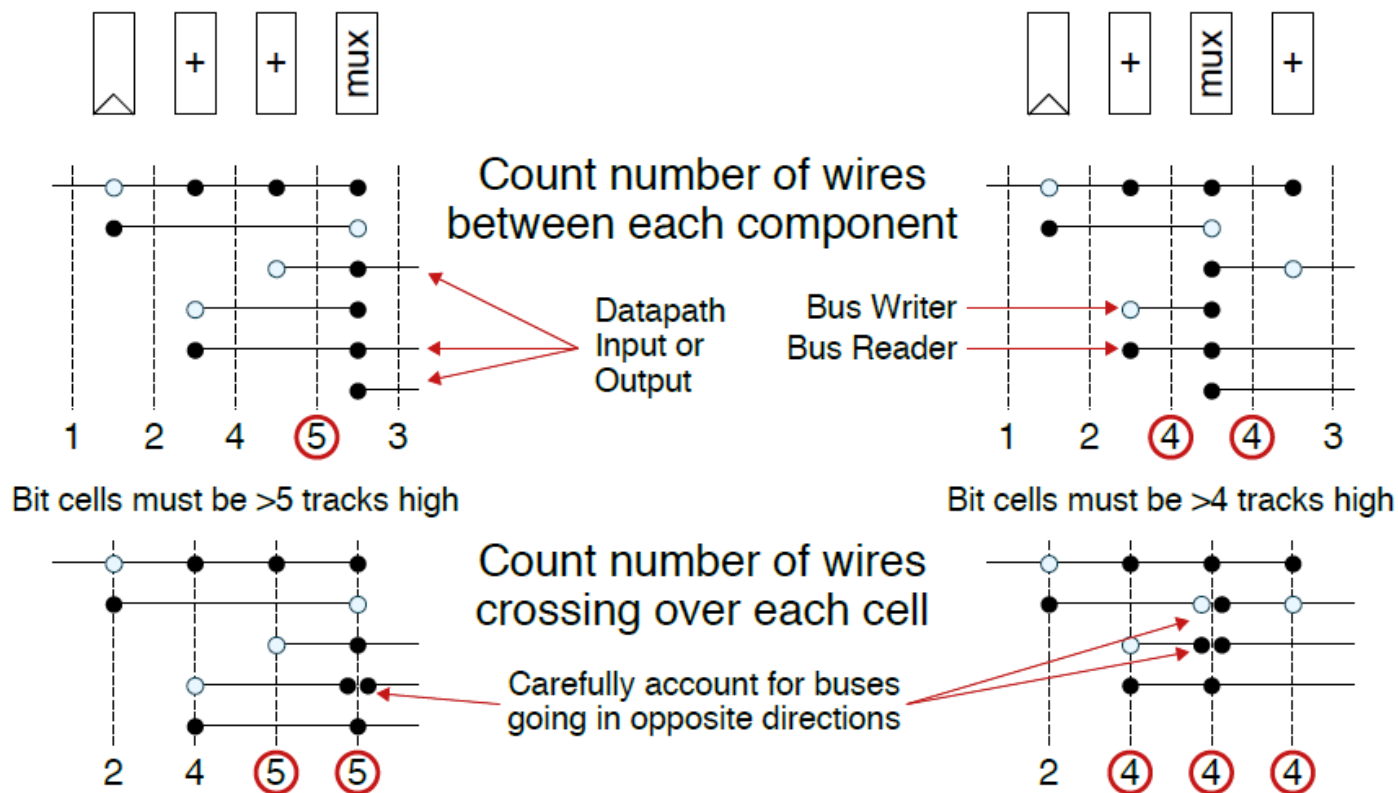


所以後來都會模組化

- 小圖畫好後存檔

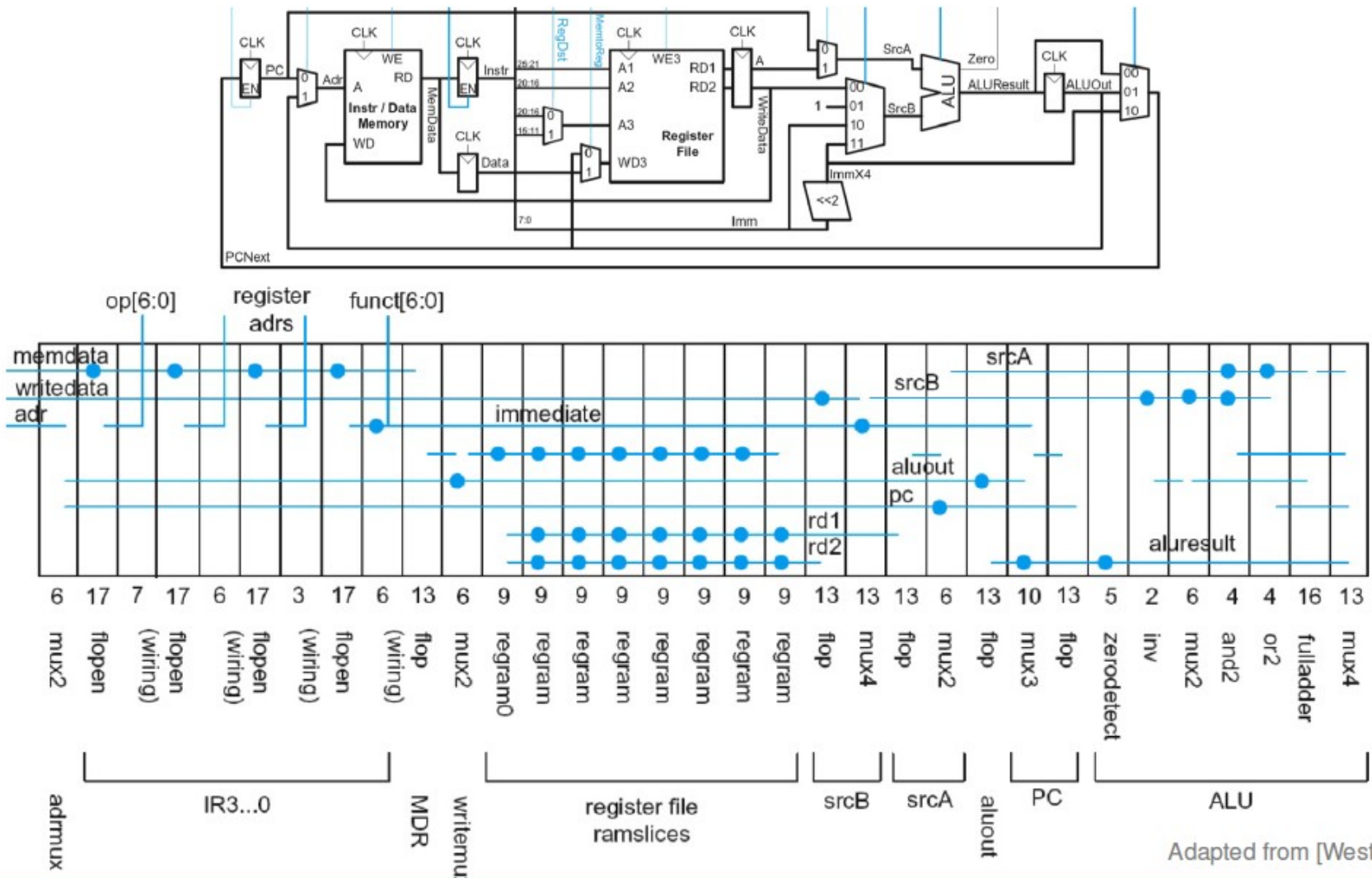
Custom Datapaths: Optimizing Datapath Layout

Reduce congestion by rearranging datapath components to minimize required number of vertical tracks per bitslice



然後用小圖拼接成大圖

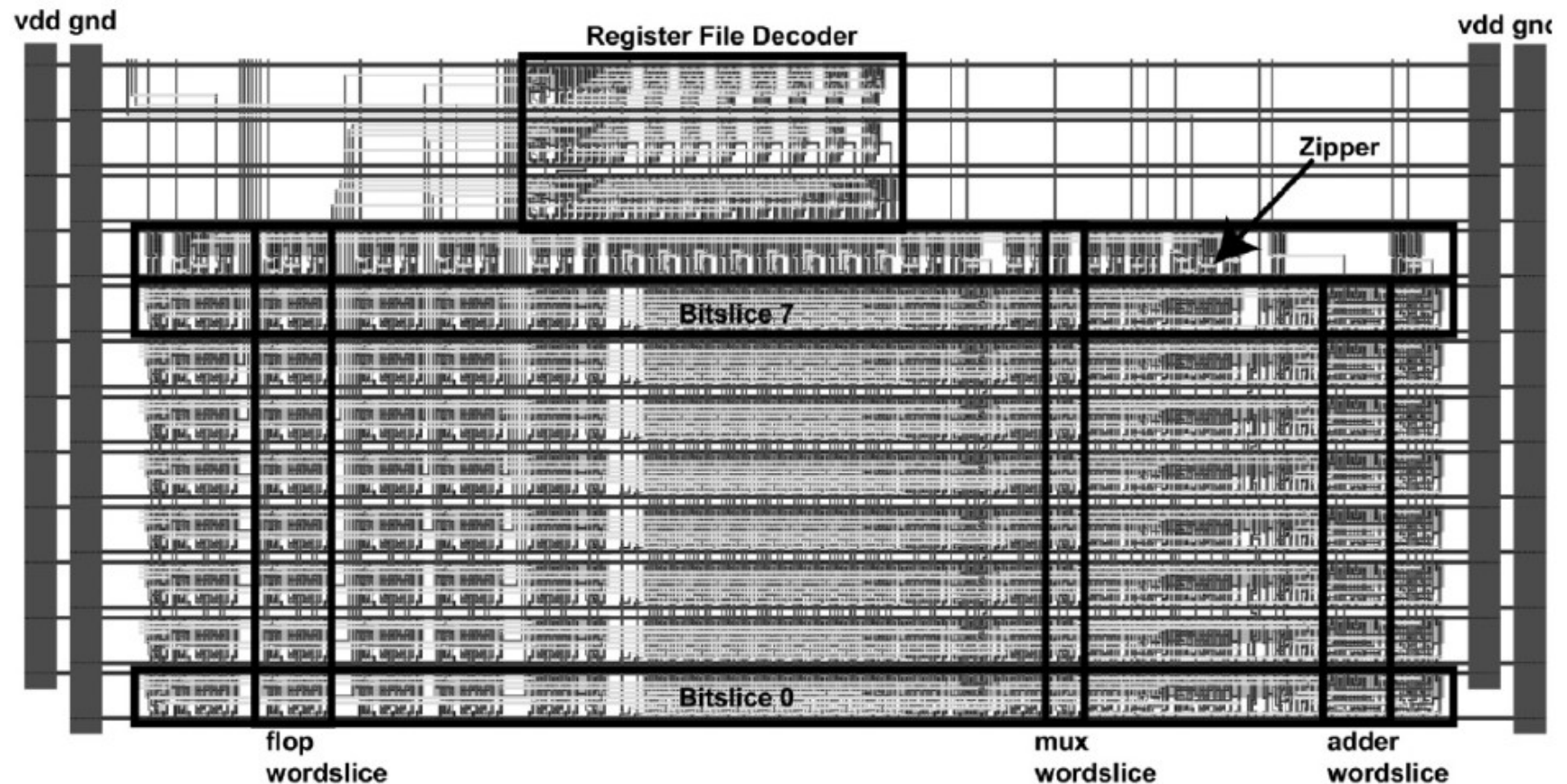
Custom Datapaths: MIPS Datapath Track Allocation



Adapted from [Weste'11]

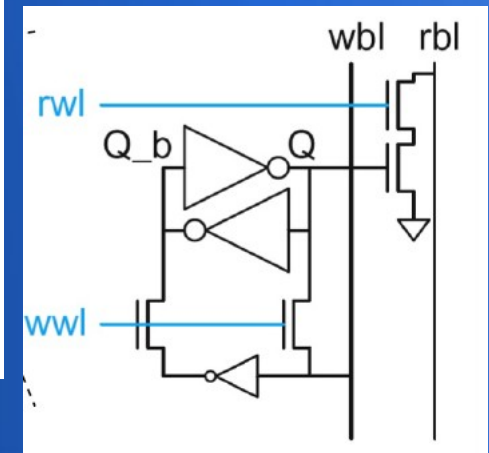
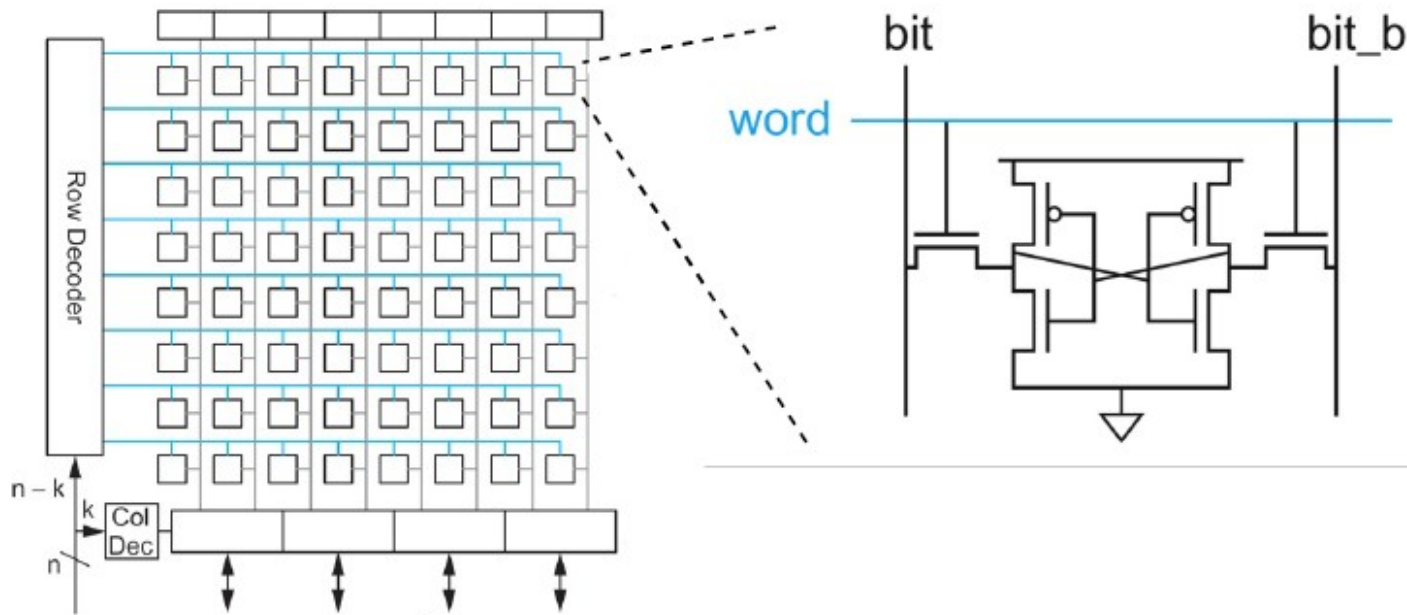
你可以想像那個複雜性

Custom Datapaths: MIPS Datapath Example (1)



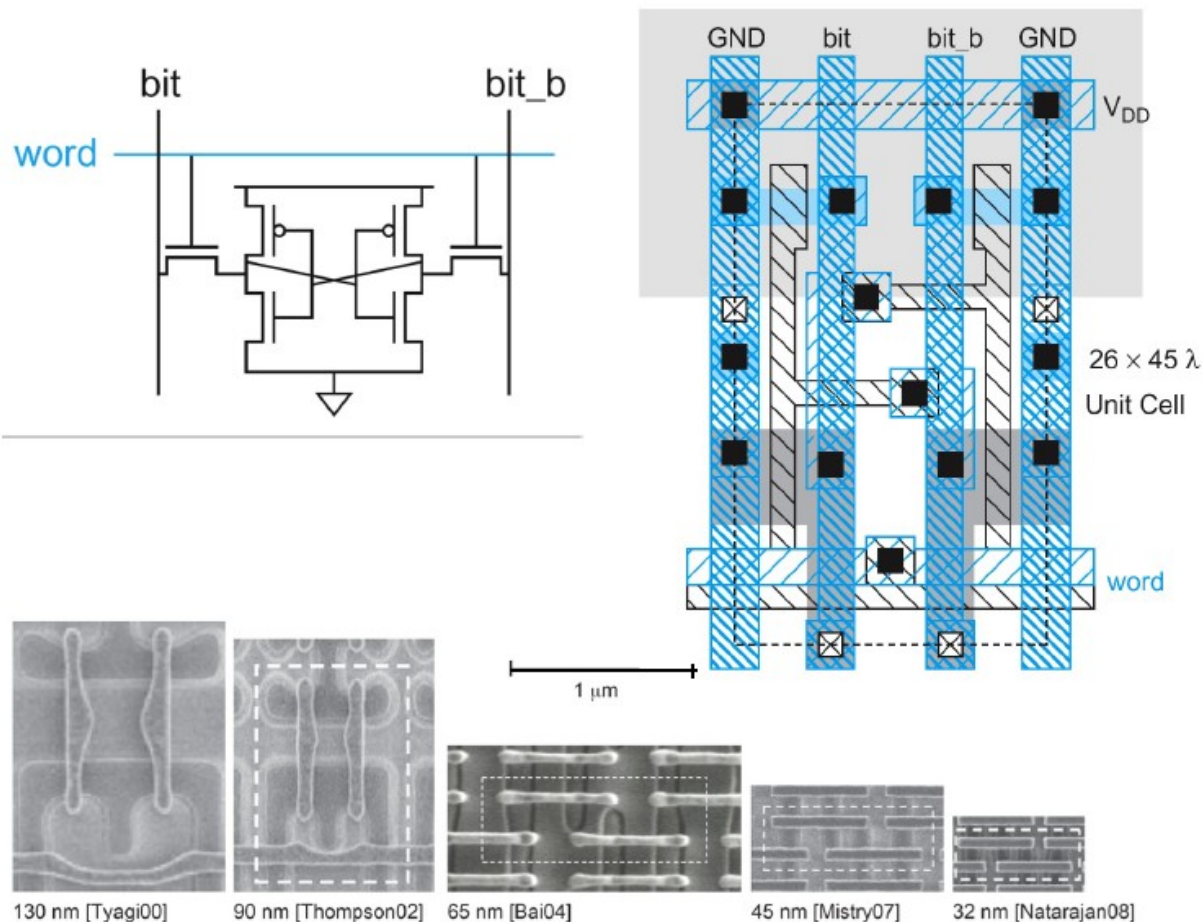
還好記憶體每格都長的一樣

Custom Memories: SRAM Circuits



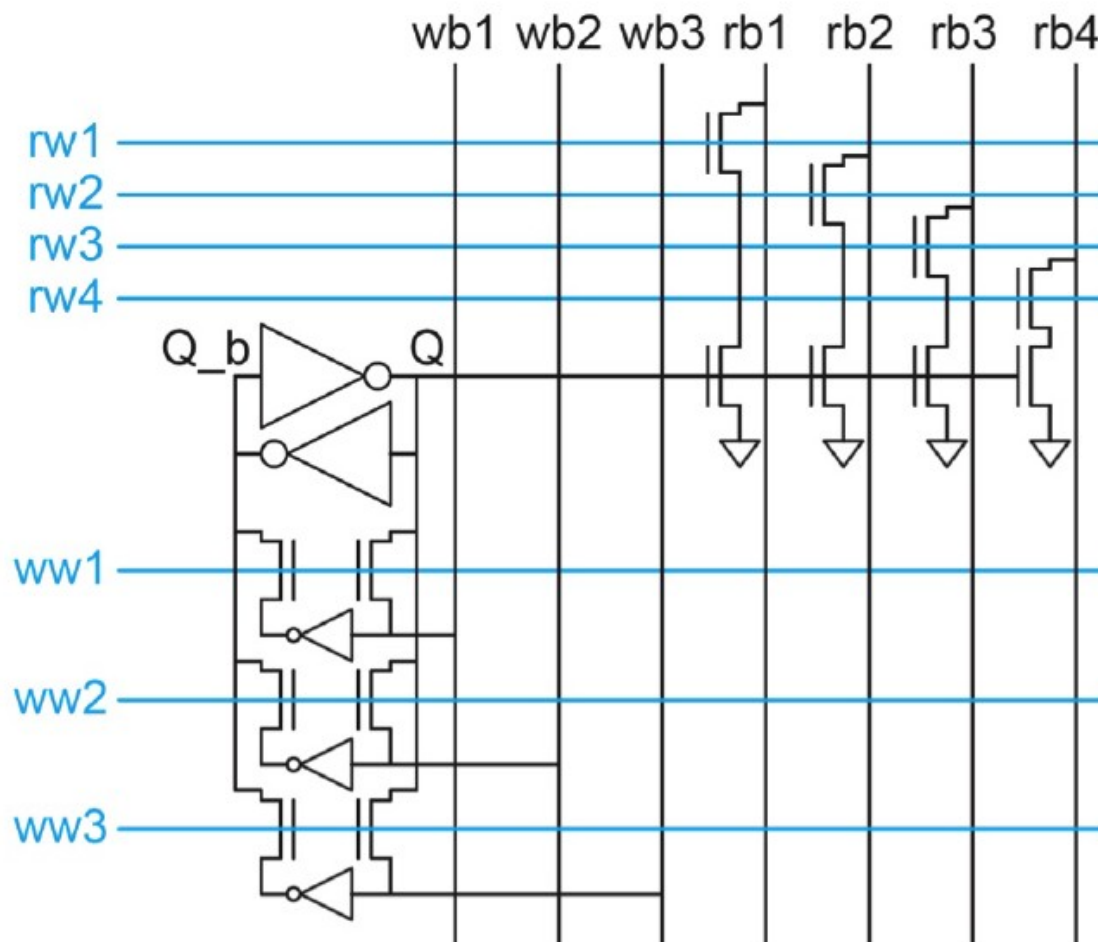
所以佈線方式也都一樣

Custom Memories: SRAM Layout



只要設計好控制線路就行

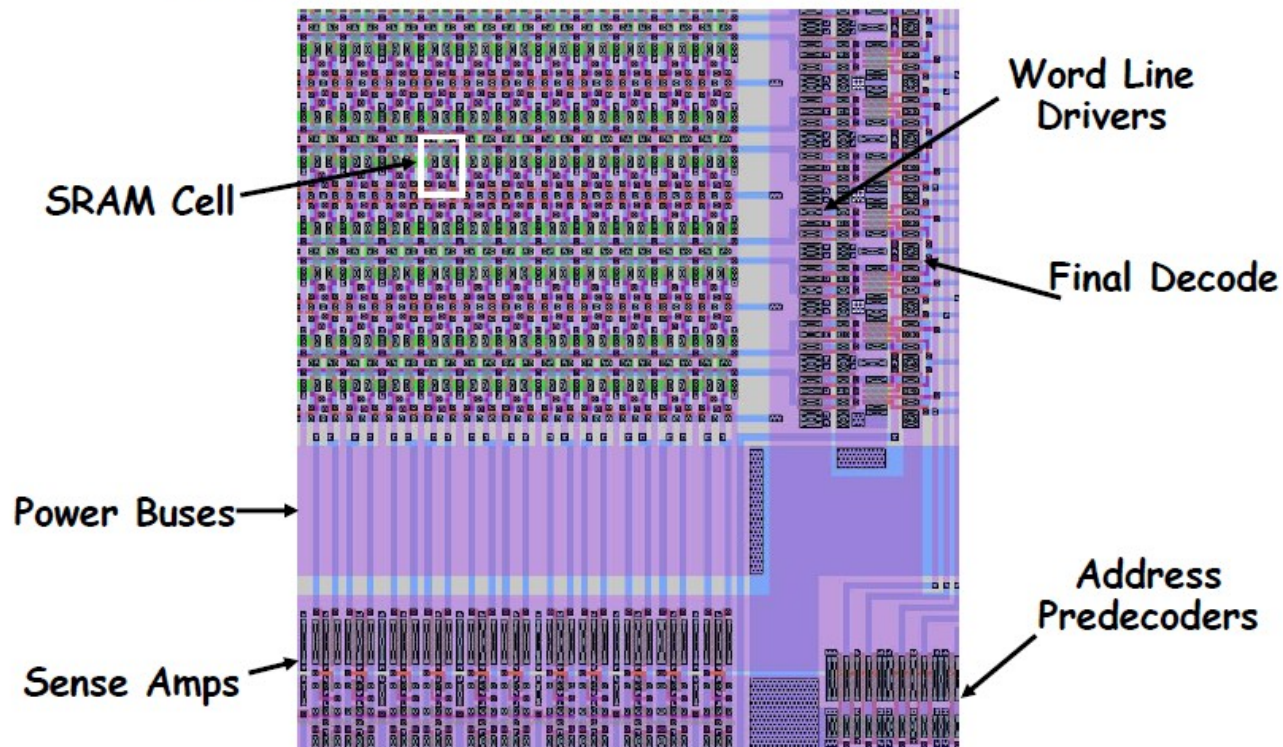
Custom Memories: Register File Circuits



所以你很少會聽到有公司是做記憶體設計的

Custom Memories: SRAM

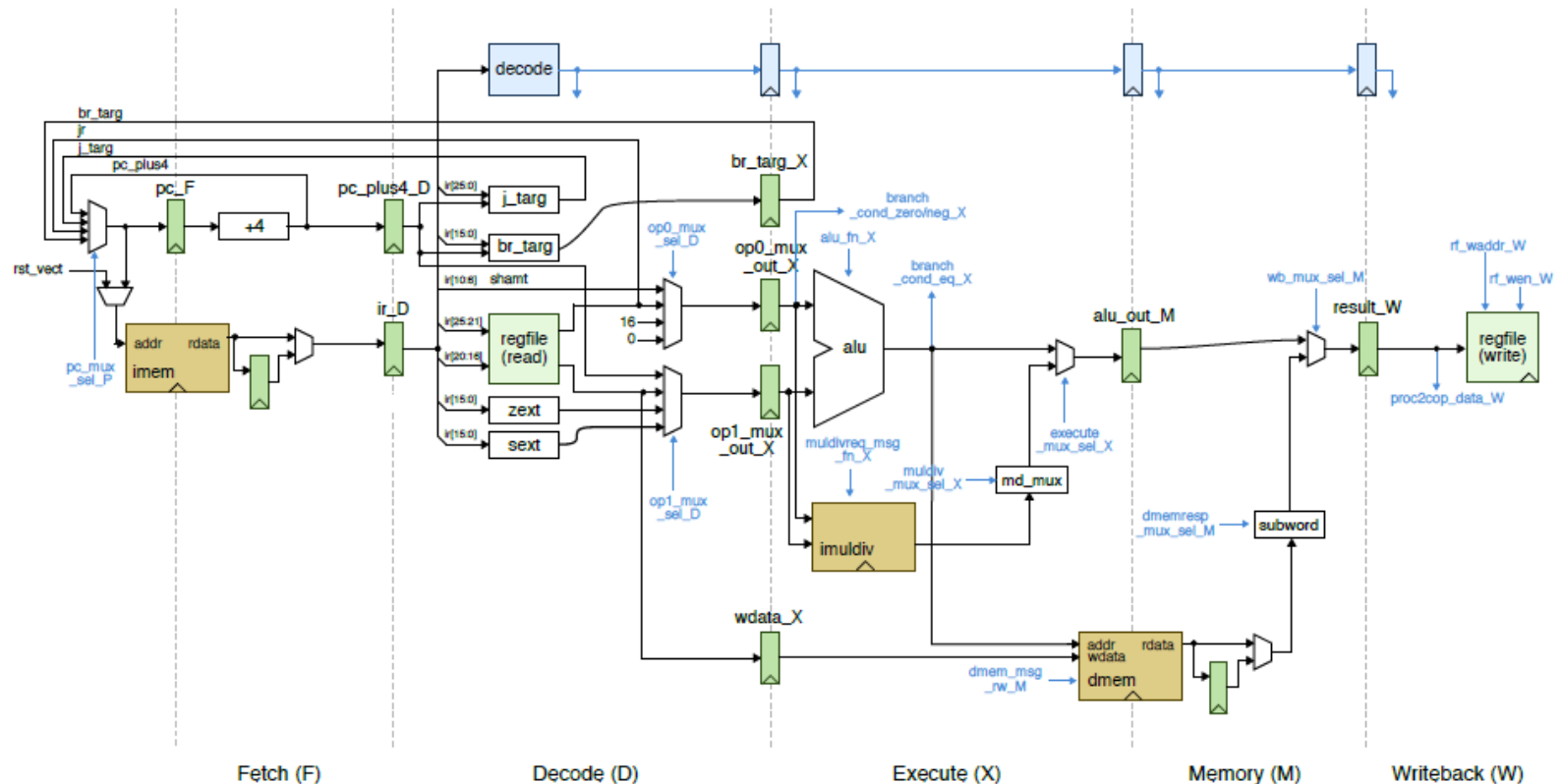
- Regular arrays built with cells that abut in two dimensions
 - Have to pitch match in both dimensions



但是對於處理器

- 那就很需要《設計》了

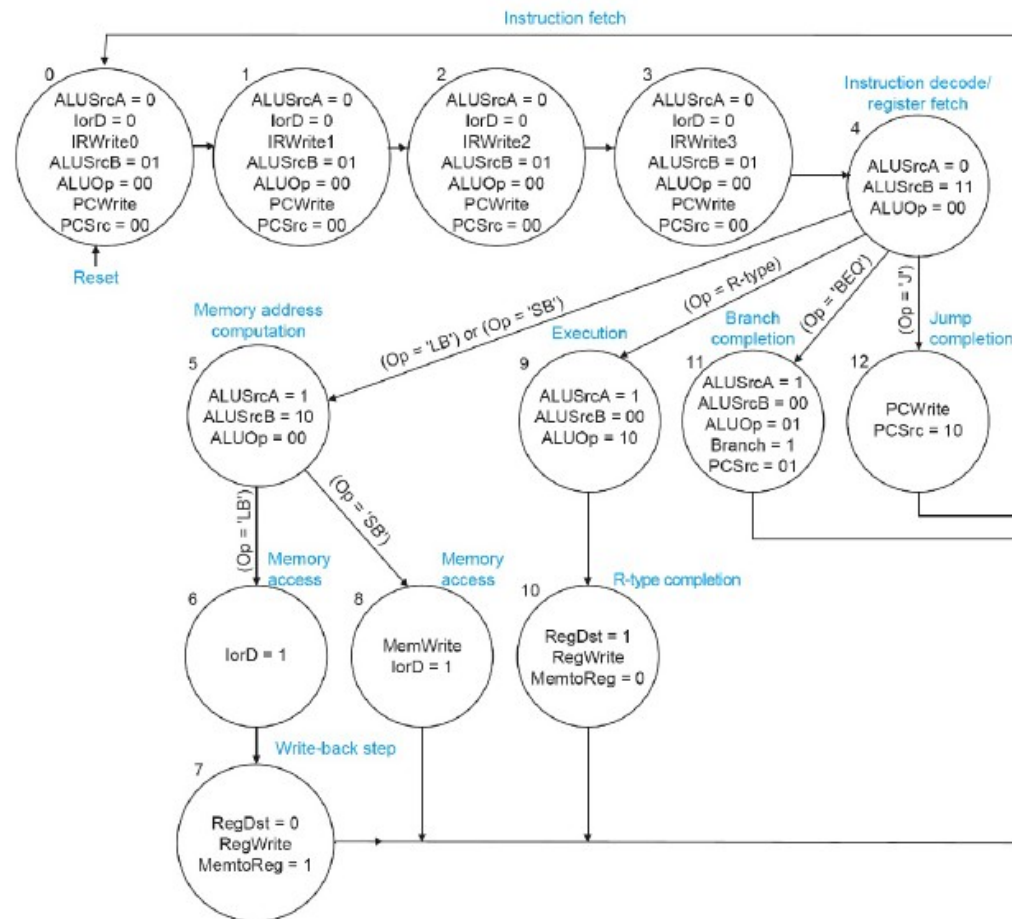
Custom Datapaths, Memories, Control



特別是控制單元

- 基本上需要量身訂做

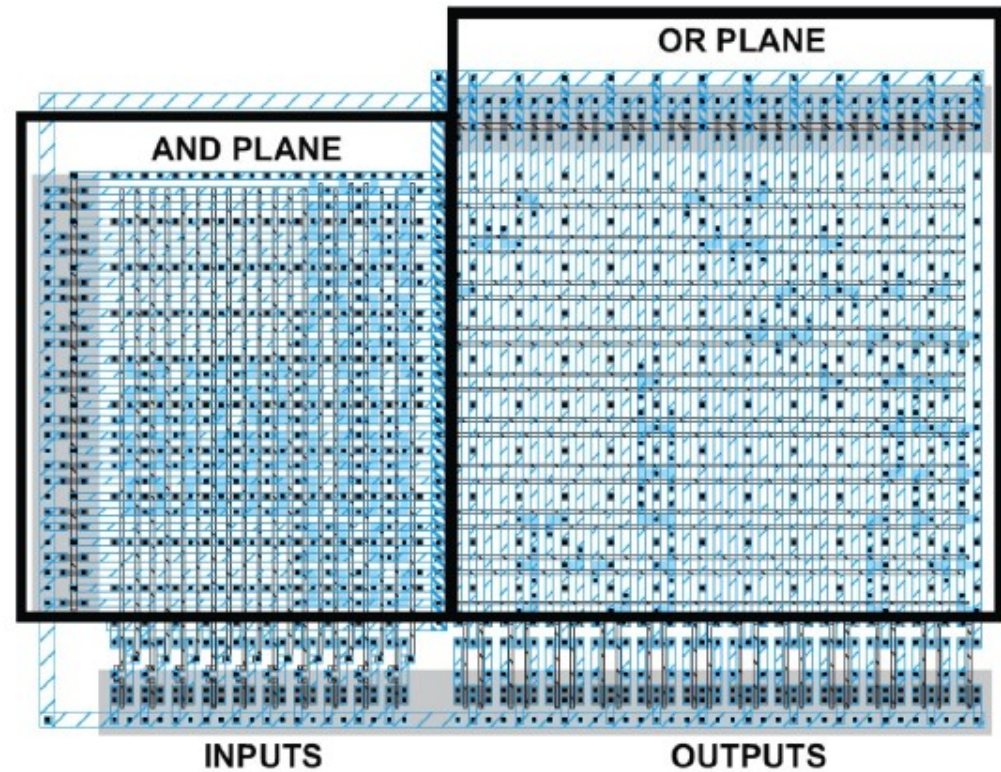
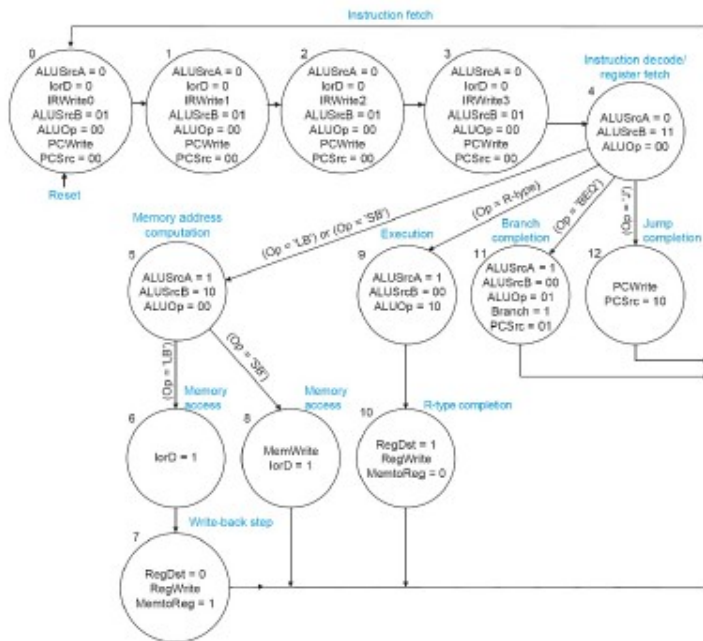
Finite-State-Machine Control Unit



若用 PLA 可程式邏輯陣列

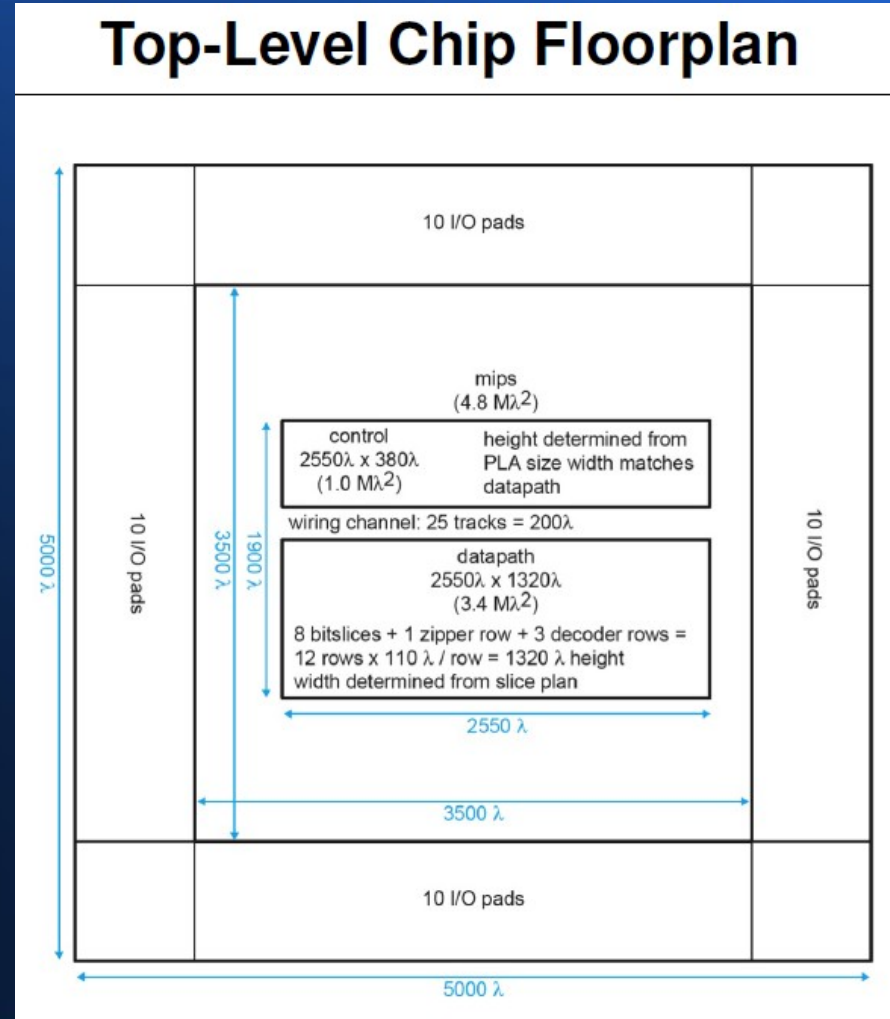
- 會是像這樣的固定佈線，只要設計連接點就行

Full Custom Control Logic with PLA



但若做成 ASIC 專用晶片

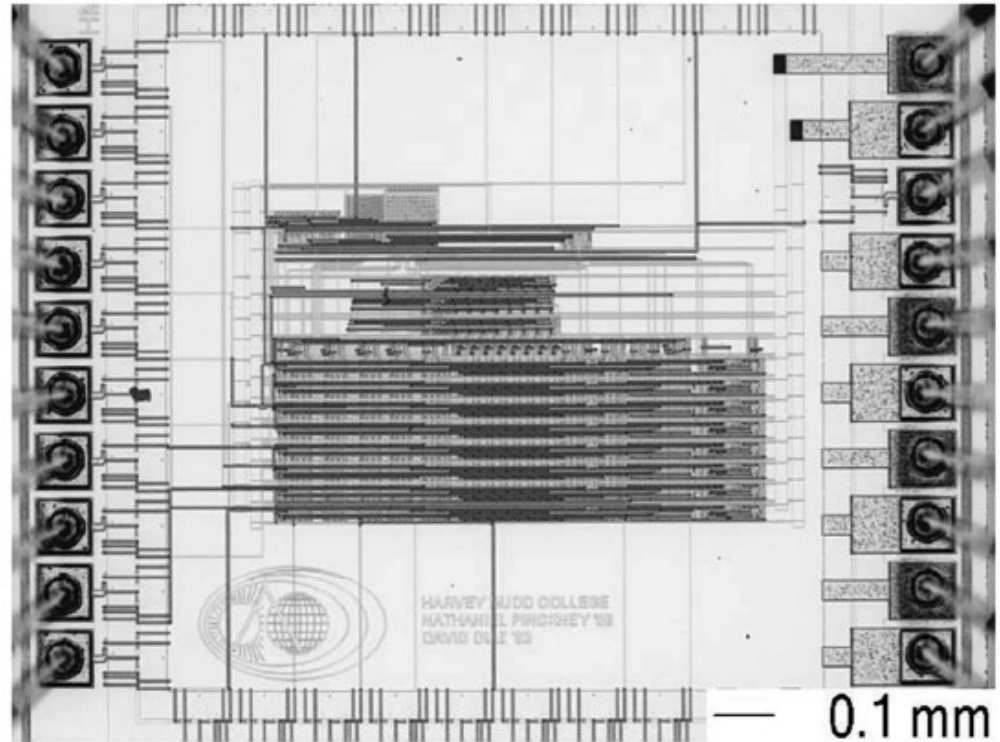
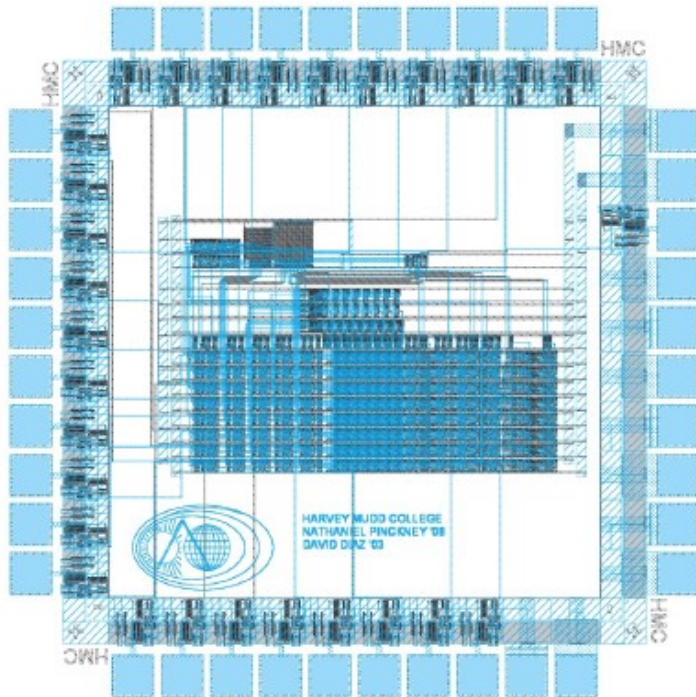
- 就得考慮各元件的位置，這就是 Floorplan 布局問題



雖然 ASIC 比較難設計

- 但是速度會比較快，量產後成本會比 PLA 便宜

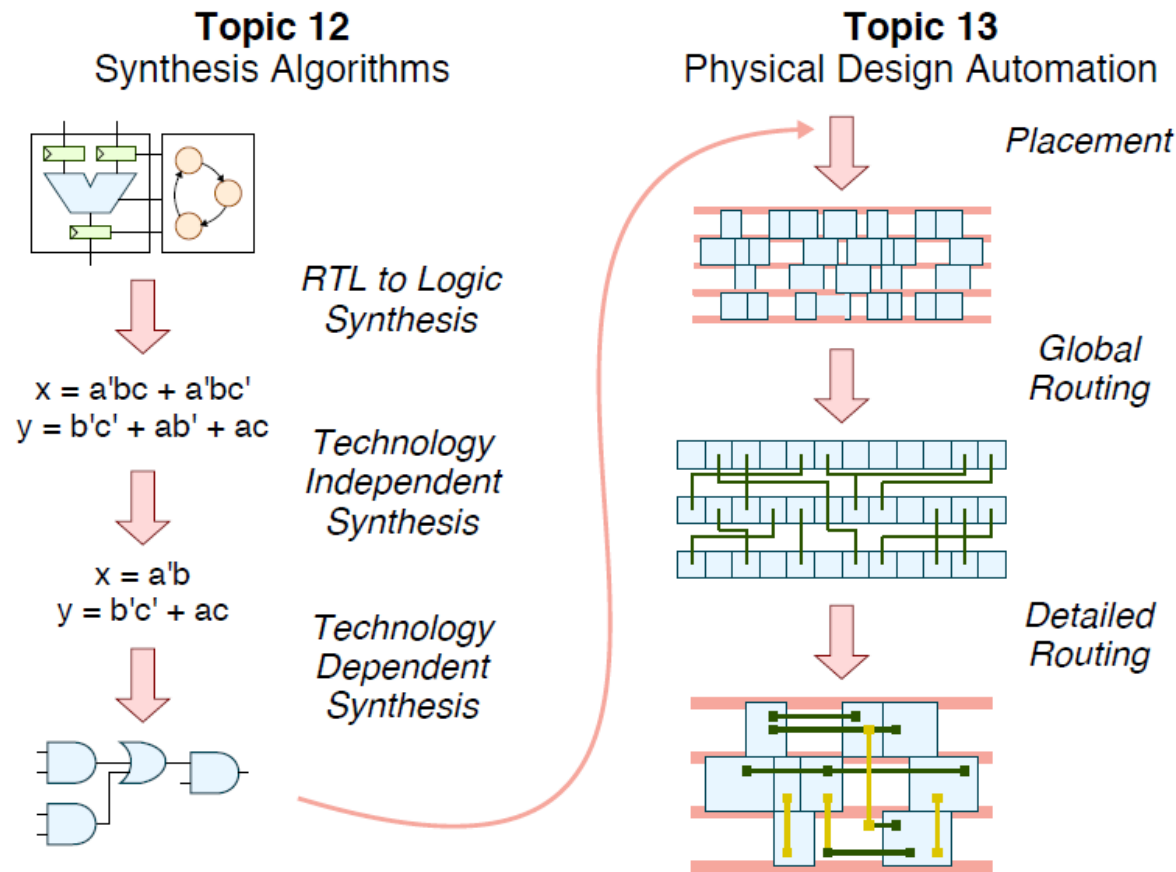
Final Full-Custom MIPS Processor



如果你曾經學過數位電路

- 那應該學過下圖中 Topic 12 的流程

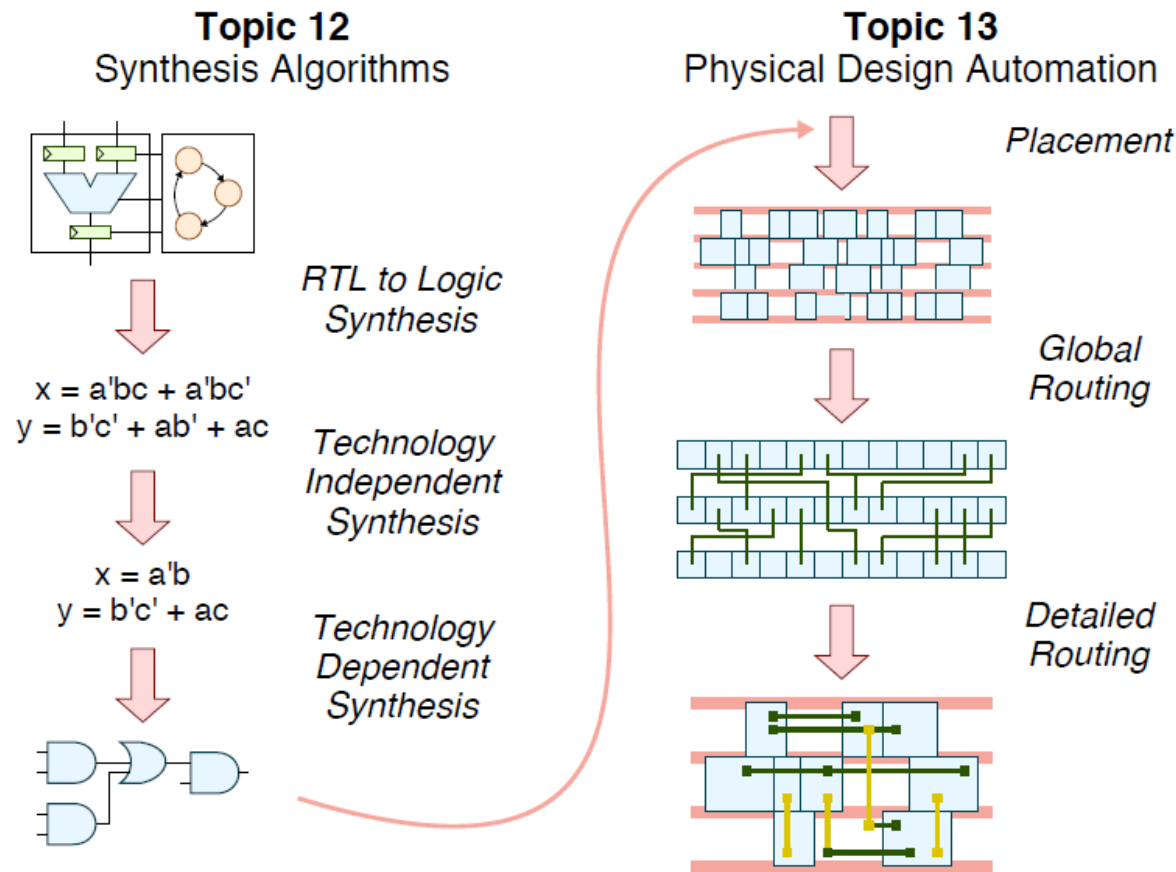
Standard-Cell-Based Flow CAD Algorithms



經過區塊和線路布局後

- 就會像 Topic 13 展示的那樣

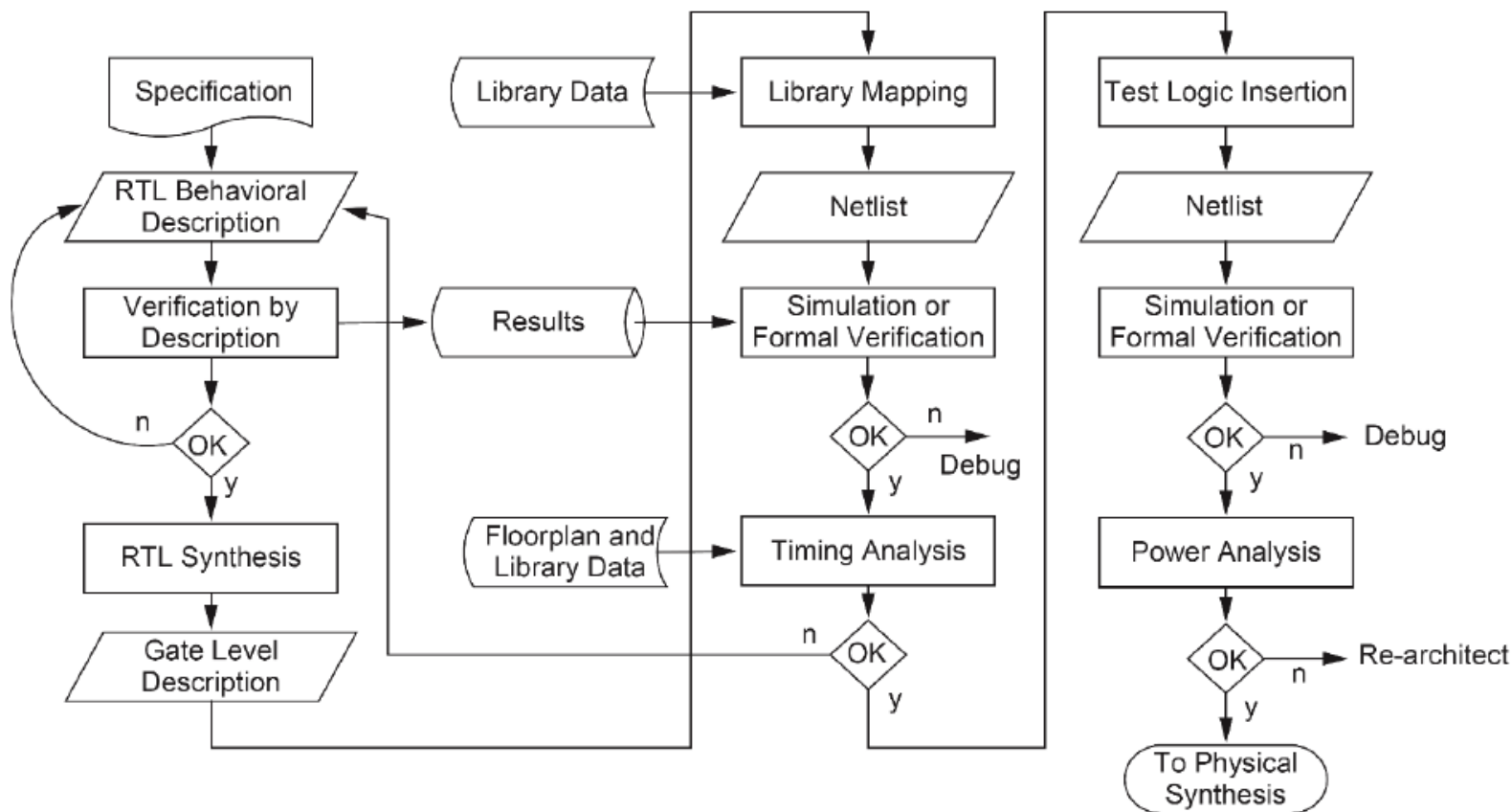
Standard-Cell-Based Flow CAD Algorithms



已經建立好的模組，就形成了圖庫

- 於是用下列流程，就可以較省力的建構新元件

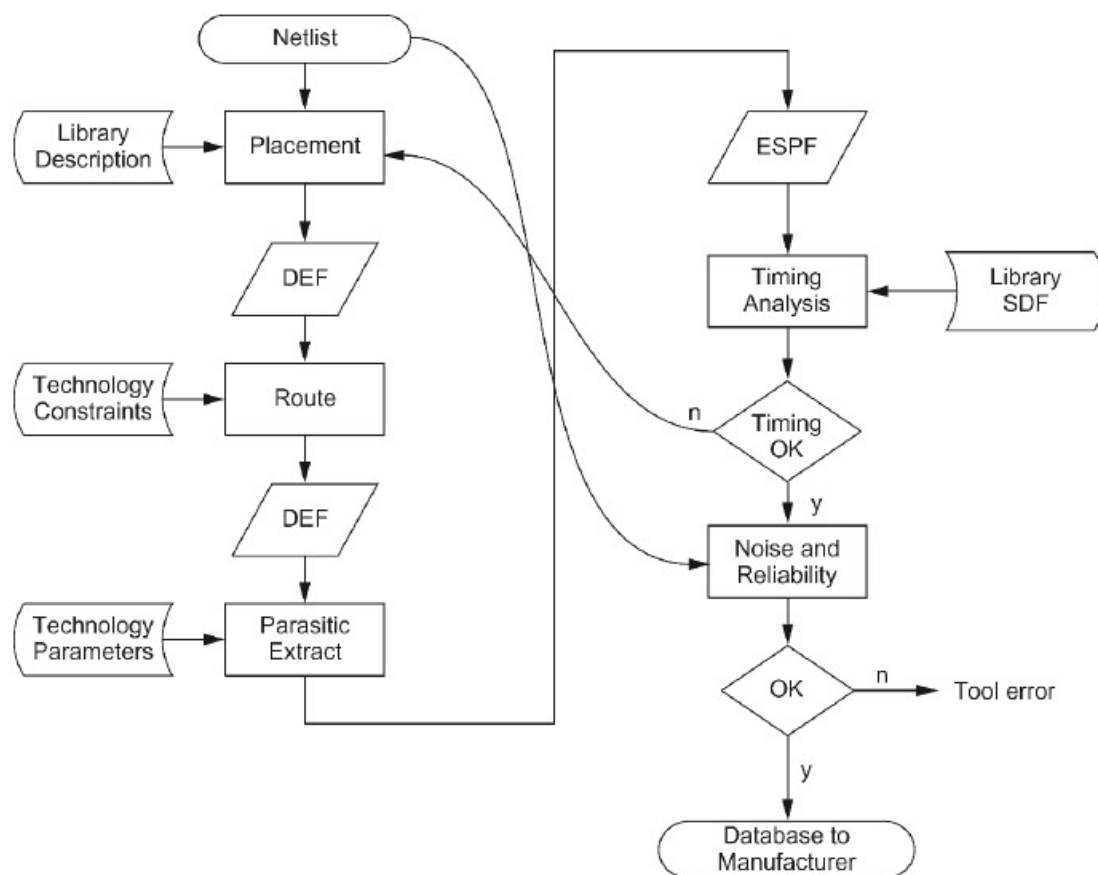
Standard-Cell-Based Front-End Flow



其實還是很複雜

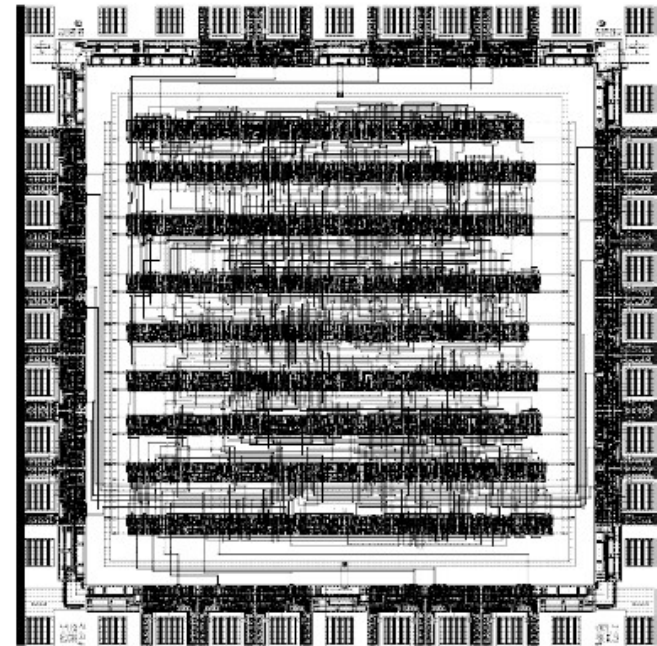
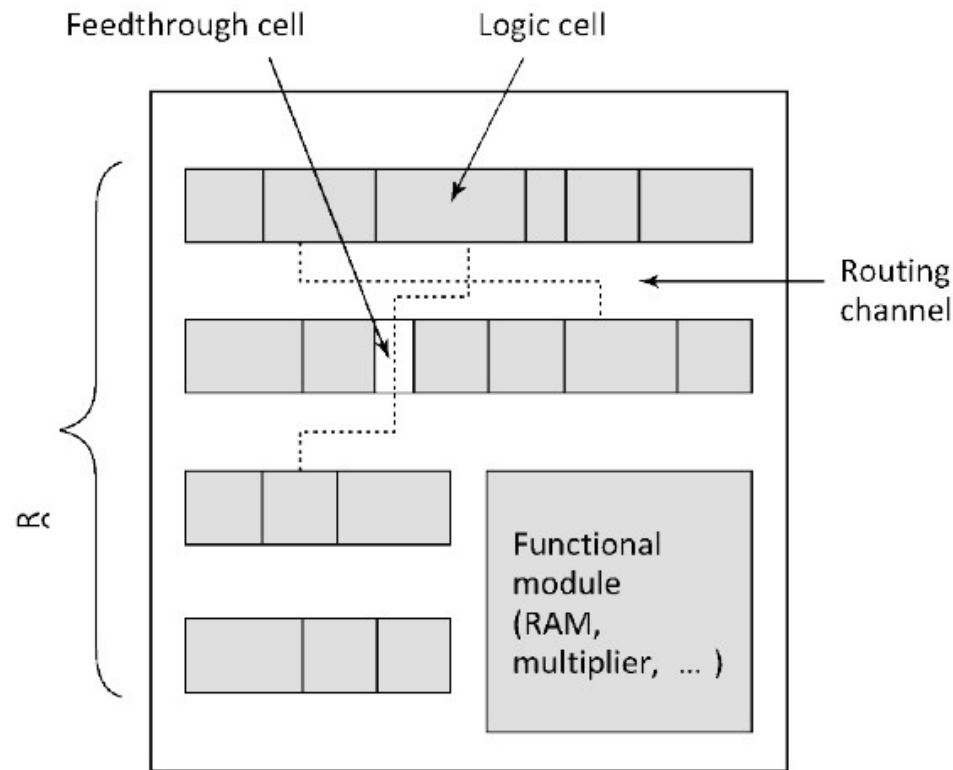
- 有分 Front-End 和 Back-End

Standard-Cell-Based Back-End Flow



老式手動設計的圖長這樣

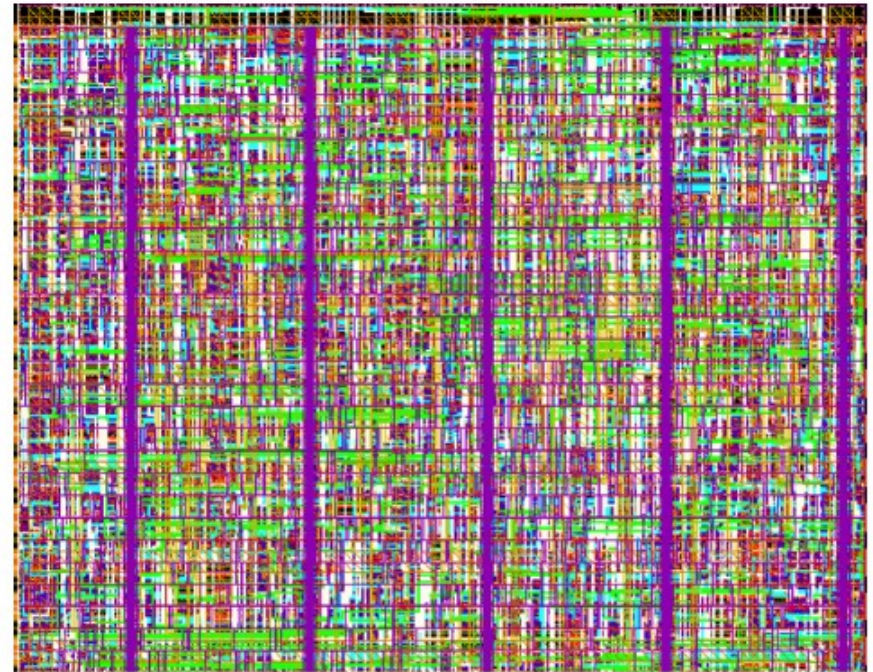
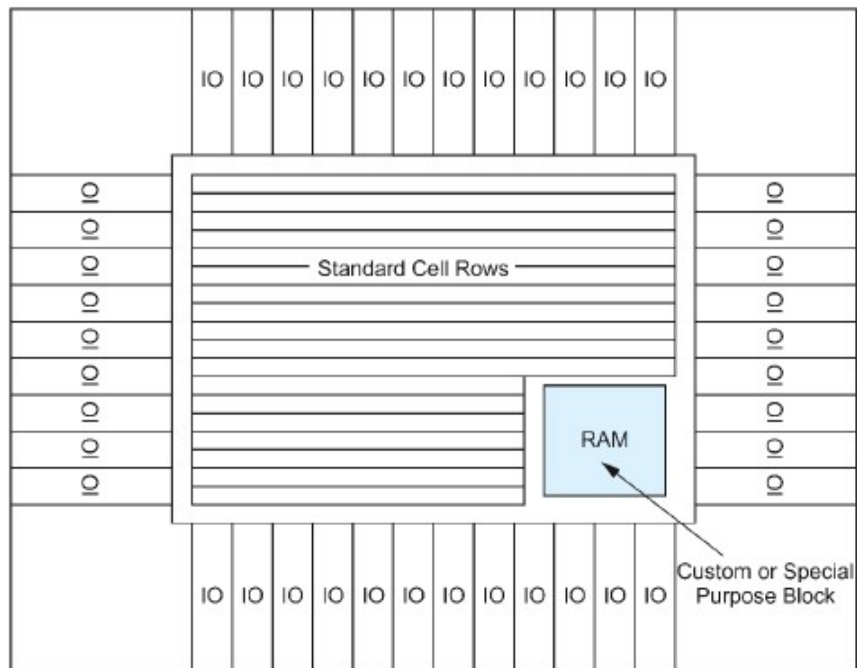
Older Standard-Cell ASICs



Limited metal layers require dedicated routing channels and feedthrough cells

新式用電腦布線的圖長這樣

Modern Standard-Cell ASICs



Increasing number of metal layers allow cells to be hidden under interconnect

你可以從元件庫中選用

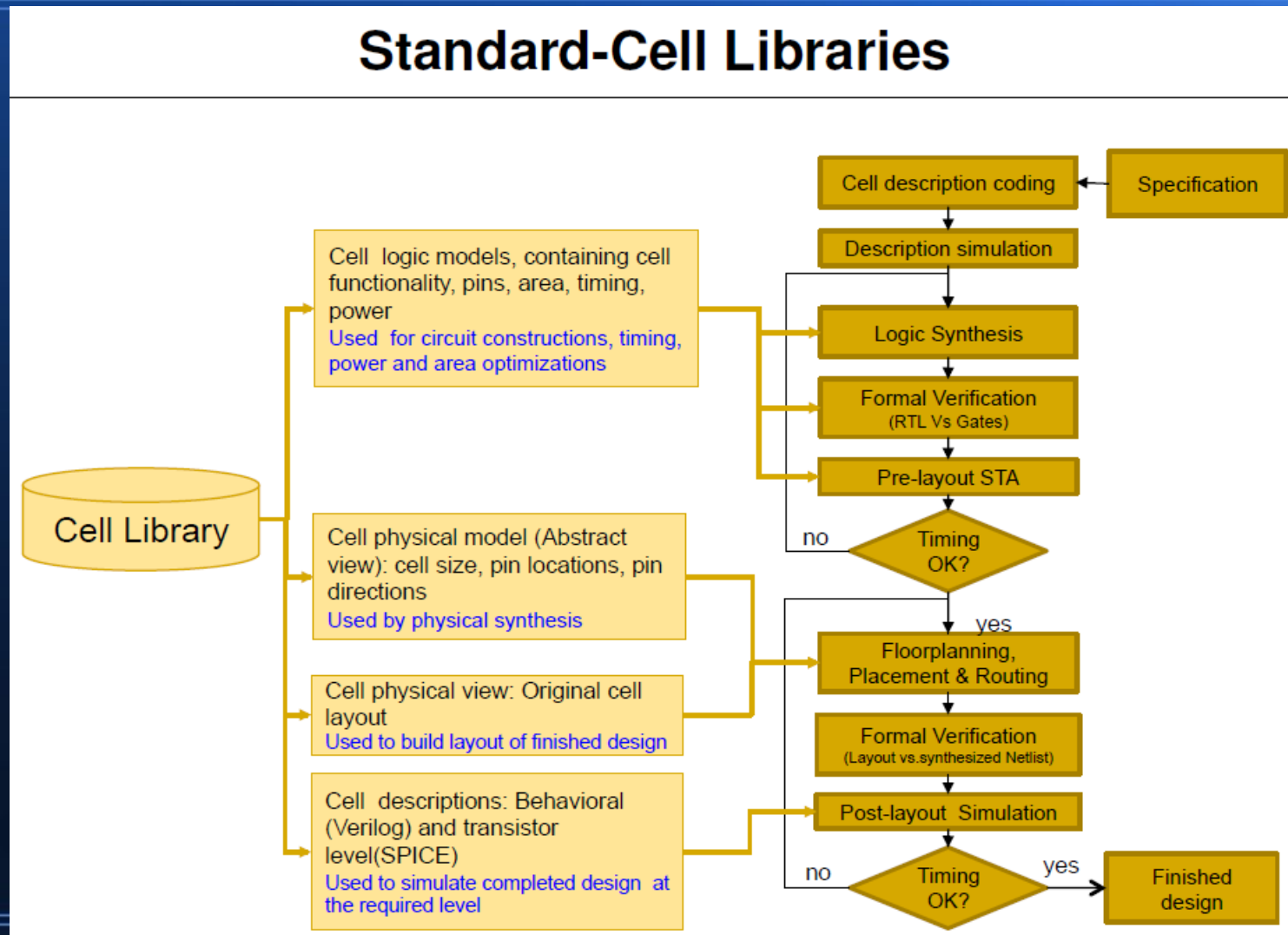
- 就像寫程式那樣 ...

Standard-Cell Libraries

Gate Type	Variations	Options
Inverter/Buffer/ Tristate Buffers		Wide range of power options, 1x, 2x, 4x, 8x, 16x, 32x, 64x minimum size inverter
NAND/AND	2–8 inputs	High, normal, low power
NOR/OR	2–8 inputs	High, normal, low power
XOR/XNOR		High, normal, low power
AOI/OAI	21, 22	High, normal, low power
Multiplexers	Inverting/noninverting	High, normal, low power
Adder/Half Adder		High, normal, low power
Latches		High, normal, low power
Flip-Flops	D, with and without synch/asynch set and reset, scan	High, normal, low power
I/O Pads	Input, output, tristate, bidirectional, bound- ary scan, slew rate limited, crystal oscillator	Various drive levels (1–16 mA) and logic levels

只是設計時要更謹慎小心

- 因為一旦設計有問題，未來量產得賠上數十億

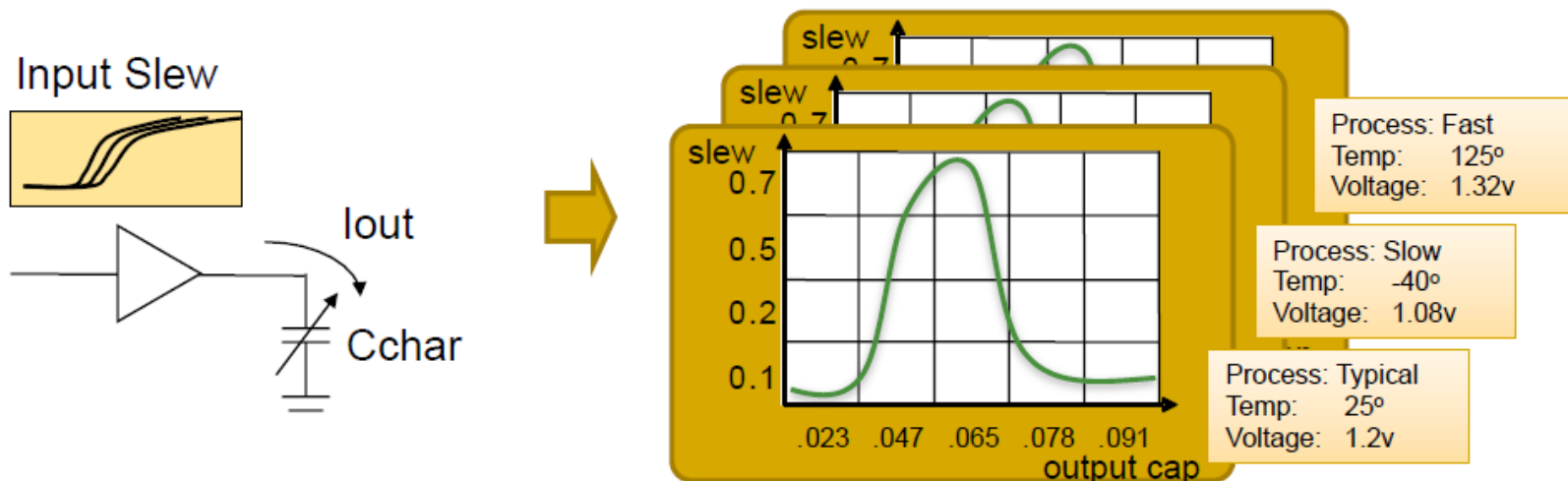


所以連阻抗溫度要精準的設定後跑模擬

- 因為一顆 IC 交給台積電，光試作就得幾千萬

Standard-Cell Electrical Characterization

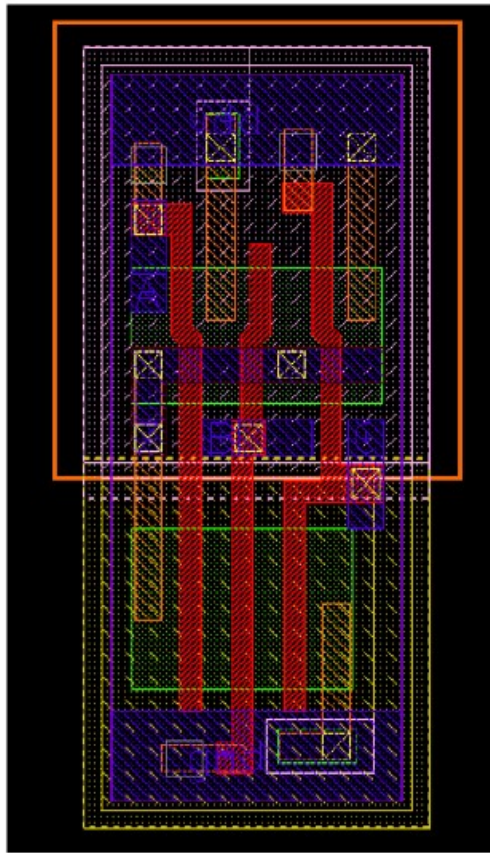
- ▶ Characterization computes cell parameter (e.g. delay, output current) depending on input variables: output load, input slew, etc.)
- ▶ Characterization is performed for various combinations of operating conditions: process, voltage, temperature (also called PVT corners)



每個元件都有精準的參數

- 包含電阻、溫度、電容、公式等等

ST Microelectronics – 3-Input NAND Gate



Path	1.2V - 125°C	1.6V - 40°C
$In1-t_{pLH}$	$0.073+7.98C+0.317T$	$0.020+2.73C+0.253T$
$In1-t_{pHL}$	$0.069+8.43C+0.364T$	$0.018+2.14C+0.292T$
$In2-t_{pLH}$	$0.101+7.97C+0.318T$	$0.026+2.38C+0.255T$
$In2-t_{pHL}$	$0.097+8.42C+0.325T$	$0.023+2.14C+0.269T$
$In3-t_{pLH}$	$0.120+8.00C+0.318T$	$0.031+2.37C+0.258T$
$In3-t_{pHL}$	$0.110+8.41C+0.280T$	$0.027+2.15C+0.223T$

3-input NAND cell
(from ST Microelectronics):
C = Load capacitance
T = input rise/fall time

這些元件

- 都存在 Cadence/Synopsys 等公司的 EDA 工具中

Standard-Cell Electrical Characterization (.lib)

```
/* Characterization for a 3-input NAND gate */
cell ( NAND3X0 ) {

    /* Overall characterization */
    cell_footprint      : "nand3x0";
    area                : 7.3728;
    cell_leakage_power  : 9.151417e+04;

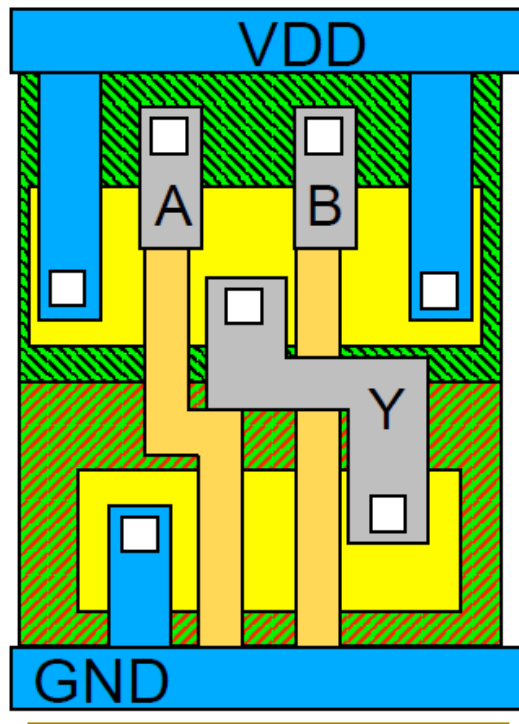
    /* Characterization for input pin IN1 */
    pin ( IN1 ) {
        direction : "input";

        /* Fixed input capacitance */
        capacitance : 2.190745;

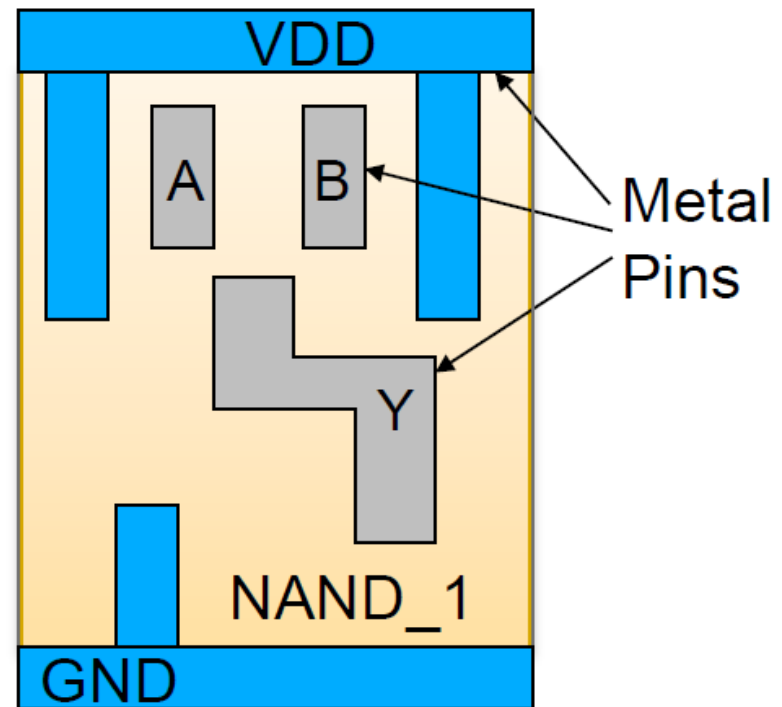
        /* Transient capacitance values */
        fall_capacitance : 2.212771;
        rise_capacitance : 2.168719;
    }
}
```

每個腳位都有精準的電氣特性描述

Standard-Cell Physical Characterization



Layout View



Abstract Physical View

每個元件

- 都有《負載、延遲、面積》等參數

SAED 90 nm Library – NAND Gate Data Sheet

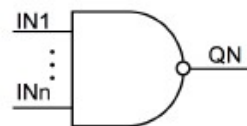


Figure 10.6. Logic Symbol of NAND

Table 10.11. NAND Truth Table (n=2,3,4)

IN1	IN2	...	INn	QN
0	X	...	X	1
X	0	...	X	1
...	...	...	...	1
X	X	...	0	1
1	1	1	1	0

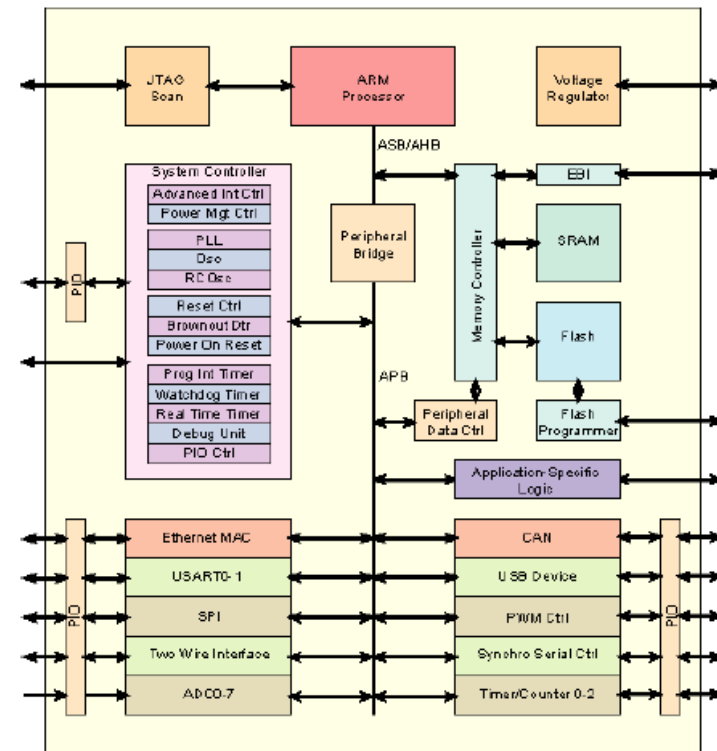
Cell Name	Operating Conditions: VDD=1.2 V DC, Temp=25 Deg.C, Operating Frequency: Freq=300 MHz, Capacitive Standard Load: Csl=13 fF				
	Cload	Prop Delay (Avg)	Power		Area
			Leakage (VDD=1.2 V DC, Temp=25 Dec.C)	Dynamic	
			ps	nW/MHz	
NAND2X0	0.5 x Csl	140	38	3583	5.5296
NAND2X1	1 x Csl	132	78	5208	5.5296
NAND2X2	2 x Csl	126	157	9191	9.2160
NAND2X4	4 x Csl	125	314	17902	14.7456
NAND3X0	0.5 x Csl	128	91	5331	7.3728
NAND3X1	1 x Csl	192	102	12200	11.9808
NAND3X2	2 x Csl	212	155	19526	12.9024
NAND3X4	4 x Csl	241	260	44937	15.6672
NAND4X0	0.5 x Csl	147	106	5357	8.2944
NAND4X1	1 x Csl	178	161	15214	12.9024

有了這些元件與 EDA

- IC 設計工業就可以往更高層次發展，像是 SOC

System-on-Chip (SoC)

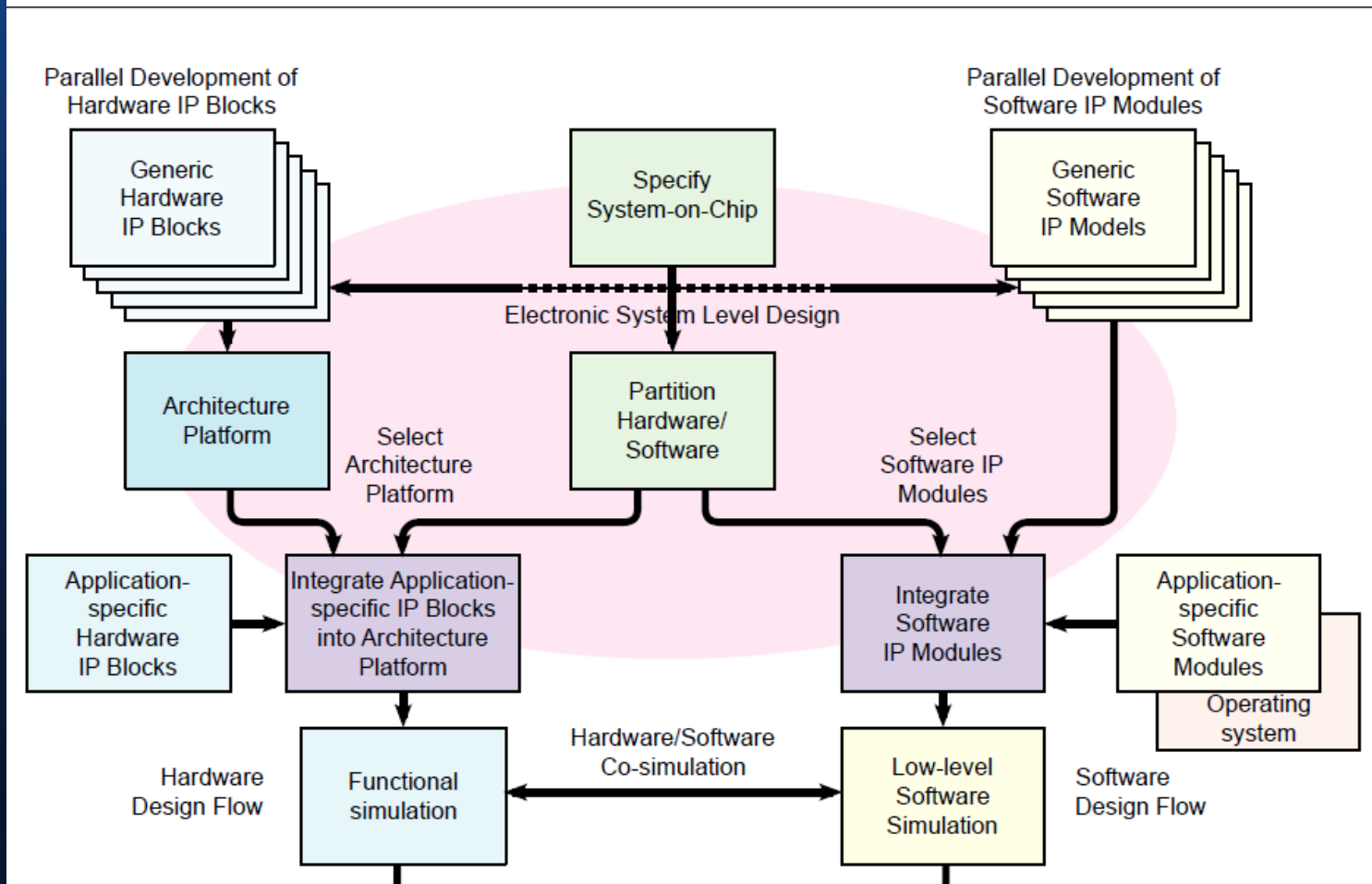
- Brings together: standard cell blocks, custom analog blocks, processor cores, memory blocks
- Standardized on-chip buses (or hierarchical interconnect) permit “easy” integration of many blocks.
 - Ex: AMBA, Sonics, ...
- “IP Block” business model: Hard- or soft-cores available from third party designers.
- ARM, inc. is the shining example. Hard- and “synthesizable” RISC processors.
- ARM and other companies provide, Ethernet, USB controllers, analog functions, memory blocks, ...



- Pre-verified block designs, standard bus interfaces (or adapters) ease integration - lower NREs, shorten TTM.

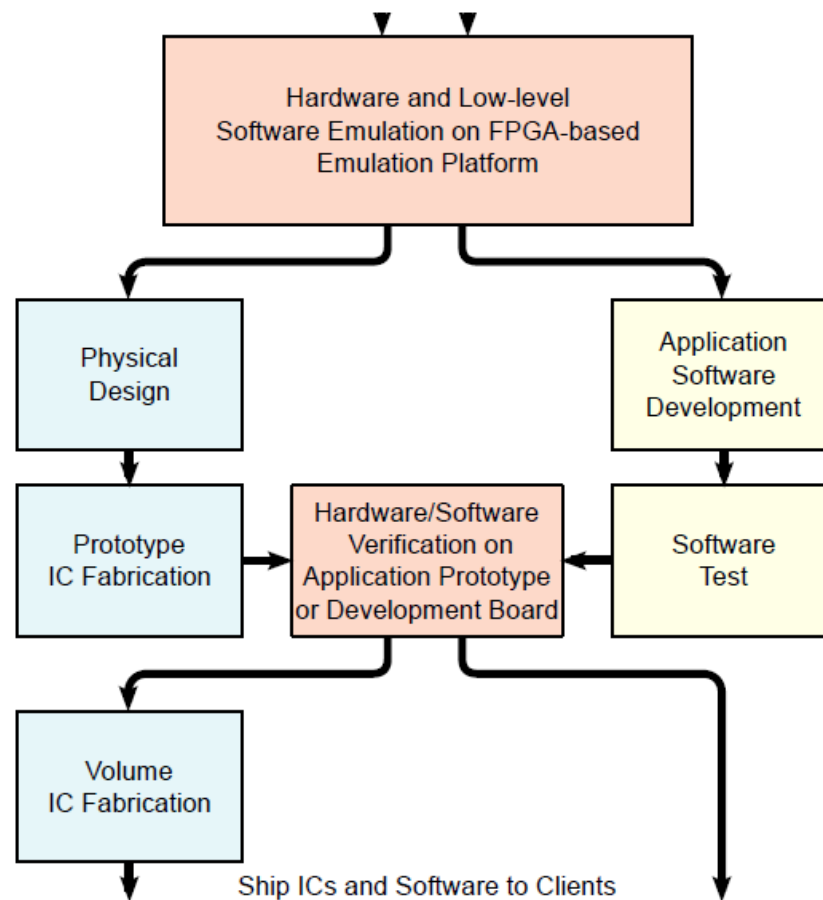
軟體也可能硬體化

SoC Hardware/Software Co-Design



於是出現了軟硬體協同設計

SoC Hardware/Software Co-Design

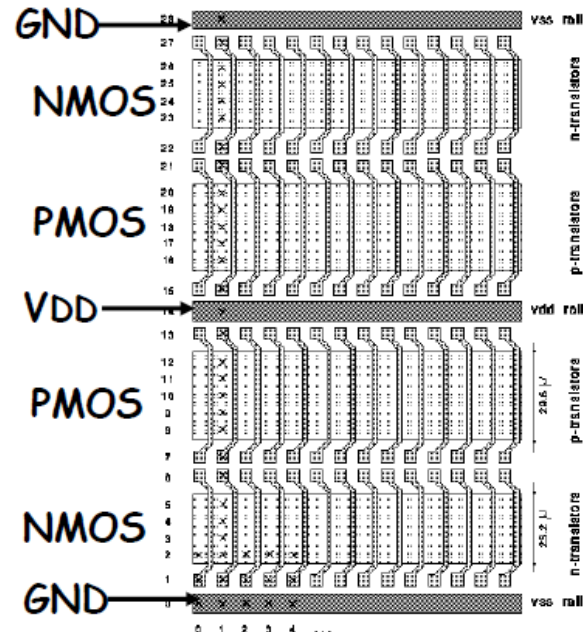


軟硬體協同設計

- 可以用在《可程式硬體》或《ASIC 專用晶片》上

Gate-Array-Based Design

- Can cut mask costs by prefabricating arrays of transistors on wafers
- Only customize metal layer for each design



- Fixed-size unit transistors
- Metal connections personalize design

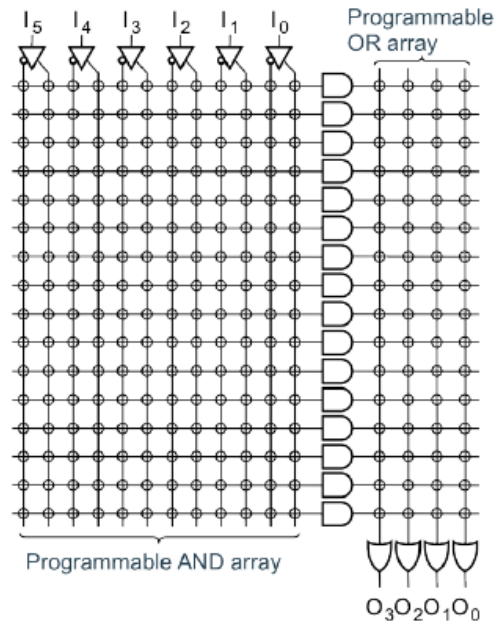
Two kinds:

- Channeled Gate Arrays
 - Leave space between rows of transistors for routing
- Sea-of-Gates
 - Route over the top of unused transistors

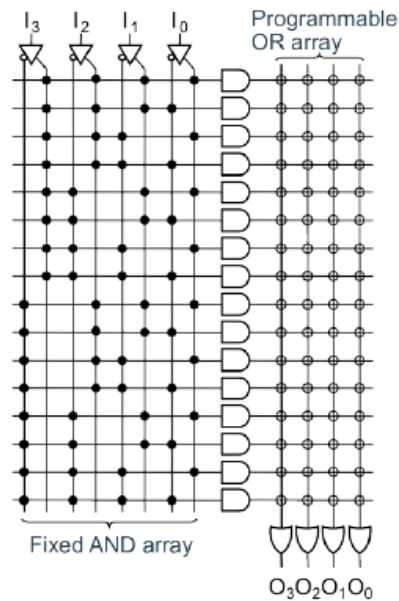
可程式硬體

- 可分為 PLA/PROM/PAL 等種類

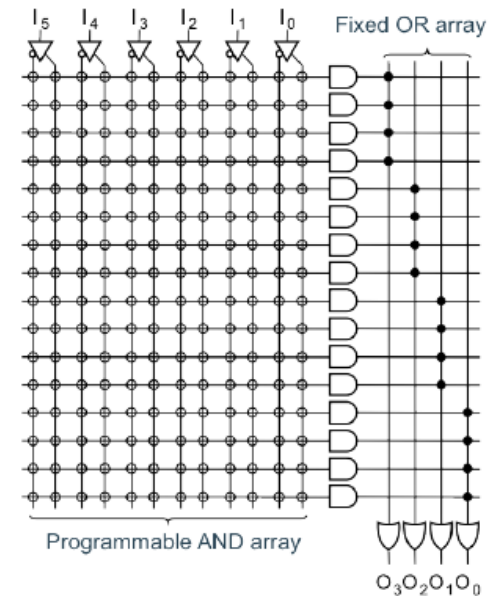
Array-Based Programmable Logic



PLA



PROM



PAL

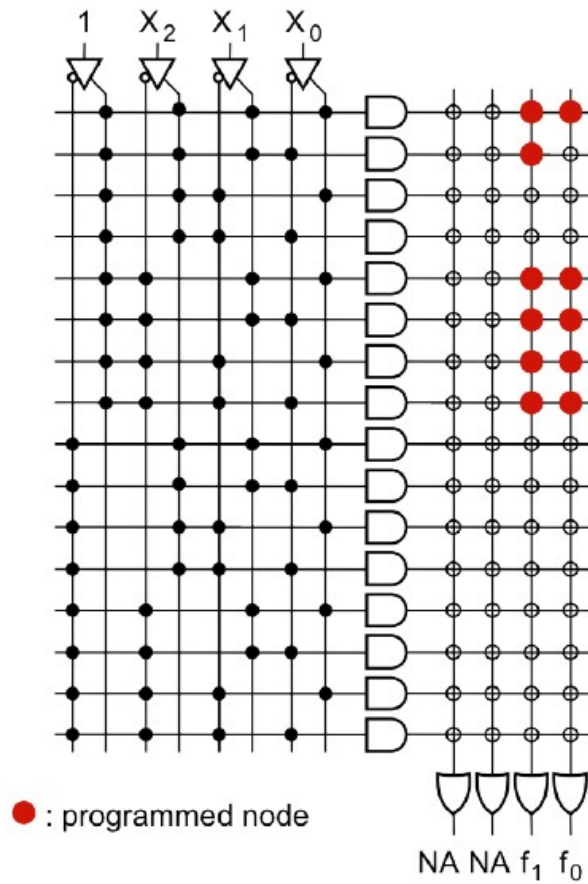
⊕ Indicates programmable connection

• Indicates fixed connection

任何邏輯算式

- 都可以用可程式硬體指定連接點設計出來

Programming a PROM



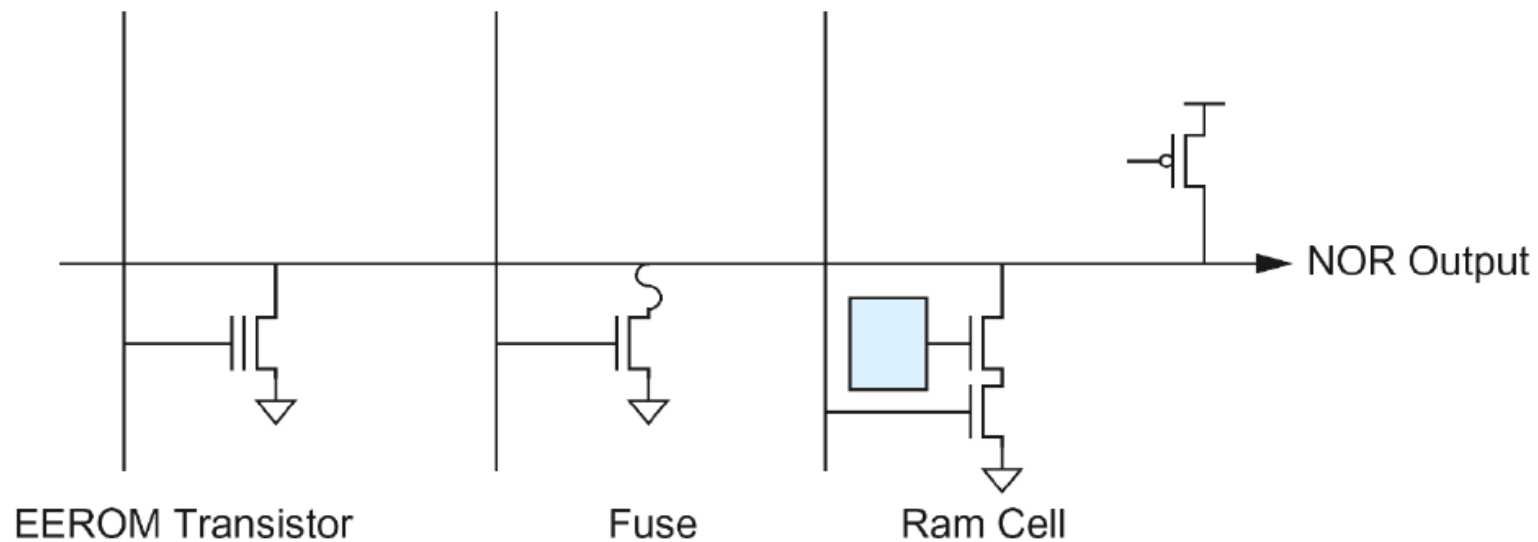
$$f_0 = x_0x_1 + \overline{x_2}$$

$$f_1 = x_0x_1x_2 + \overline{x_2} + \overline{x_0}x_1$$

其中的連接點

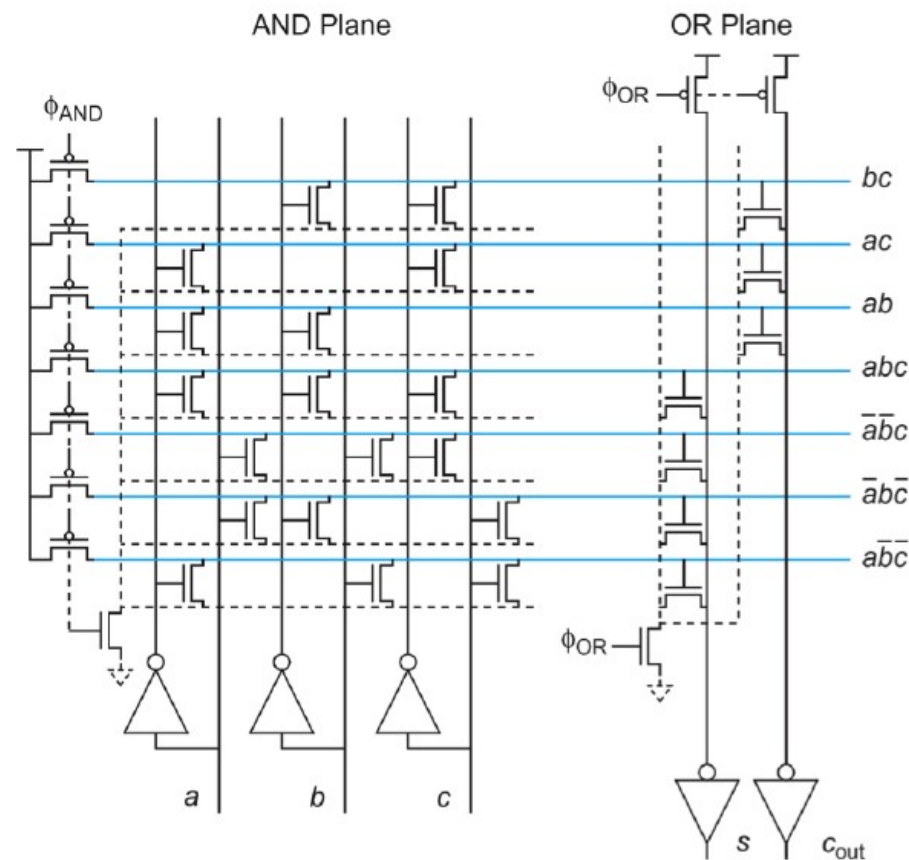
- 有分為 EEPRON/Fuse/Ram Cell 等類型

Various Programmability Options for PLA/PROM NOR Structure



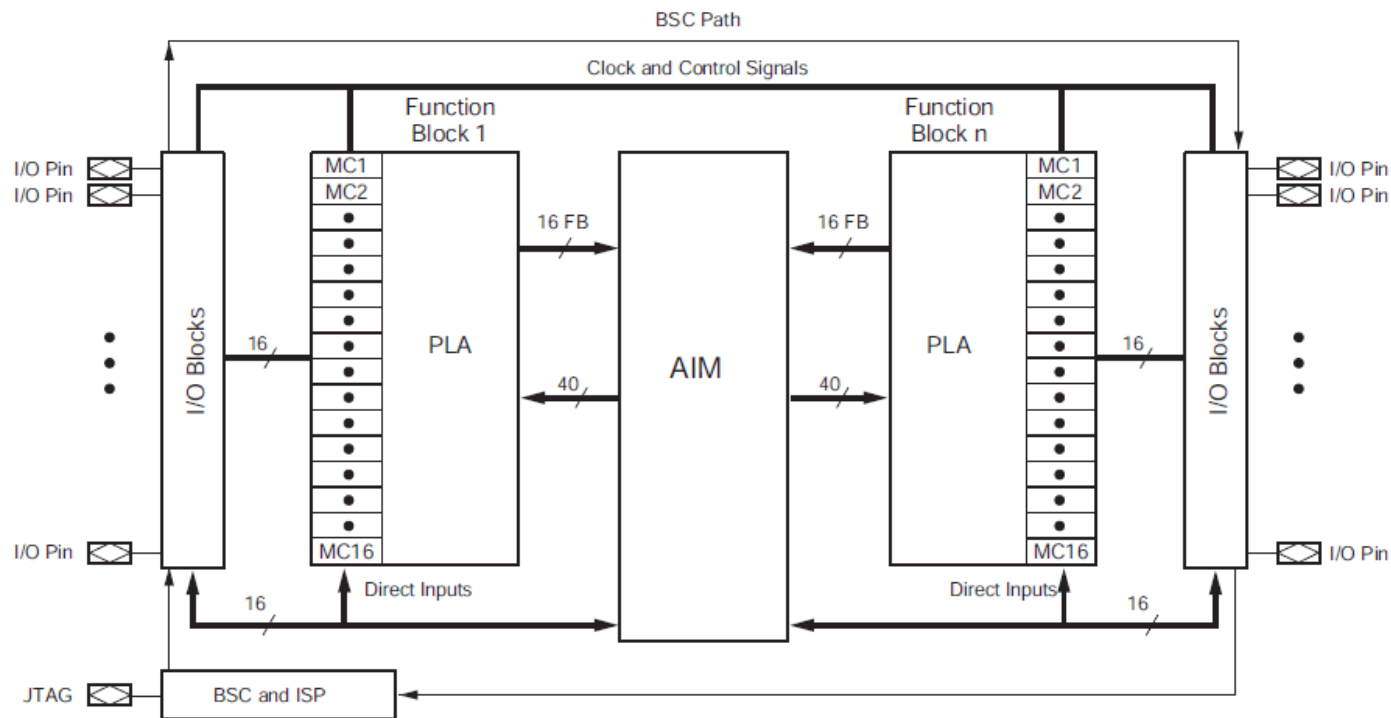
更細緻的 PLA 電路 MOS 結構長這樣

PLA Implemented with Dynamic Logic



由 PLA 組合成的 CPLD 長這樣

Modern Complex Programmable Logic Devices

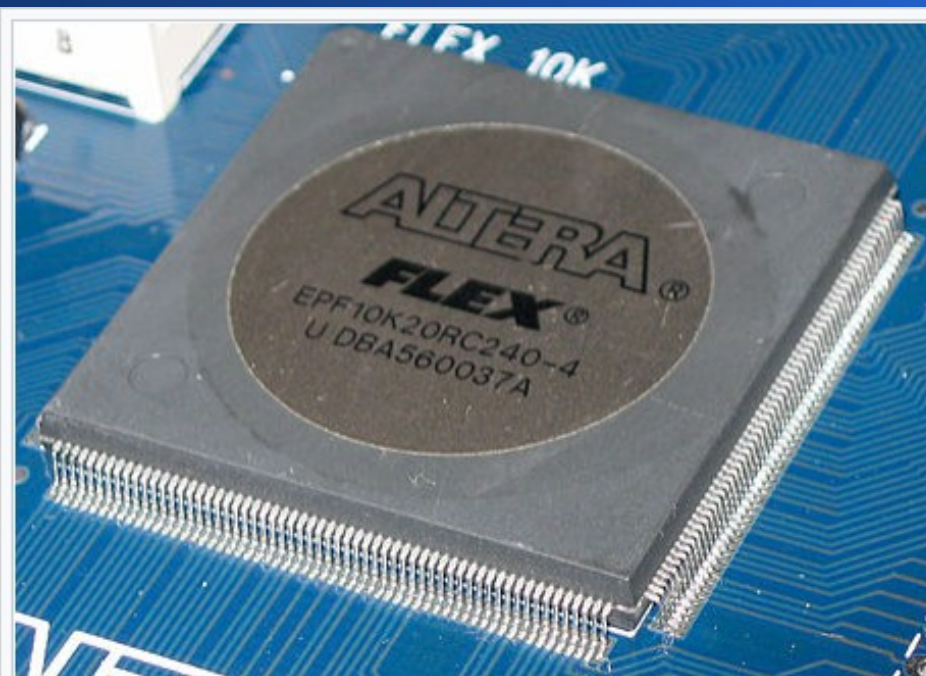


Xilinx CoolRunner-II CPLD: 2–32 PLAs integrated on a single chip

PLAs support 40 inputs, 56 product terms, and 16 output terms

PLAs chained together through a programmable “Advanced Interconnect Matrix”

而容量更大的 FGPA 可程式電路



一個有20,000個單元的Altra FPGA

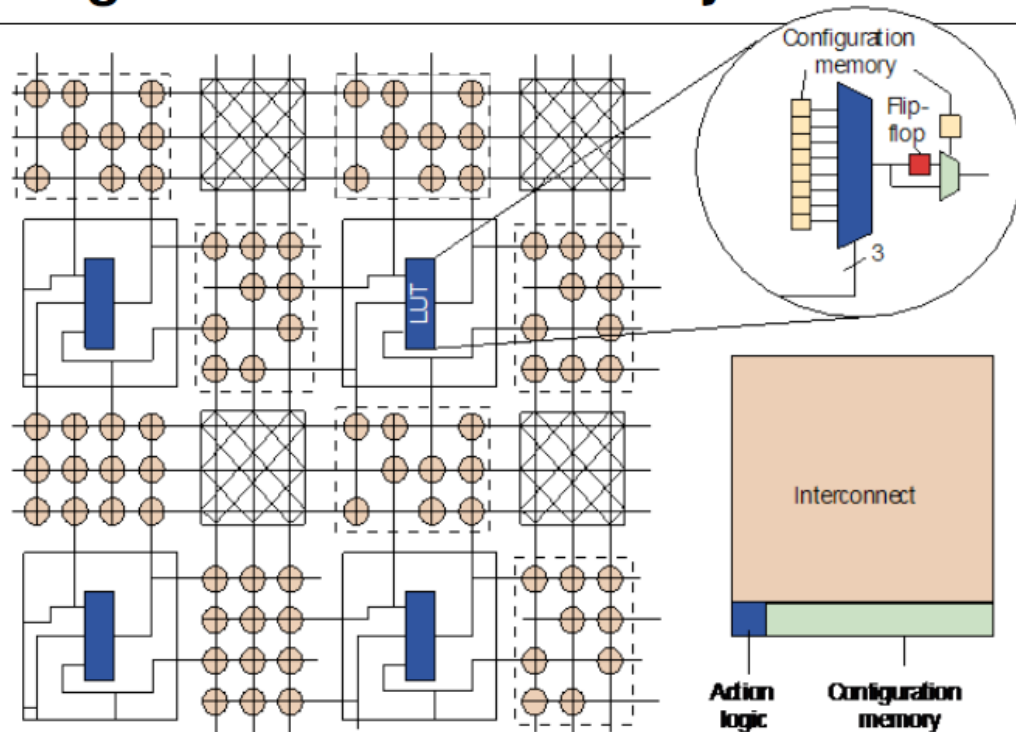


其基礎為 LUT

- 每個 LUT 是一張小真值表，連接成 CLB 區塊

Field-Programmable Gate-Arrays

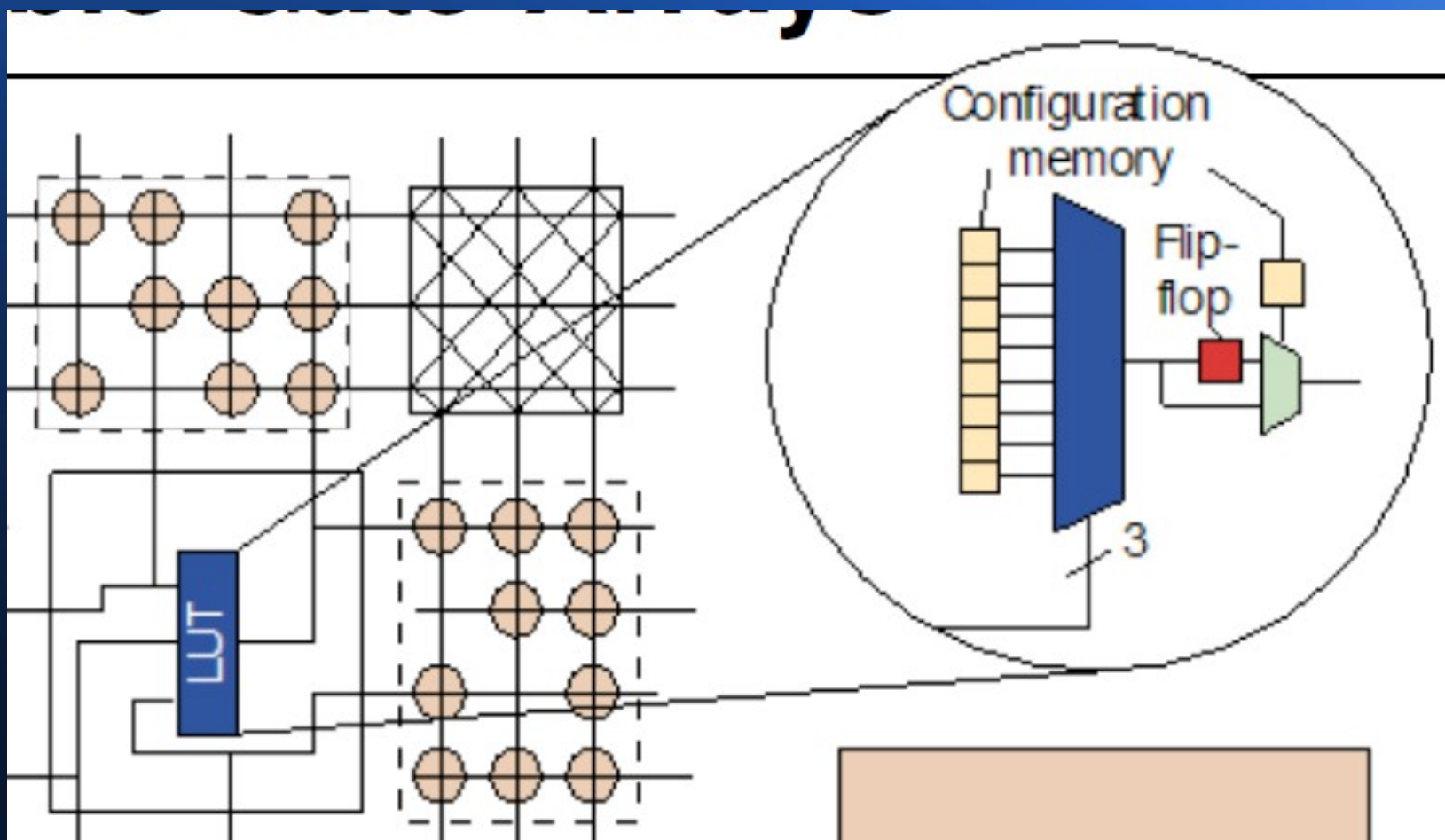
- Two-dimensional array of simple logic- and interconnection-blocks.
- Typical architecture: LUTs implement any function of n -inputs ($n=3$ in this case).
- Optional Flip-flop with each LUT.



- ▶ Fuses, EPROM, or Static RAM cells are used to store the "configuration".
 - ▶ Here, it determines function implemented by LUT, selection of Flip-flop, and interconnection points.
- ▶ Many FPGAs include special circuits to accelerate adder carry-chain and many special cores: RAMs, MAC, Enet, PCI, SERDES, ...

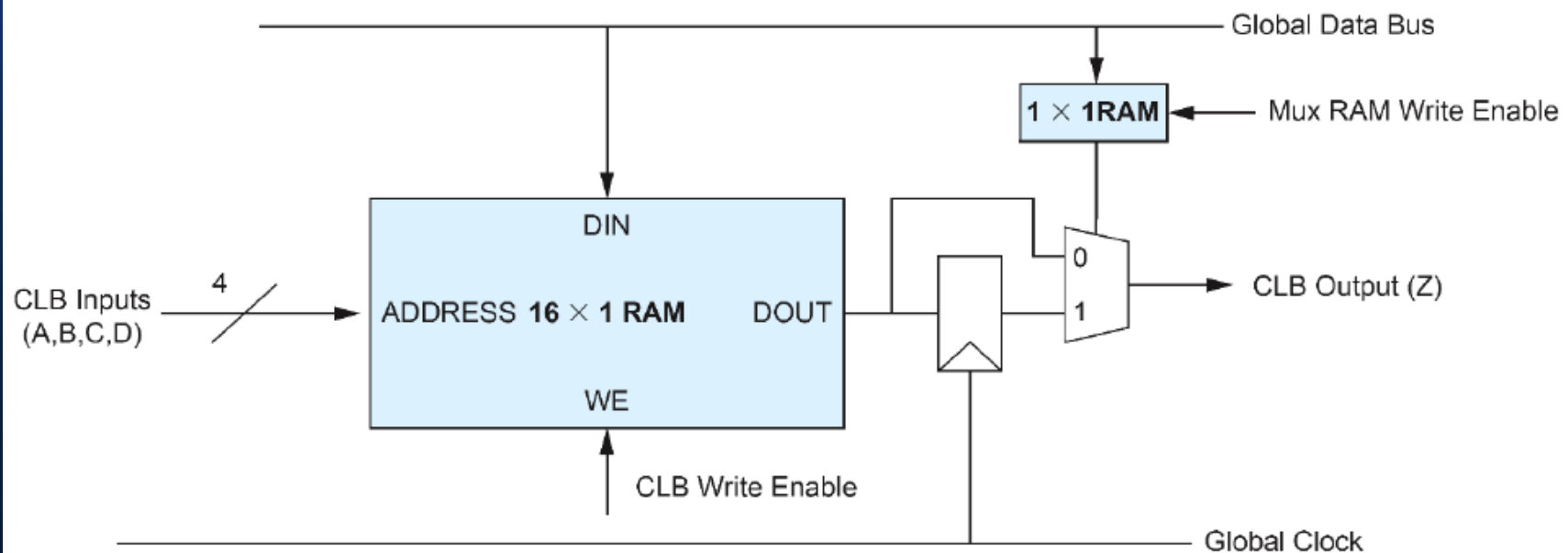
Adapted from [Asanovic'11]

LUT 的結構長這樣



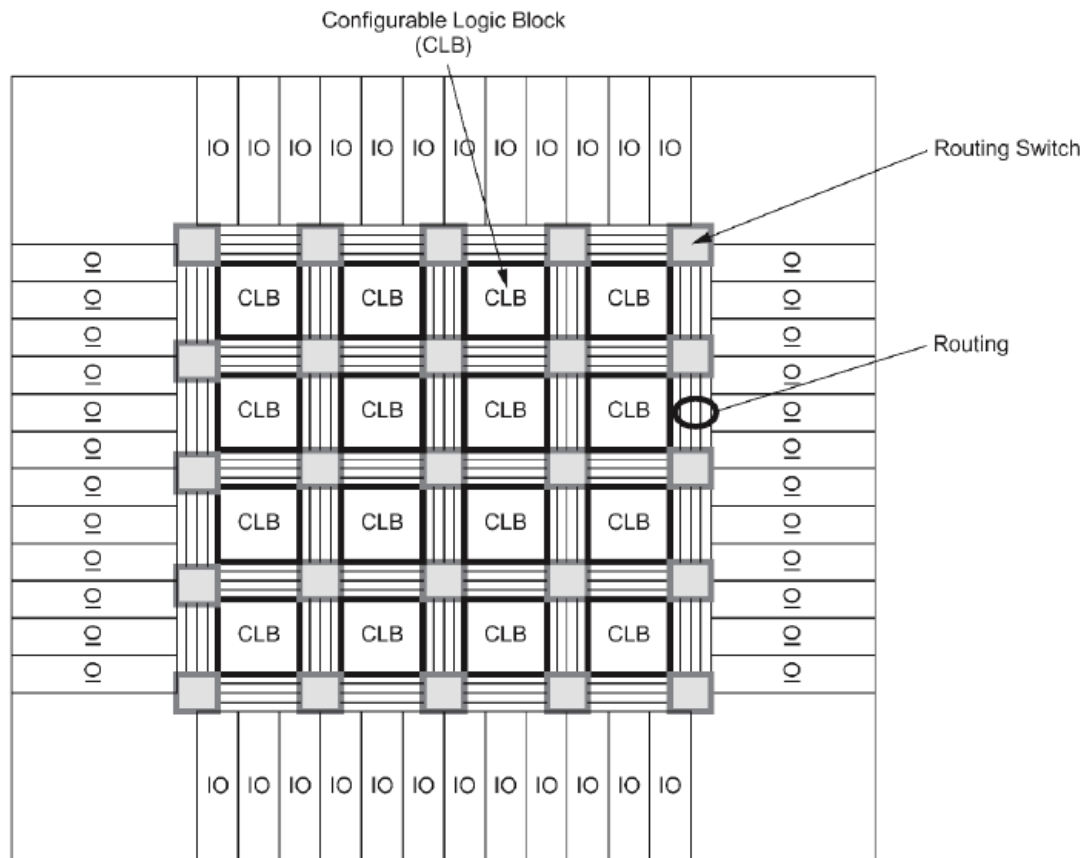
CLB 的結構長這樣

Simplified FPGA Combinational Logic Block



很多 CLB 區塊組合成 FPGA 晶片

Simplified FPGA Floorplan



雖然生產量很大時，ASIC 較便宜

- 但是在十萬片以下時，反而用 FPGA 比較划算

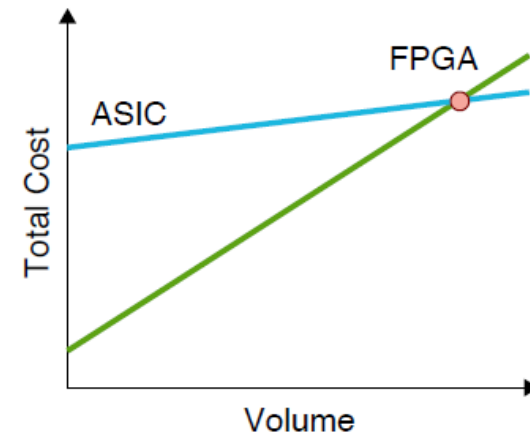
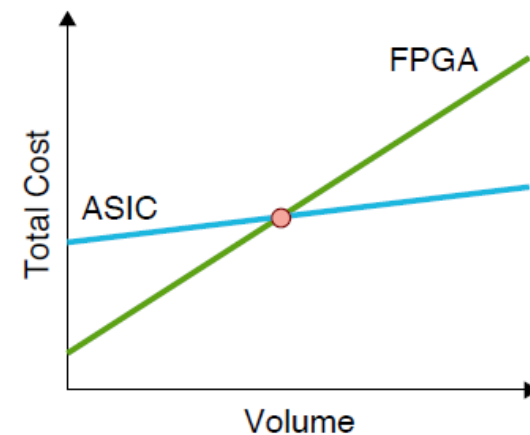
ASIC vs. FPGA

▶ Traditional Argument

- ▶ ASIC: High NRE (\$2M for 0.35 μm chip), low marginal cost, best efficiency
- ▶ FPGA: Low NRE, high marginal cost, lower efficiency
- ▶ Cross-over point: around 10,000

▶ Current Trends

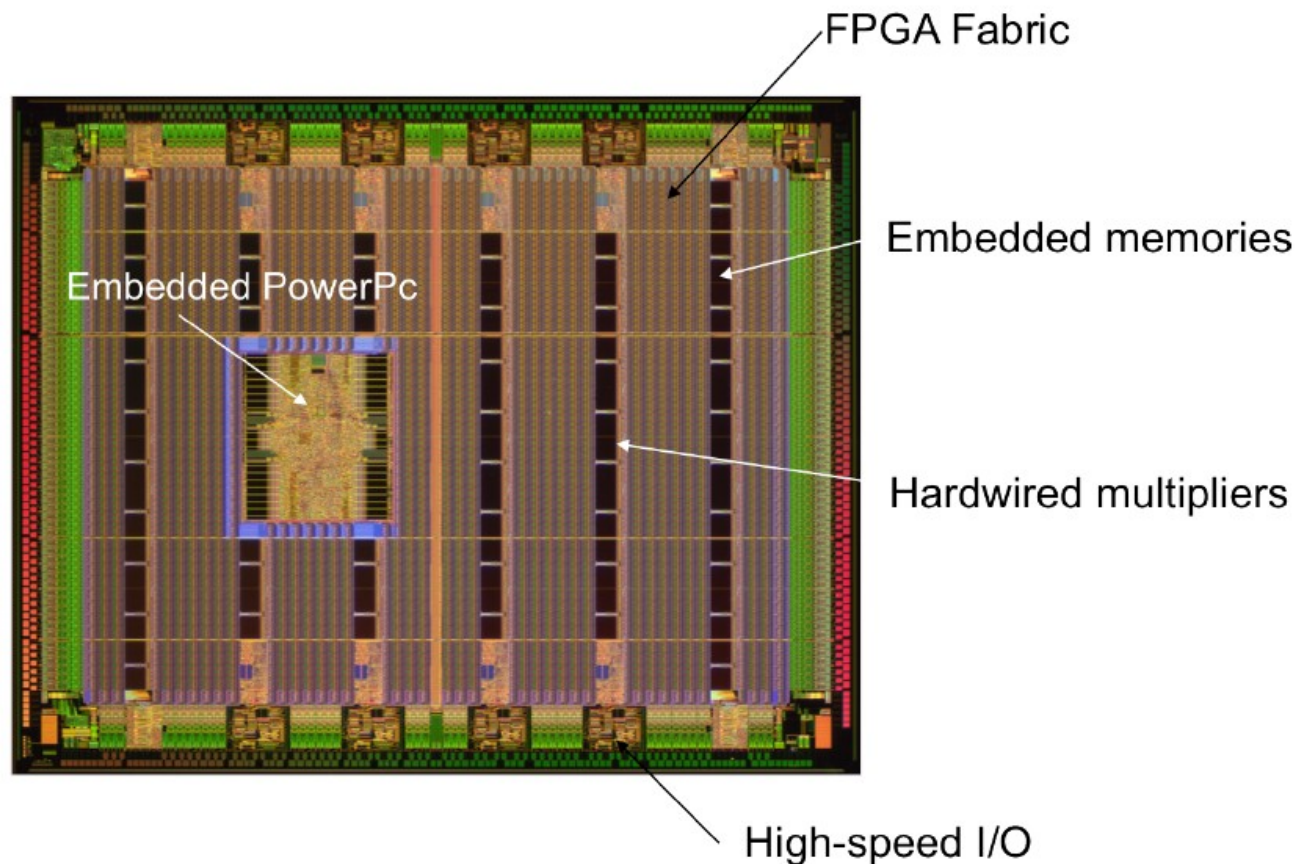
- ▶ ASIC: Increasing NRE (\$40M for 90 nm chip) due to design costs, verification costs, mask costs, etc
- ▶ FPGA: Better able to track Moore's law, integrating fixed function blocks
- ▶ Cross-over point: around 100,000



當然也可以將 CPU 放入 FPGA 中

- 這樣 ASIC 和 FPGA 就融合為一了

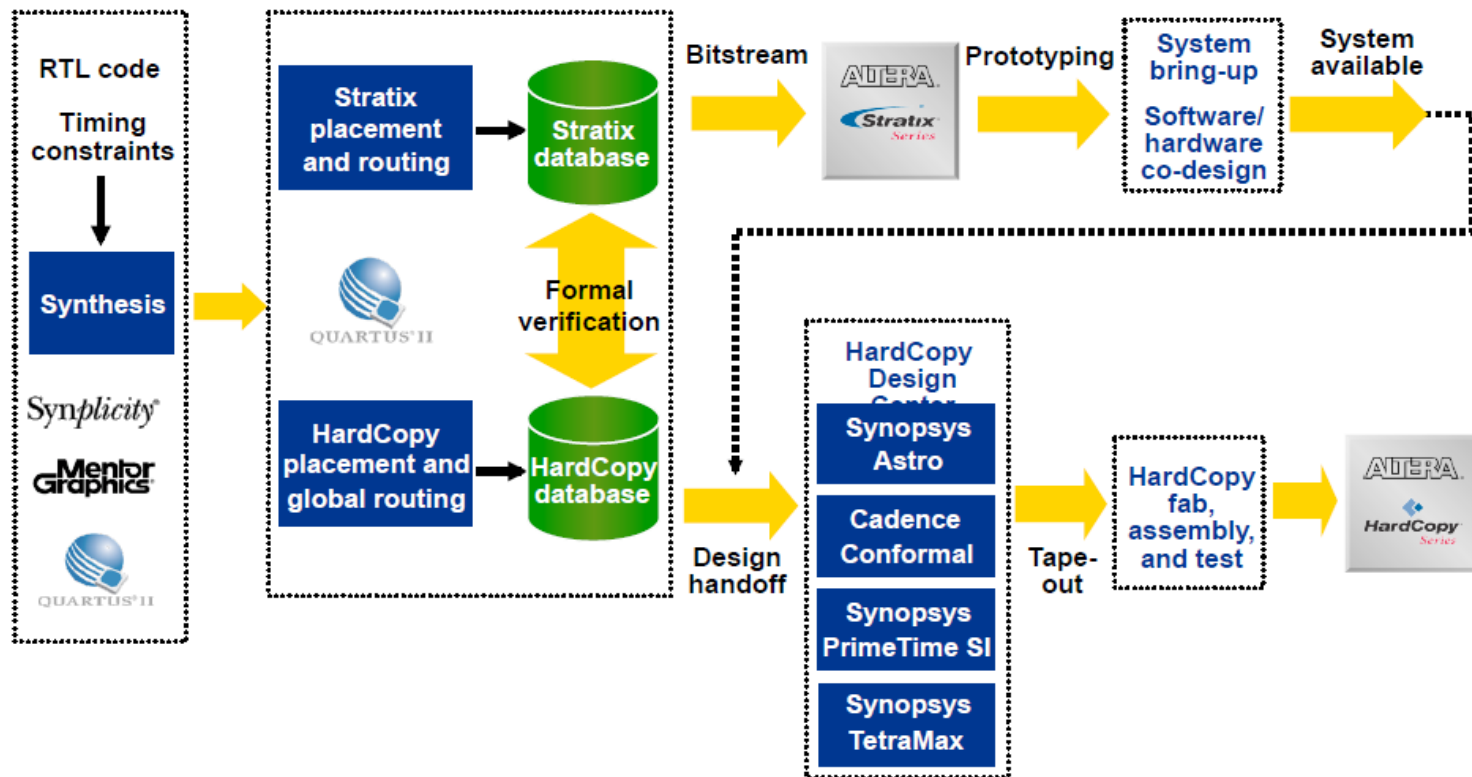
Xilinx Vertex-II Pro – FPGA w/ Hard Processor



Altera 的 HardCopy 系列產品

- 也是整合了 ASIC+FPGA 的晶片，兼具速度與彈性

Altera HardCopy – FPGA to Gate-Array-Like Tapeout



我們可以將 IC 為下列三類

- 量產型的 Full-Custom ASIC
- 客製化的 Semi-Custom ASIC
- 可程式化的 CPLD/FPGA 等 IC

如果我們比較速度

- 量產型的 Full-Custom ASIC 最快
- 客製化的 Semi-Custom ASIC 會慢一些
- 可程式化 CPLD/FPGA 等 IC 最慢

為何 Custom IC 會比 ASIC 慢呢？

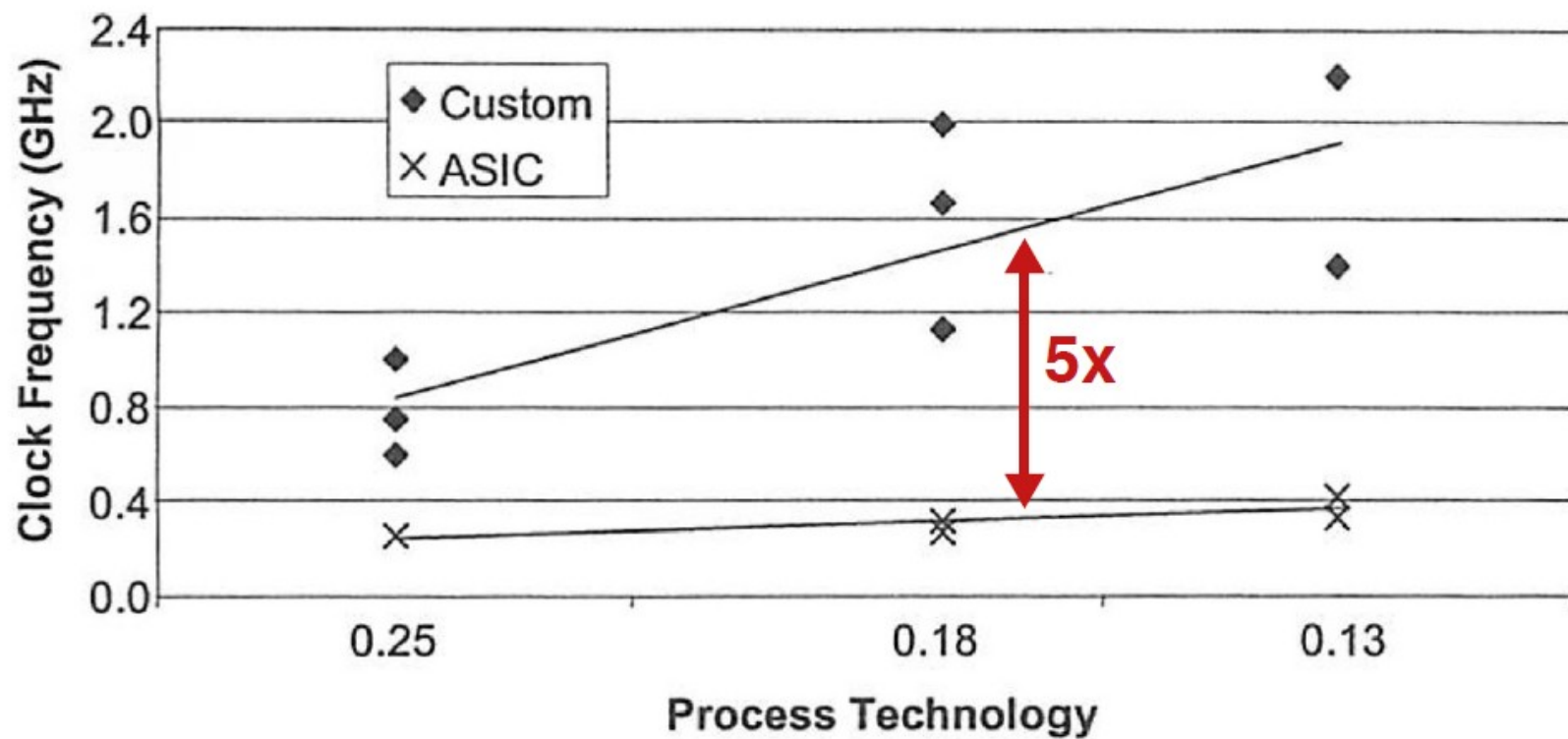
- 不是都一樣交給台積電代工嗎？

這兩者的差別

- Full-Custom ASIC 需要設計人員完成所有電晶體和互連線的詳細版圖
- Semi-Custom ASIC 則是採用標準元件建構出來的，耗費的人力較少

以 Clock 頻率而言大約差了五倍

Clock Frequency for High-Performance Processors

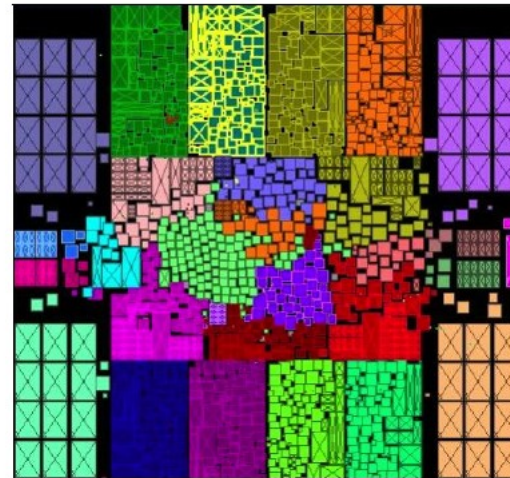


但是對 Semi-Custom 而言

- 有一些方法可以提升效能，例如把元件拼接後再跑優化軟體，例如優化面積（Placement）

Physical Synthesis

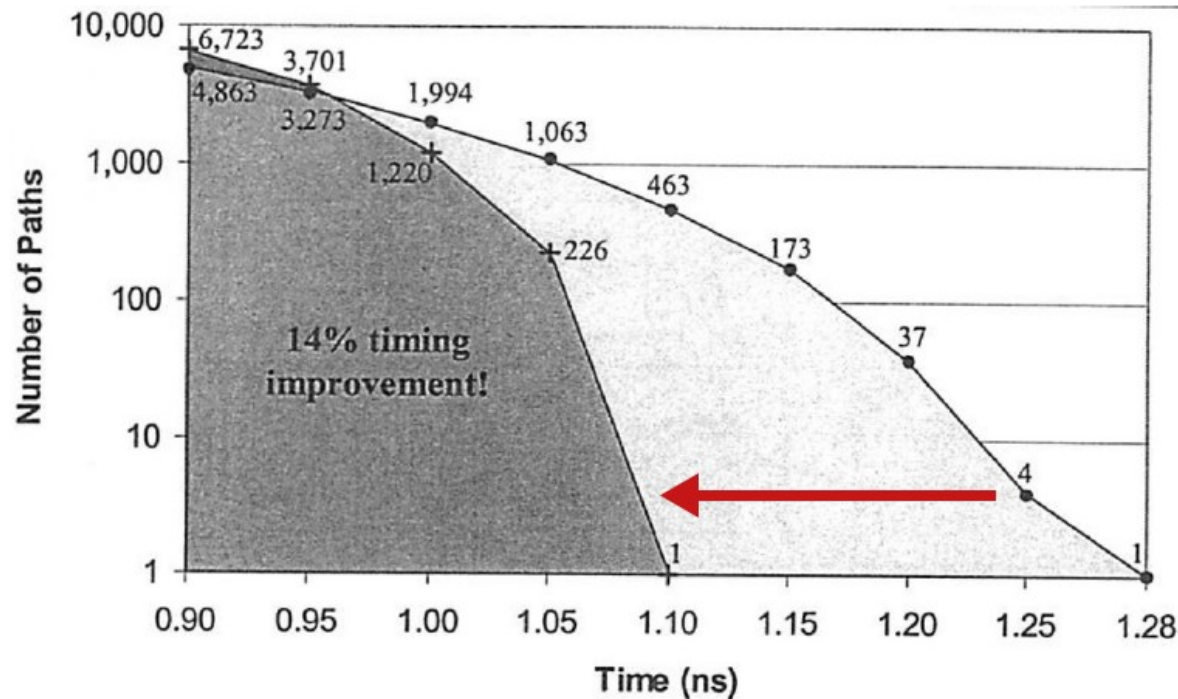
- ▶ Physical synthesis tool performs first-iteration mapping to logic gates before performing a preliminary “rough” placement
- ▶ Preliminary placement incorporates macro-floorplanning and does not need to produce a “legal” fine-grain placement of each cell
- ▶ Preliminary placement is used to create more accurate wire load estimates, and then tool re-synthesizes design
- ▶ Physical synthesis tool iterates until meets timing



這種局部優化

- 也能降低晶片面積並改善效能

Benefit of Automatic Resizing

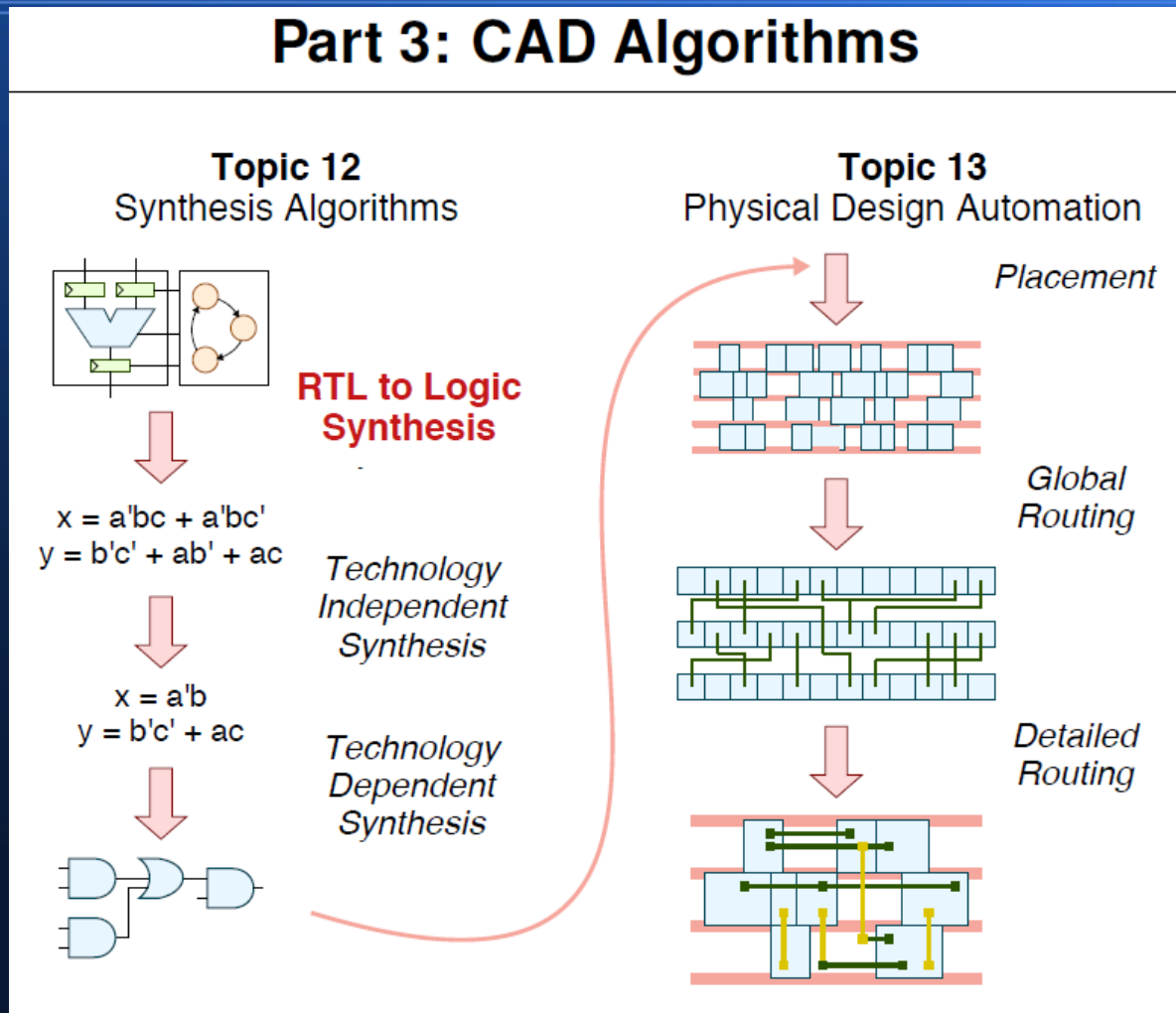


Improvement for customized standard-cell generation during logic synthesis for an adder design in 0.18 μm

接下來

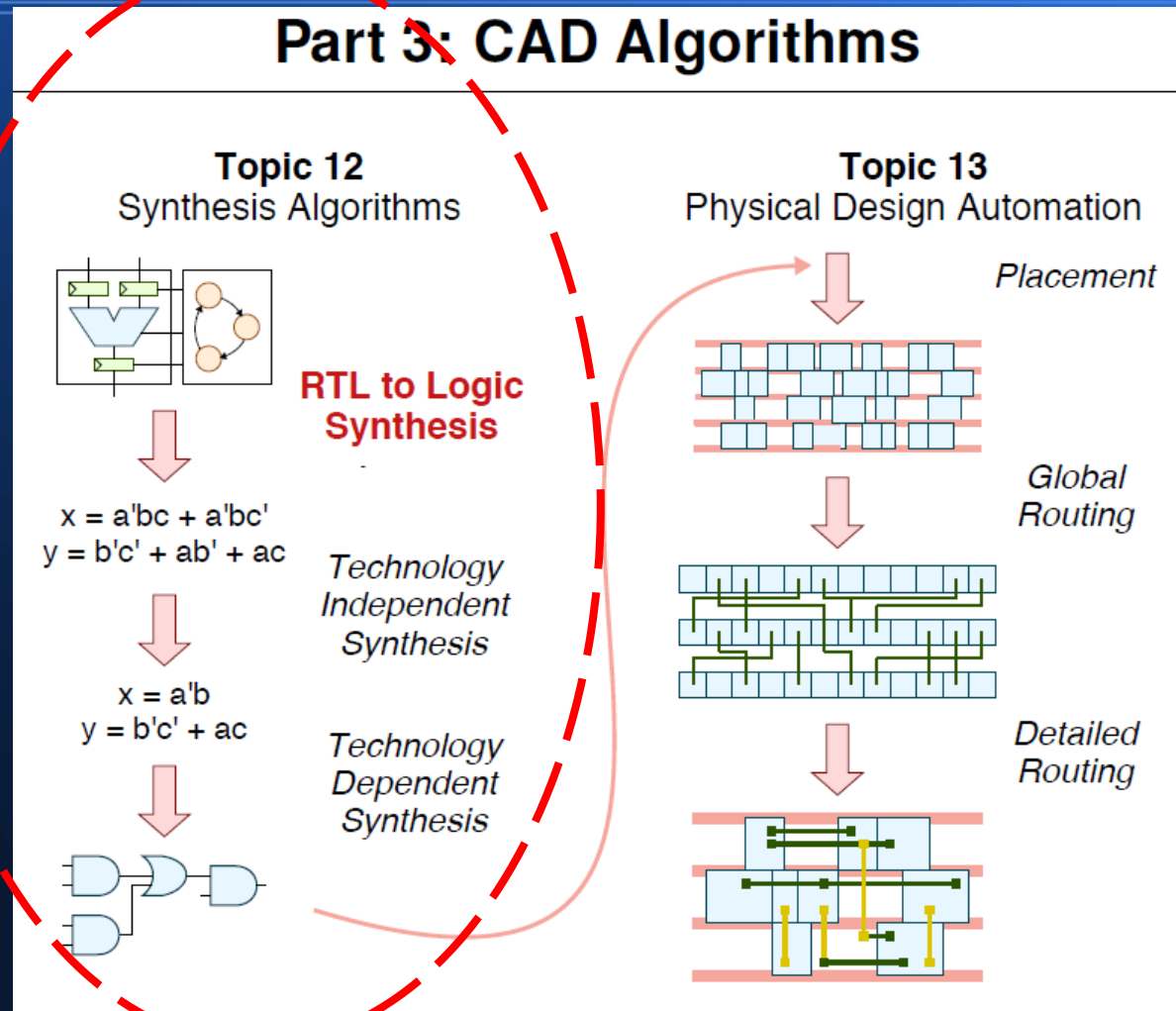
- 就讓我們看看 EDA 工具是如何對 IC 進行優化的

Part 3: CAD Algorithms



首先看看 Synthesis Algorithm

- 中文是合成（或綜合）演算法



讓我們看看

- 維基百科中邏輯綜合的歷史

邏輯綜合的發展可以追溯到喬治·布林 (1815 - 1864) 對邏輯代數的研究 (邏輯代數現在也被稱為「布林代數」) 。1938年，克勞德·香農展示了如何使用邏輯代數來描述電路開關切換的過程。^[1]在早期，邏輯設計牽涉了對真值表的處理 (如利用卡諾圖) 。通過將一系列規則將卡諾圖上的某些項進行合併，可以得到最小化的邏輯，即邏輯式可以得到簡化。人工進行上述的操作可以處於四到六個變量的卡諾圖。但是以圖形表示的卡諾圖不適合算法的導出，因此人們需要研究代數形式的邏輯化簡算法。^{[2]:189}

對邏輯最小化進行自動化的第一步是引入奎因 - 麥克拉斯基算法，它可以在計算機上執行。進行邏輯簡化的目的是得到使用最少數量的邏輯裝置，並精確地對表達原始邏輯式的意義。人們可以使用卡諾圖來完成變量較少的邏輯函數的化簡。現在，Espresso啟發式邏輯簡化器 (Espresso heuristic logic minimizer) 成為了完成這一過程更有效的工具。早期數位邏輯的另一個研究領域是對有限狀態機的簡化和編碼，這一工作對於當時的設計者來說並不是件容易的工作。邏輯綜合的應用大多於數位計算機設計相關。IBM和貝爾實驗室在邏輯綜合自動化工具發展的早期扮演了關鍵的角色。從分立的邏輯閘到可程式化邏輯陣列 (Programmable logic arrays, PLA) 促進了高效率的兩級邏輯最小化的發展，這是由於兩級描述中的最小化過程可以減少可程式化邏輯陣列的面積。

和編譯器技術一樣

- EDA 也會將 Verilog 轉換成 Single Assignment

Step 1: Single Assignment Form

```
1 // 1b inputs: y, z, a, q
2 // 2b inputs: f
3 // 3b inputs: g, c, b, e
4
5 wire x = y && z;
6
7 wire [2:0] b
8     = (a) ? {2'b0,x} : c;
9
10 wire [2:0] d, h;
11 always @(*) begin
12
13     d = b + e;
14     if ( q ) d = 3'b101;
15
16     if ( f )
17         h = 3'b0;
18     else
19         h = g << 1;
20
```

For each output, create exactly one assignment that is a function only of the inputs

```
1 wire x = y && z;
2
3 wire [2:0] b
4     = (a) ? {2'b0,x} : c;
5
6 wire [2:0] d
7     = (q) ? 3'b101
8         : ((a) ? {2'b0,x} : c)
9         + e;
10
11 wire [2:0] h
12     = (f) ? 3'b0 : ( g << 1 );
```

然後針對每個位元，轉成布林運算式

Step 2: Bit Blast Outputs

```
1 // 1b inputs: y, z, a, q
2 // 2b inputs: f
3 // 3b inputs: g, c, b, e
4
5 wire x = y && z;
6
7 wire [2:0] b
8     = (a) ? {2'b0,x} : c;
9
10 wire [2:0] h
11     = (f) ? 3'b0
12         : ( g << 1 );
```

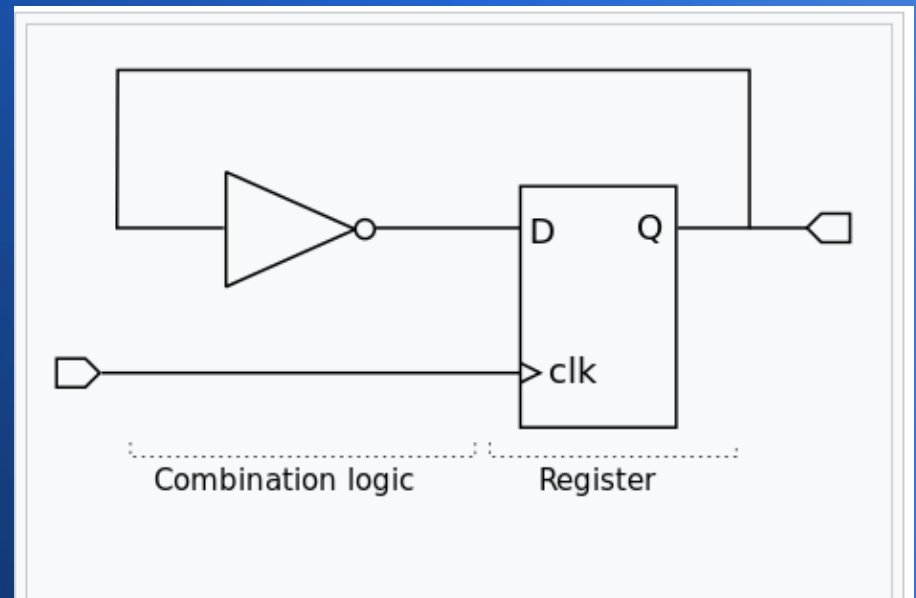
Generate separate assignment for each bit, removes arithmetic operators leaving only boolean operators (assume ternary operator is short hand for equivalent boolean operator)

```
1 wire x = y && z;
2
3 wire b[0] = (a) ? x      : c[0];
4 wire b[1] = (a) ? 1'b0 : c[1];
5 wire b[2] = (a) ? 1'b0 : c[2];
6
7 wire h[0] = (f[0]|f[1]) ? 1'b0 : 1'b0;
8 wire h[1] = (f[0]|f[1]) ? 1'b0 : g[0];
9 wire h[2] = (f[0]|f[1]) ? 1'b0 : g[1];
```

轉換後的表示法稱為 RTL

- Register Transfer Level (暫存器傳輸級)

```
D <= not Q;  
  
process(clk)  
begin  
    if rising_edge(clk) then  
        Q <= D;  
    end if;  
end process;
```



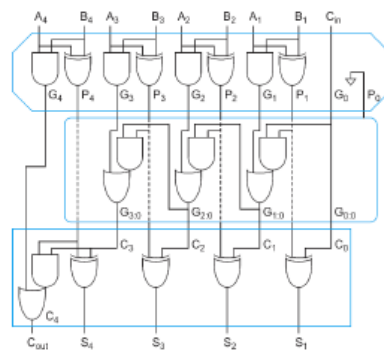
圖中所示為一個能夠按照時間脈衝交替改變輸出邏輯電平的簡單電路。左邊的反相器（帶圓點的三角形）構成了組合邏輯（Combination logic）部分，而右邊的暫存器（Register）則保持當前的狀態。

經過路徑綜合後

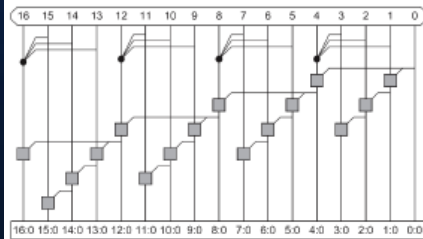
- 就得到了一組電路圖

RTL Datapath Synthesis

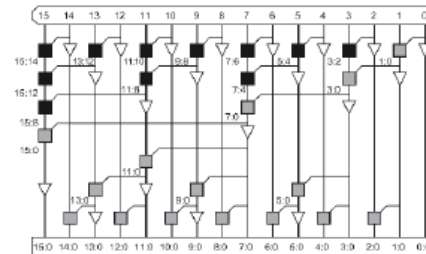
wire [15:0] a = b + c;



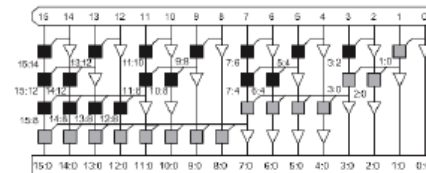
Ripple-Carry



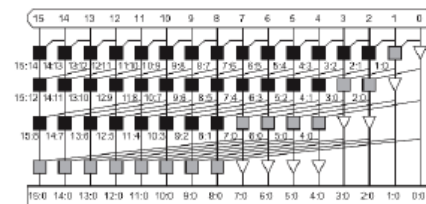
Carry Lookahead



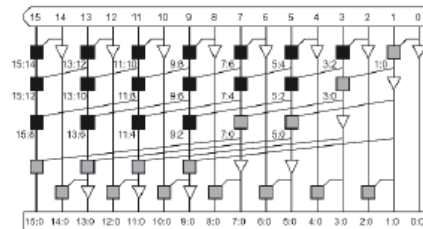
(a) Brent-Kung



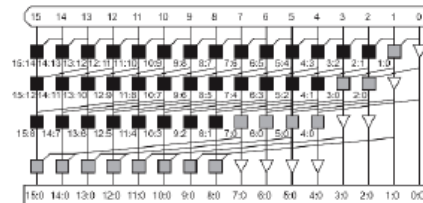
(b) Sklansky



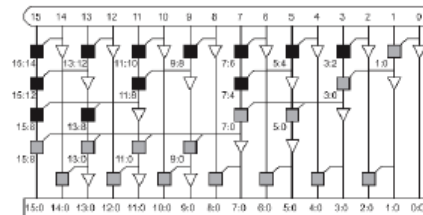
(c) Kogge-Stone



(d) Han-Carlson



(e) Knowles [2,1,1]



(f) Ladner-Fischer

Parallel-Prefix Tree-Based

綜合時牽涉到許多優化方法

- 像是兩階段布林優化，有限狀態機優化等等

Technology-Independent Synthesis

- ▶ **Two-level boolean minimization** – based on assumption that reducing the number of product terms in an equation and reducing the size of each product term will result in a smaller/faster implementation
- ▶ **Optimizing finite-state machines** – look for equivalent FSMs (i.e., FSMs that produce the same outputs give the same sequence of inputs) that have fewer states
- ▶ **FSM state encodings** – minimize implementation area (= size of state storage + size of logic to implement next state and output functions).

Note that none of these optimizations are completely isolated from the target technology, but experience has shown that it's advantageous to reduce the size of the problem as much as possible before starting the technology-dependent optimizations.

數位邏輯裡的卡諾圖

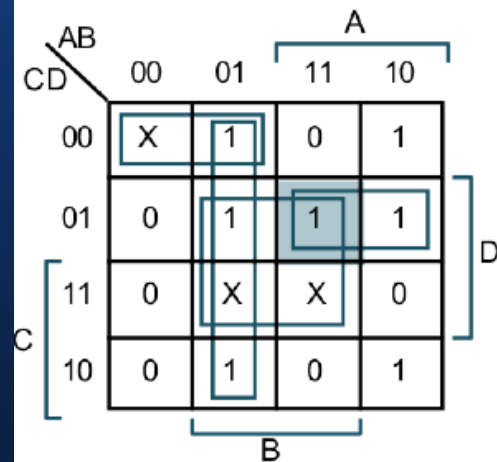
- 也是一種邏輯綜合優化方法

Karnaugh Map Method Review

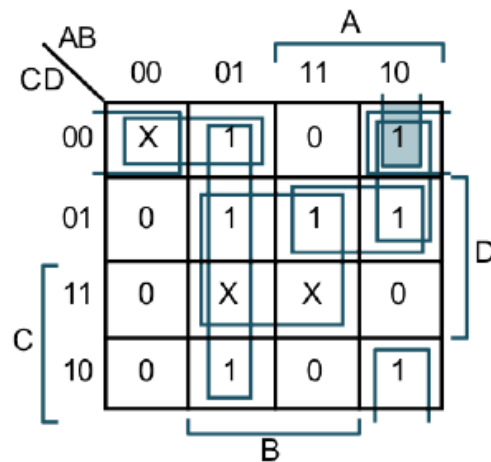
- ▶ 1. Choose an element of ON-set not already covered by an implicant
- ▶ 2. Find “maximal” groups of 1’s and X’s adjacent to that element. Remember to consider top/bottom row, left/right column, and corner adjacencies. This forms prime implicants.
- ▶ Repeat steps 1 and 2 to find all prime implicants
- ▶ 3. Revise the 1’s elements in the K-map. If covered by single prime implicant, it is *essential*, and participates in the final cover. The 1’s it covers do not need to be revisited.
- ▶ 4. If there remain 1’s not covered by essential prime implicants, then select the smallest number of prime implicants that cover the remaining 1’s

以下是一些卡諾圖的範例

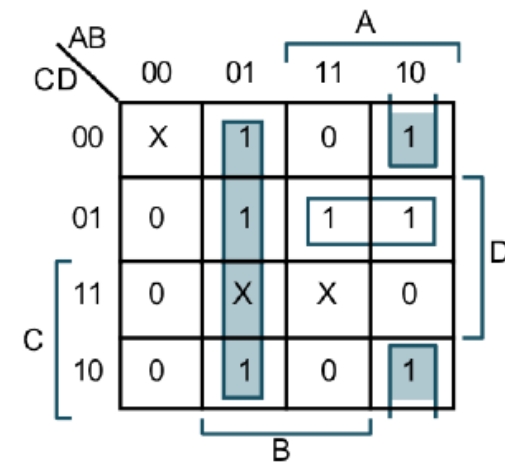
Karnaugh Map Method Example



**Primes around
 $A B C' D$**



**Primes around
 $A B' C' D'$**



**Essential Primes
with Min Cover**

但是卡諾圖只適合四個變數以下的電路

把函式包含的所有最小項，以「1」填入變數卡諾圖對應編號的小格內。

例 1: “表決” 邏輯函數: $Z(A, B, C) = \sum m(3, 5, 6, 7)$
其“卡諾圖”:

BC \ A	00	01	11	10
0			1	
1		1	1	1

例 2:

作 $Z = AB + \bar{A}C$ 的卡諾圖:

BC \ A	00	01	11	10
0		1	1	
1			1	1

而且卡諾圖是兩層邏輯

- 一層 AND ， 一層 OR

- 例如

$$\begin{aligned}x &= a'bc + a'bc' \\y &= b'c' + ab' + ac\end{aligned}$$

以下方法也是兩層邏輯的優化法

- 像是
 - Quine-McCluskey (QM) Method
 - Heuristic Espresso Method

Quine–McCluskey (QM) Method

- 請參考維基百科

Quine-McCluskey (QM) Method

- ▶ Quine-McCluskey method is an exact algorithm which finds a minimum-cost sum-of-products implementation of a boolean function
- ▶ Four main steps
 - ▷ 1. Generate prime implicants
 - ▷ 2. Construct prime implicant table
 - ▷ 3. Reduce prime implicant table
 - a. Remove essential prime implicants
 - b. Row dominance
 - c. Column dominance
 - d. Iterate at this step until no further reductions
 - ▷ 4. Solve prime implicant table

Espresso Algorithm

Espresso Algorithm

- ▶ 1. EXPAND implicants (choice of which implicants requires heuristic)
- ▶ 2. REMOVE-REDUNDANCY
- ▶ 3. REDUCE implicants (choice of which implicants requires heuristic)
- ▶ 4. EXPAND implicants (choice of which implicants requires heuristic)
- ▶ 5. REMOVE-REDUNDANCY
- ▶ 6. Goto step 3

兩個算法比較如下

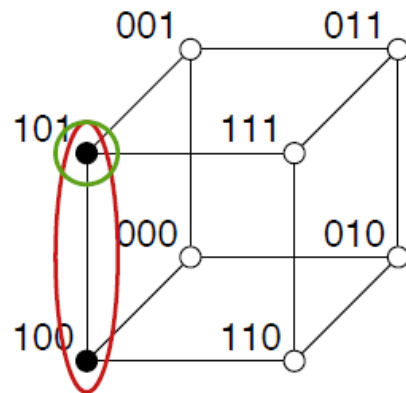
Heuristic Espresso Method

- ▶ Quine-McCluskey
 - ▷ Number of prime implicants grows rapidly with number of inputs
 - ▷ Finding a minimum cover is NP-complete (i.e., computationally expensive)
- ▶ Espresso
 - ▷ Don't generate all prime implicants (i.e., QM step 1)
 - ▷ Carefully select a subset of primes that still covers ON-set
 - ▷ Heuristically explore space of covers
 - ▷ Similar in spirit (but more structured) to finding primes in K-map

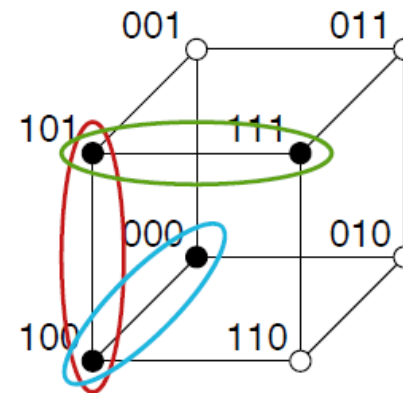
透過消除多餘者 (Redundancy)

Redundancy in Boolean Space

- ▶ Every point in boolean space is an assignment of values to variables
- ▶ Redundancy involves inclusion or covering in boolean space
- ▶ Irredundant cover has no redundancy



$g = AB'C$, $h = AB'$
 g is redundant



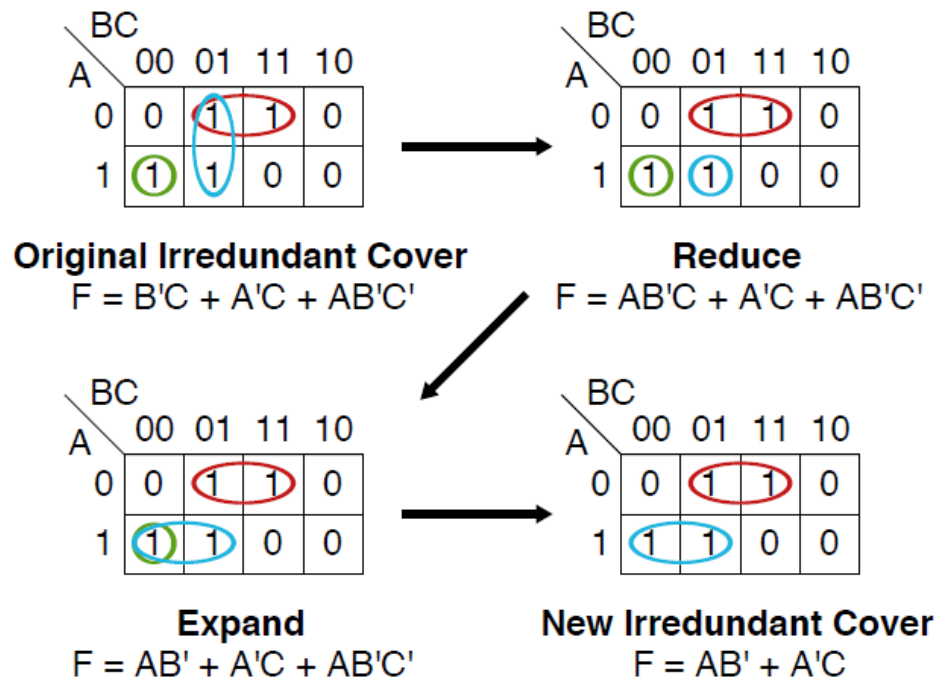
$f = AC$, $g = B'C'$, $h = AB'$
 h is redundant

Can exhaustively check if each
prime is redundant with any other prime

可以得到無多餘的涵蓋

Irredundant Covers vs. Minimal Covers

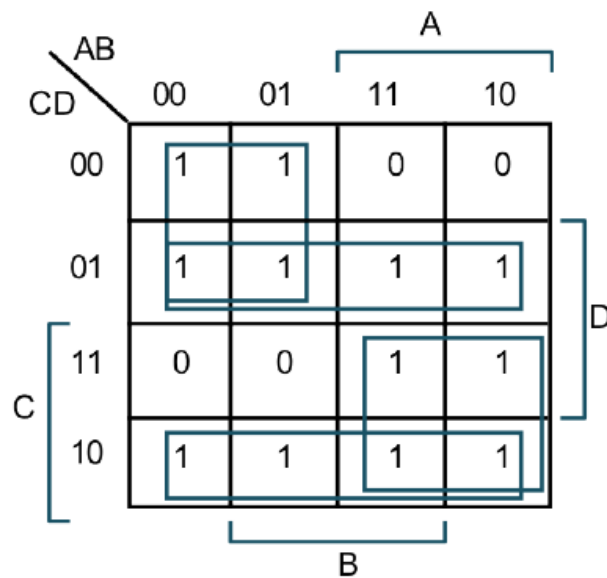
- ▶ Irredundant cover is not necessarily a minimal cover
- ▶ Can use reduce, expand, remove redundancy operations to explore space of irredundant covers



Adapted from [Zhou'02]

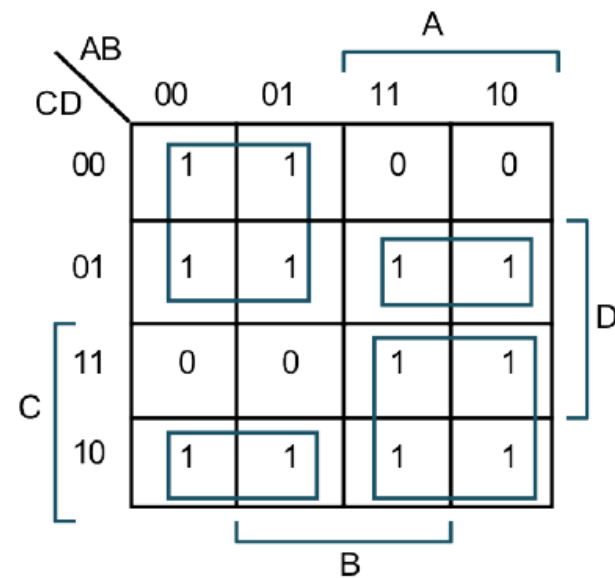
以下是 Espresso 算法的範例

Espresso Example



Initial Set of Primes found by Steps 1 and 2 of the Espresso Method

4 primes, irredundant cover, but not a minimal cover!

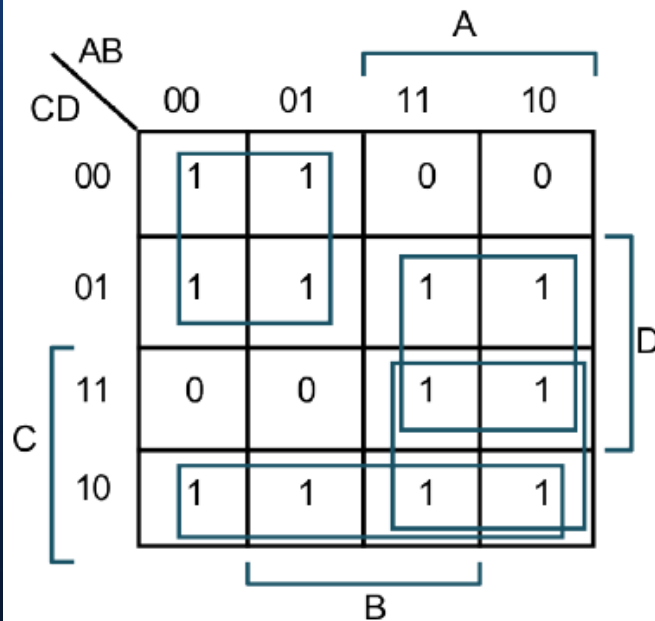


Result of REDUCE:
Shrink primes while still covering the ON-set

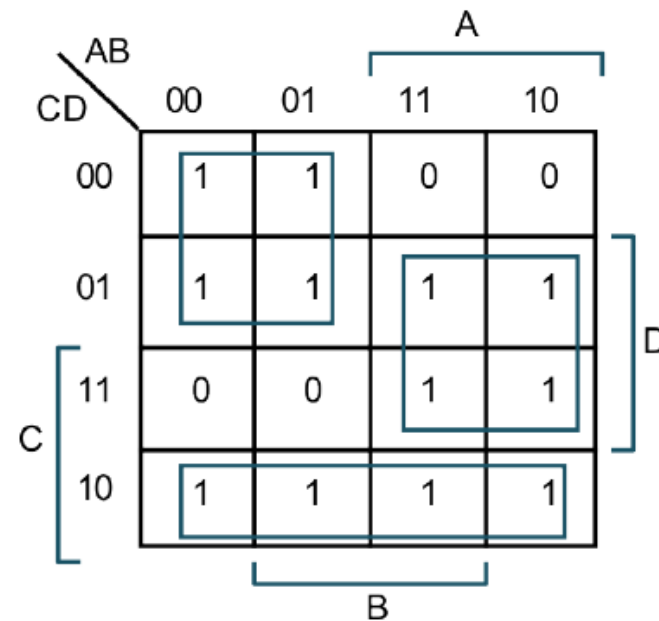
Choice of order in which to perform shrink is important

Espresso 的範例（續）

Espresso Example



Second EXPAND generates a different set of prime implicants



REMOVE-REDUNDANCY generates only three prime implicants!

兩種都是兩階層的布林式優化算法

- 但還有更複雜的多層優化的問題

Two-Level Logic vs. Multi-Level Logic

2-Level:

$$f_1 = AB + AC + AD$$

$$f_2 = \bar{A}B + \bar{A}C + \bar{A}E$$

6 product terms which cannot be shared.

24 transistors in static CMOS

Multi-level:

Note that $B + C$ is a common term in f_1 and f_2

$$K = B + C$$

$$f_1 = AK + AD$$

$$f_2 = \bar{A}K + \bar{A}E$$

3 Levels

20 transistors in static CMOS
not counting inverters

在此就不介紹多層優化的算法了

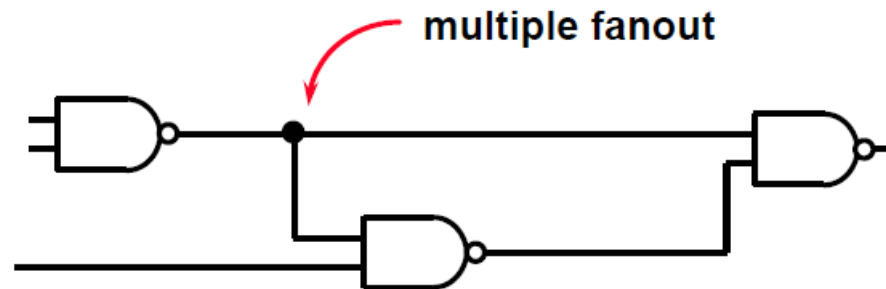
Two-Level Logic vs. Multi-Level Logic

- ▶ Two-Level Logic
 - ▷ At most two gates between primary input and primary output
 - ▷ Real life circuits: programmable logic arrays
 - ▷ Exact optimization methods: well-developed, feasible
 - ▷ Heuristic methods also possible
- ▶ Multi-Level Logic
 - ▷ Any number of gates between primary input and primary output
 - ▷ Most circuits in real life are multi-level
 - ▷ Smaller, less power, and (in many cases) faster
 - ▷ Exact optimization methods: few, high complexity, impractical
 - ▷ Heuristic methods pretty much required

從邏輯式到電路

Technology Mapping

- ▶ Once minimized logic equations, next step is to map each equation to the gates in the target standard cell library
- ▶ Classic approach uses DAG covering (K. Keutzer)
 - ▷ “Normal Form” : use basic gates (e.g., 2-input NAND gates, inverters)
 - ▷ Represent logic equations as input netlist in normal form (subjective DAG)
 - ▷ Represent each library gate in normal form (primitive DAG)
 - ▷ GOAL: Find a min cost covering of subjective DAG by primitive DAGs
- ▶ Sound algorithmic approach, but is NP-hard optimization problem

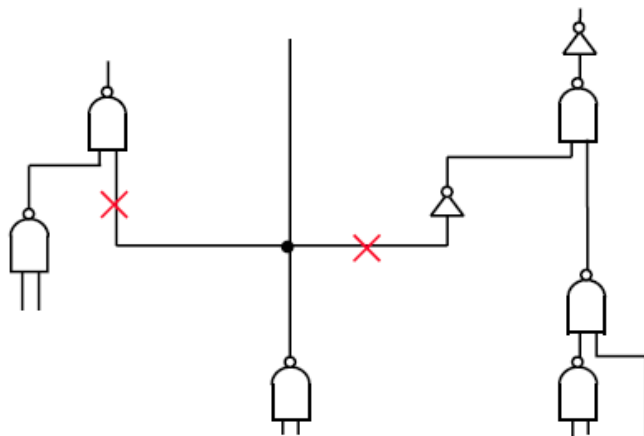


樹狀結構比較好處理

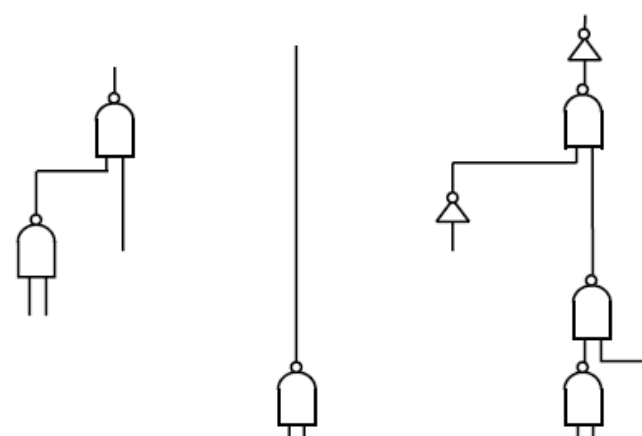
- 可以透過分割成子樹來處理

Tree Heuristic Transformation

If subject and primitive DAGs are trees, efficient algorithm can find optimum cover in linear time via dynamic programming, so use tree covering heuristic approach by partitioning graph into subtrees



Original Graph



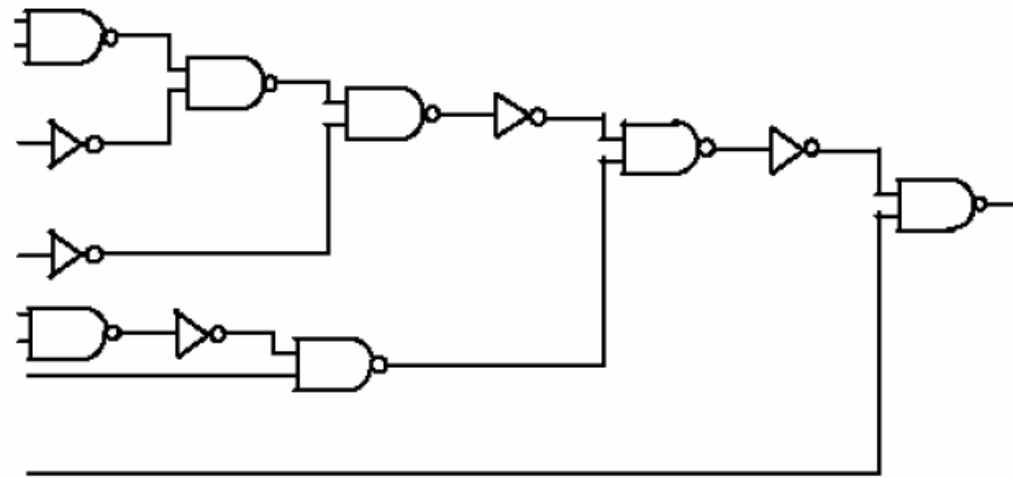
Partitioned Graph

目的是要尋找好的映射關係

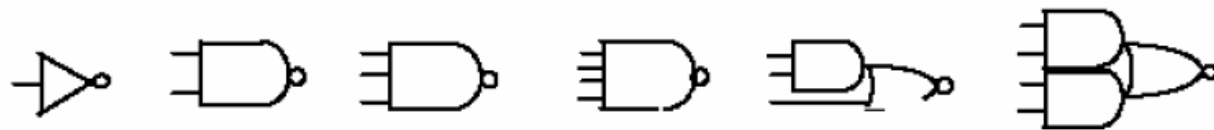
- 將邏輯式映射到元件庫

Technology Mapping Example

Problem statement: find an “optimal” mapping of this circuit:



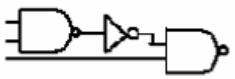
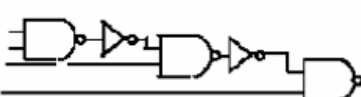
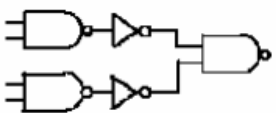
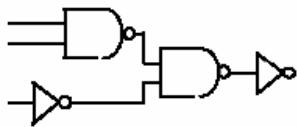
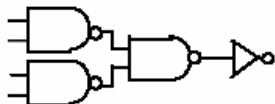
Into this library:



標準庫的元件

- 都會有對應的正規樹狀表達式

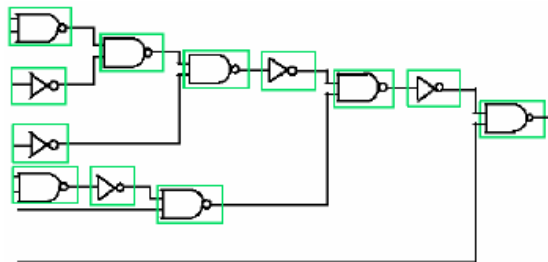
Primitive DAGs for Standard-Cell Library Gates

	Element/Area Cost		Tree Representation (normal form)
INVERTER	2		
NAND2	3		
NAND3	4		
NAND4	5		
			
AOI21	4		
AOI22	5		

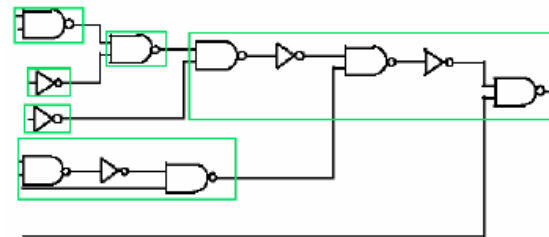
不同的映射方式

- 會有不同的面積

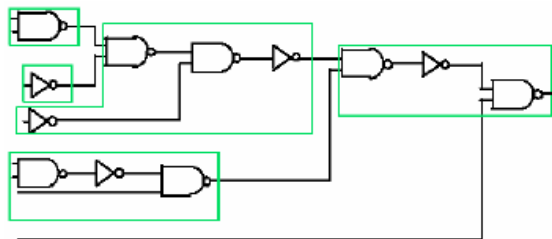
Possible Covers



7 NAND2 (3) = 21
5 INV (2) = 10
Area cost 31



2 INV = 4
2 NAND2 = 6
1 NAND3 = 4
1 NAND4 = 5
Area cost 19



1 INV = 2
1 NAND2 = 3
2 NAND3 = 8
1 AOI21 = 4
Area Cost 17

Hmmm. Seems promising but
is there a systematic and
efficient way to arrive at the
optimal answer?

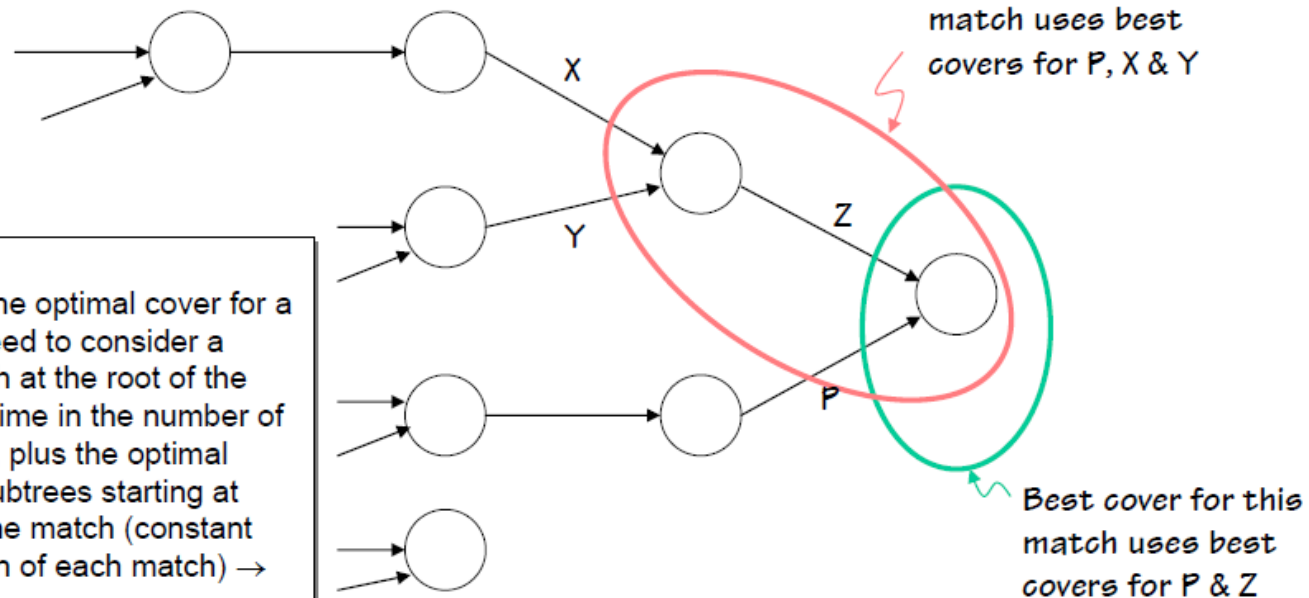
動態規劃法

- 是尋找優化映射的方法之一

Use Dynamic Programming!

Principle of optimality: Optimal cover for a tree consists of a best match at the root of the tree plus the optimal cover for the sub-trees starting at each input of the match.

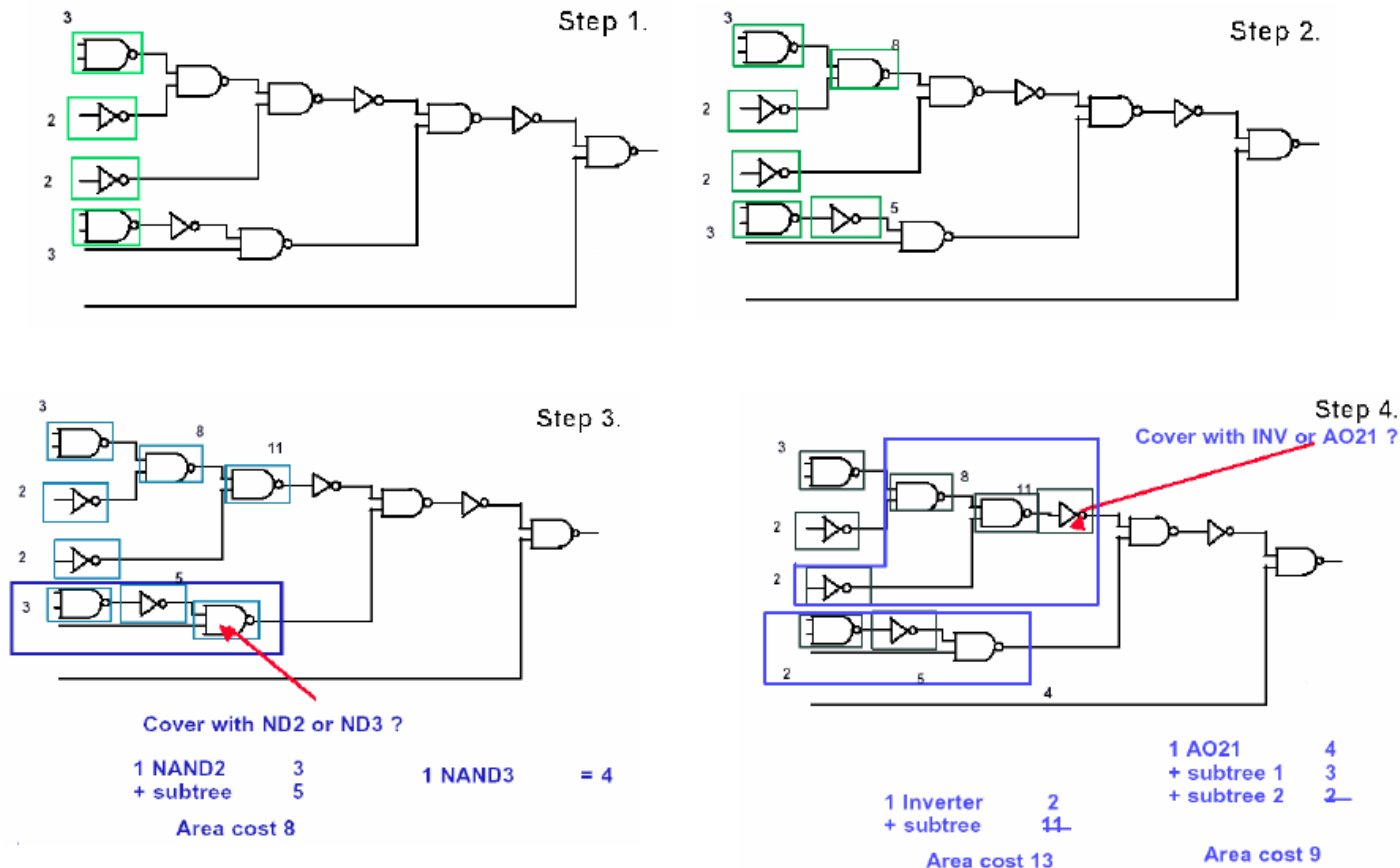
Complexity:
To determine the optimal cover for a tree we only need to consider a best-cost match at the root of the tree (constant time in the number of matched cells), plus the optimal cover for the subtrees starting at each input to the match (constant time in the fanin of each match) $\rightarrow O(N)$



透過動態規劃

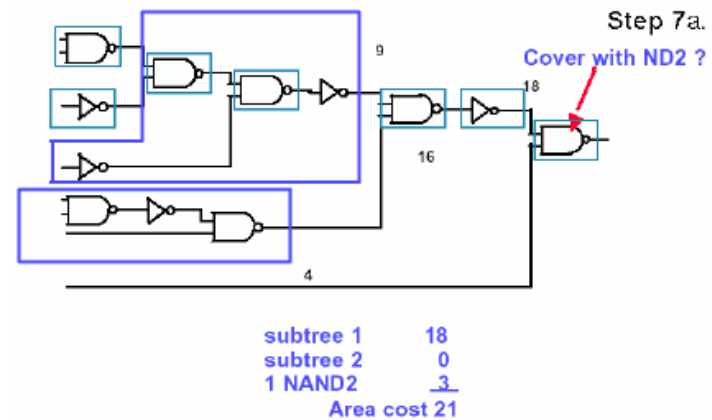
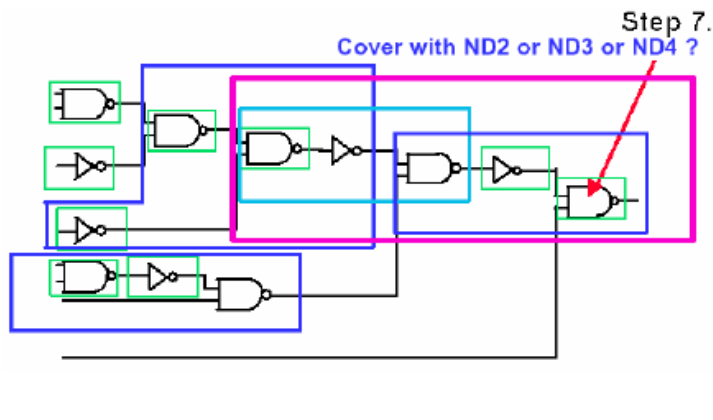
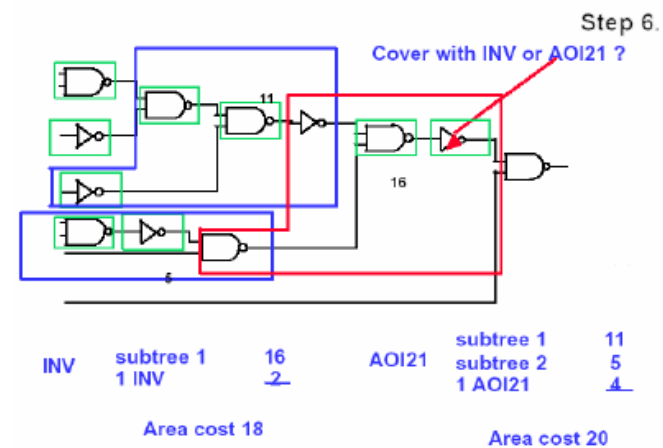
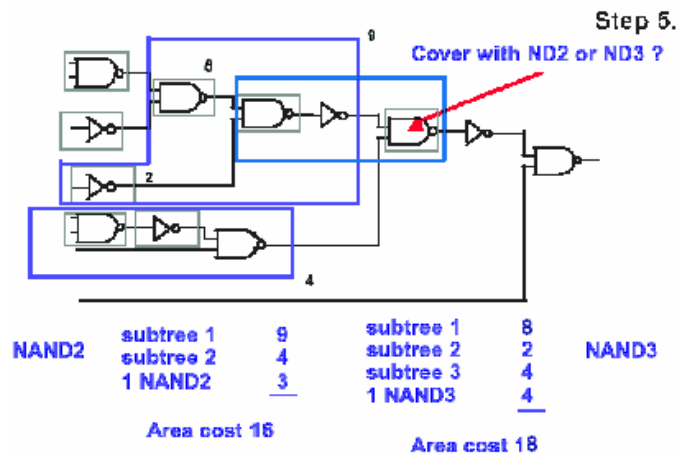
- 從第一層開始，逐步涵蓋所有電路

Optimal Tree Covering Example



繼續逐步成長

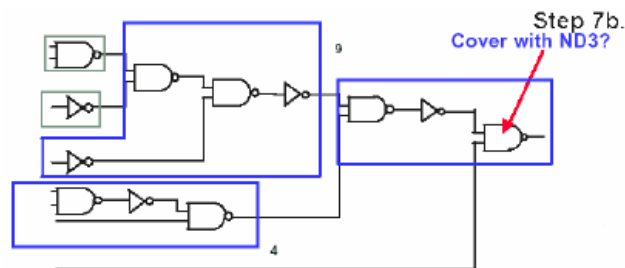
Optimal Tree Covering Example



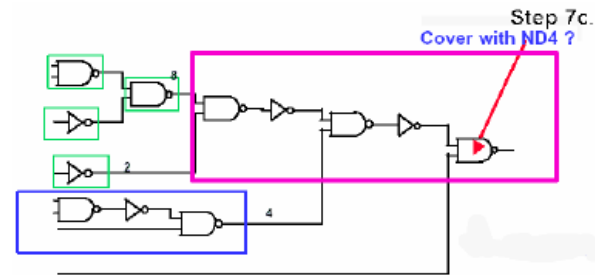
最後計算出

- 一個優化後的映射電路

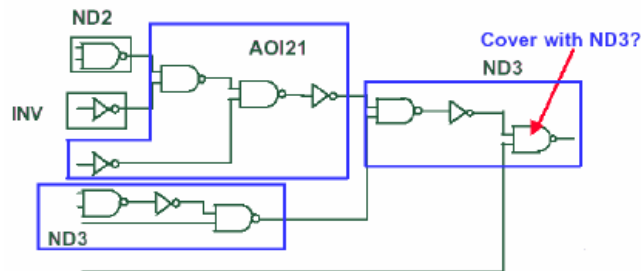
Optimal Tree Covering Example



subtree 1	9
subtree 2	4
subtree 3	0
1 NAND3	4
Area cost	17



subtree 1	8
subtree 2	2
subtree 3	4
subtree 4	0
1 NAND3	5
Area cost	19



INV	2
ND2	3
2 ND3	8
AOI21	4
Area cost	17

This matches our earlier intuitive cover, but accomplished systematically.

Refinements: timing optimization
incorporating load-dependent delays,
optimization for low power.

當然，要考慮的因素不只面積

- 還有功率，延遲等等因素

Refinements: timing optimization
incorporating load-dependent delays,
optimization for low power.

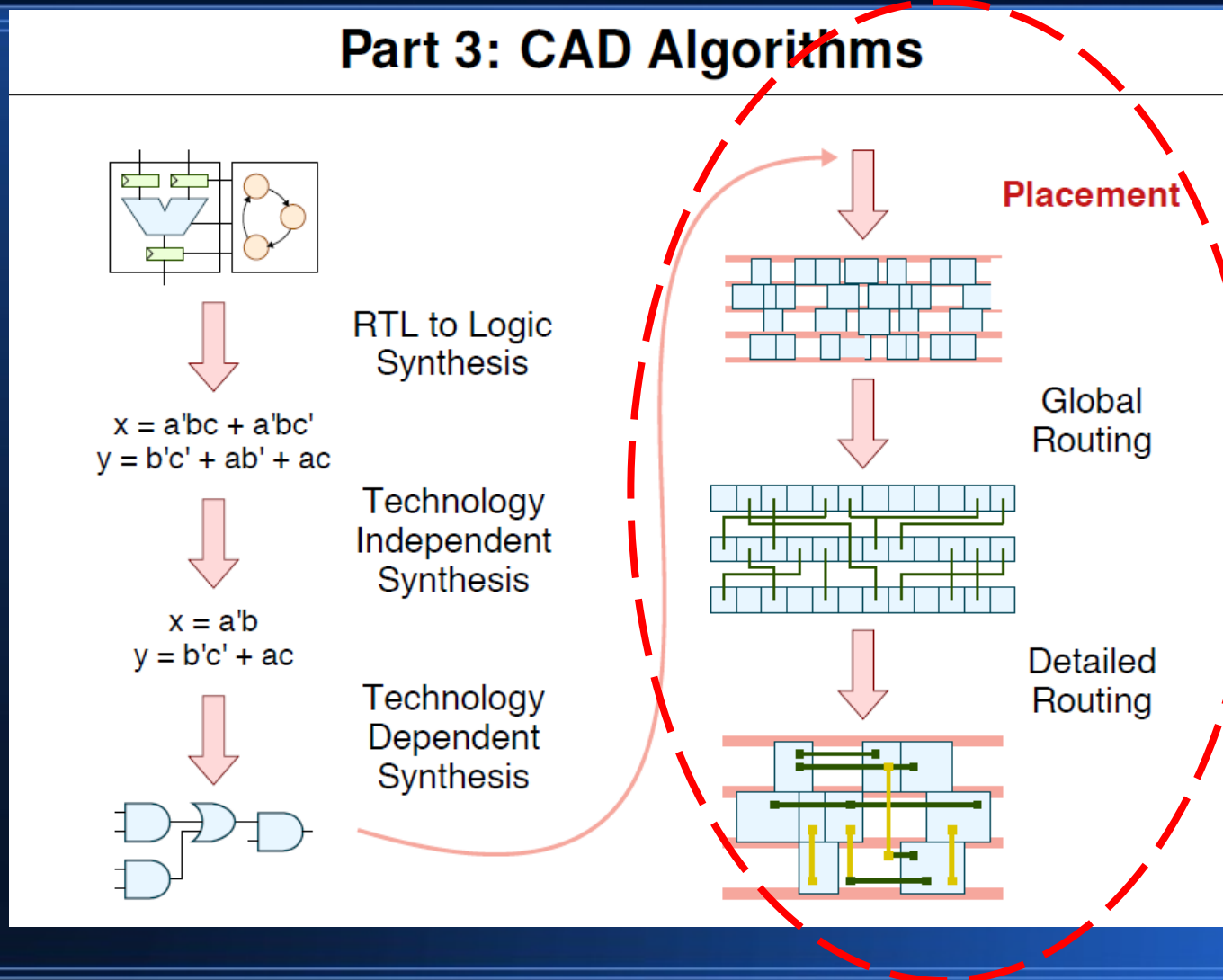
- 所以可以制定因素間的加權公式，作為分數

接著讓我們進入

- Physical Design Automation
- 實體設計自動化的主題

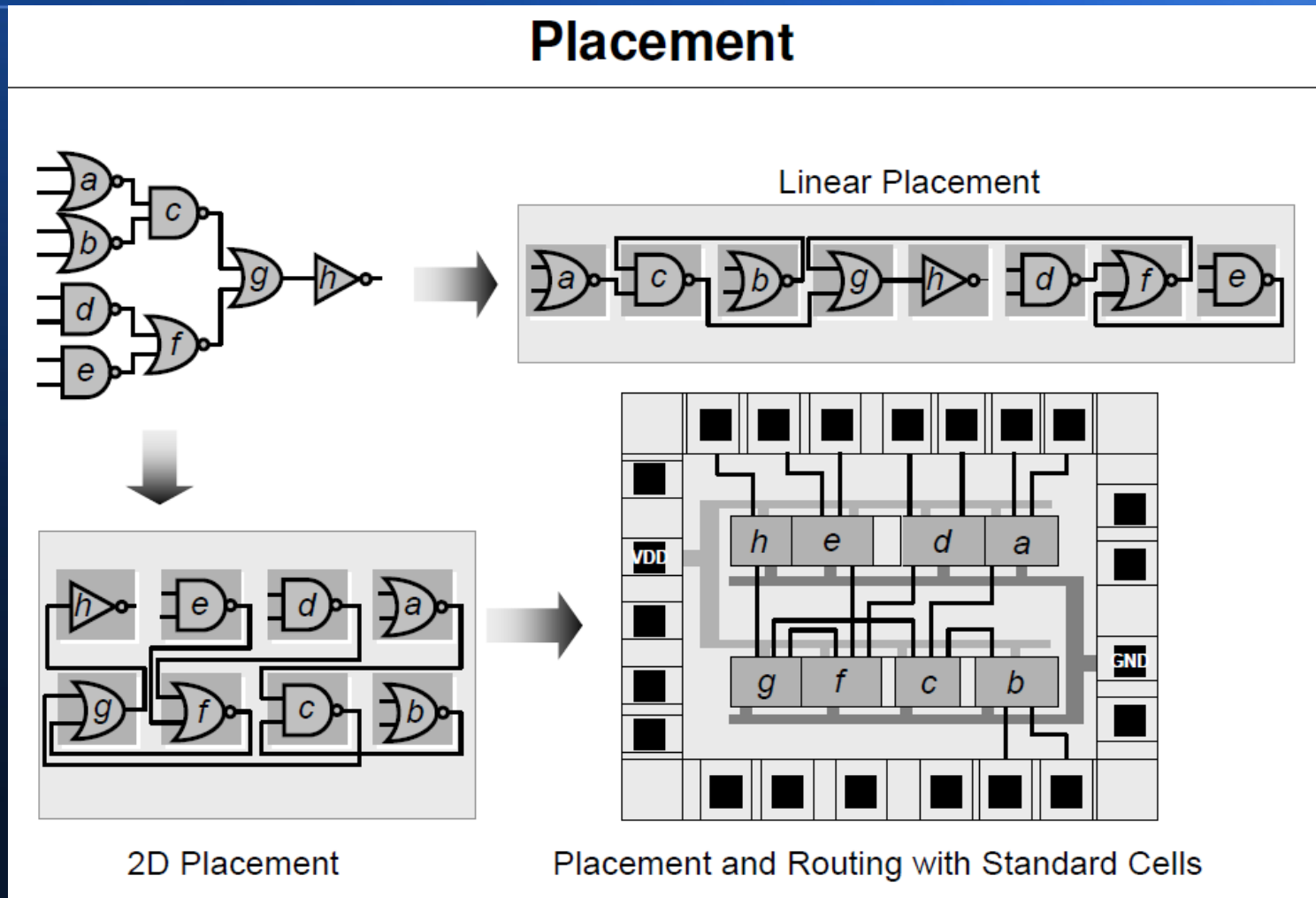
Physical Design Automation

- 主要包含 Placement 和 Routing



Placement 優化問題

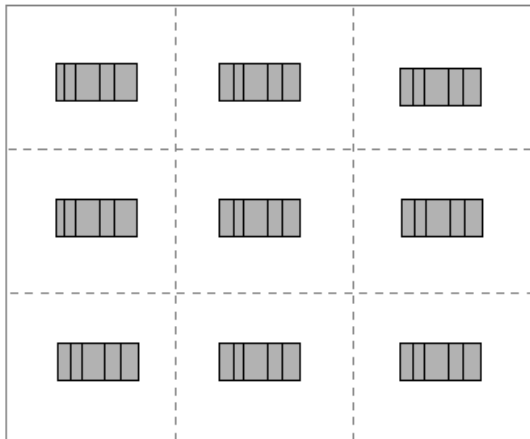
- 是你如何置放每個區塊



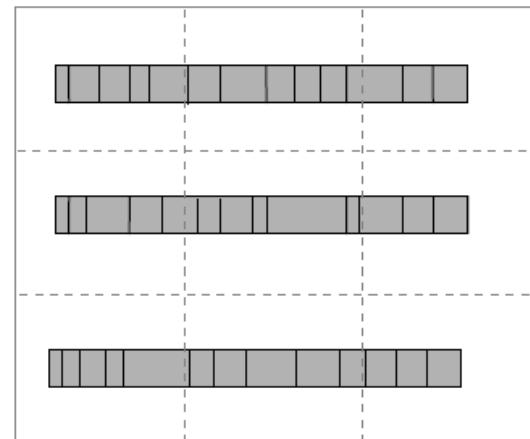
Placement 問題通常可以分為兩類

Global vs. Detailed Placement

Global
Placement



Detailed
Placement

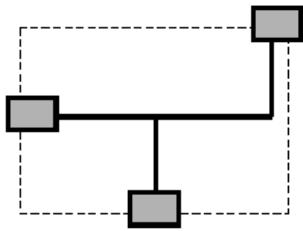


優化的目標除了線長

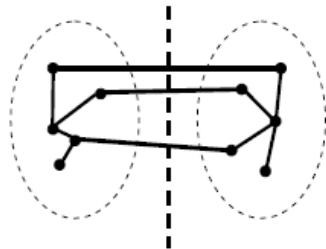
- 還包含《切割數 / 密集度 / 時間延遲》等考量

Placement Objective Functions

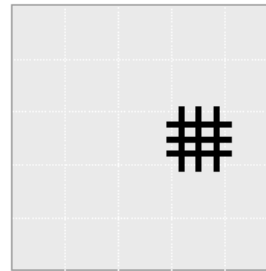
Total Wirelength



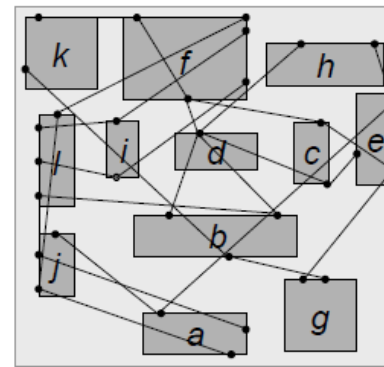
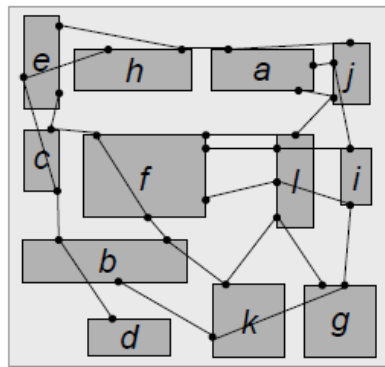
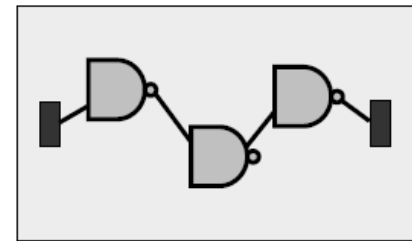
Number of Cut Nets



Wire Congestion



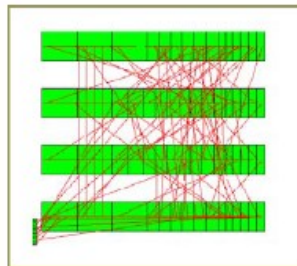
Signal Delay



好的 Placement

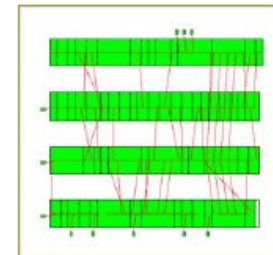
- 通常線路也較短

Good vs. Bad Placement



Bad placement causes routing congestion resulting in:

- Increases in circuit area (cost) and wiring
- Longer wires → more capacitance
 - Longer delay
 - Higher dynamic power dissipation

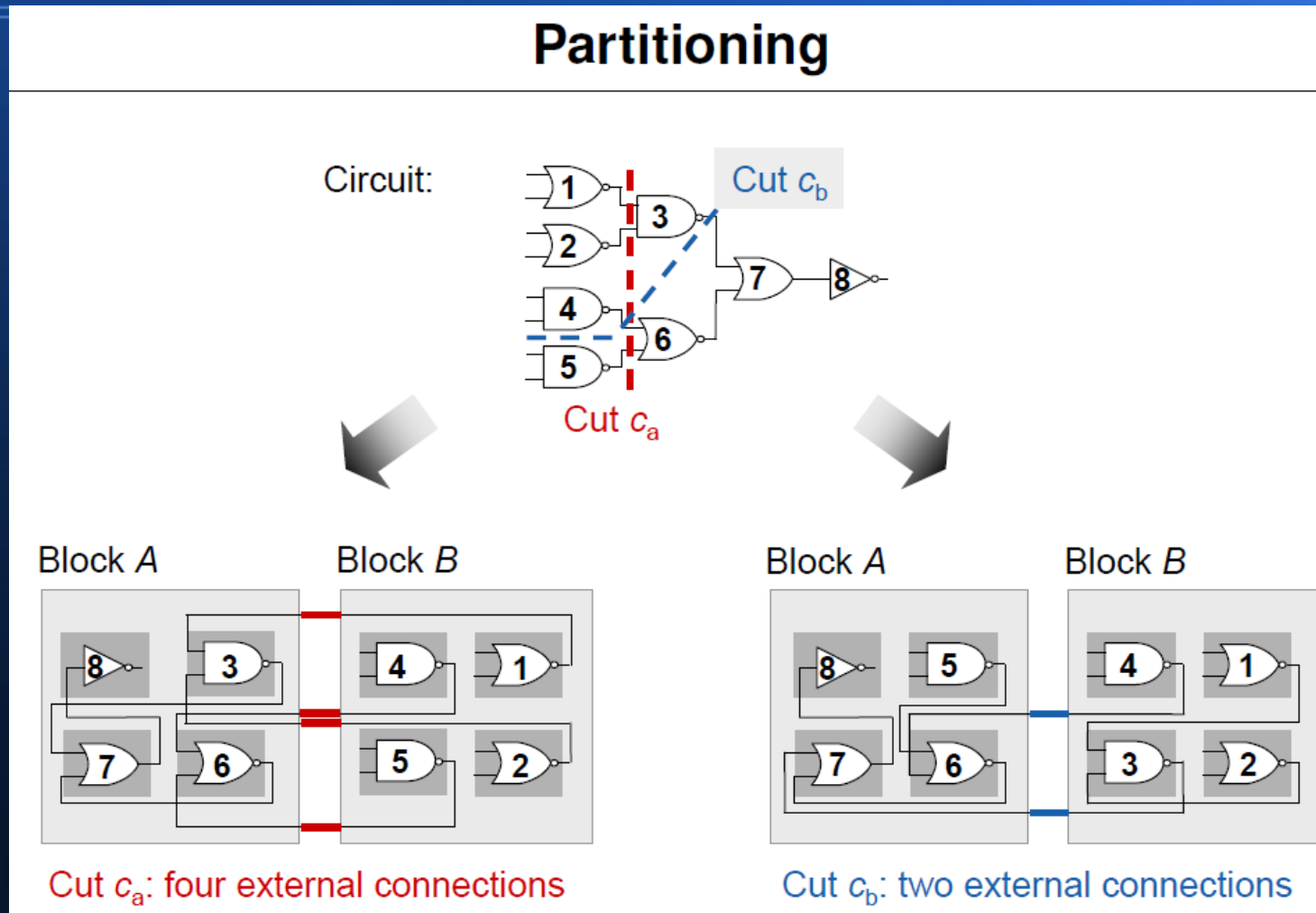


Good placement

- Circuit area (cost) and wiring decreases
- Shorter wires → less capacitance
 - Shorter delay
 - Less dynamic power dissipation

分割的好

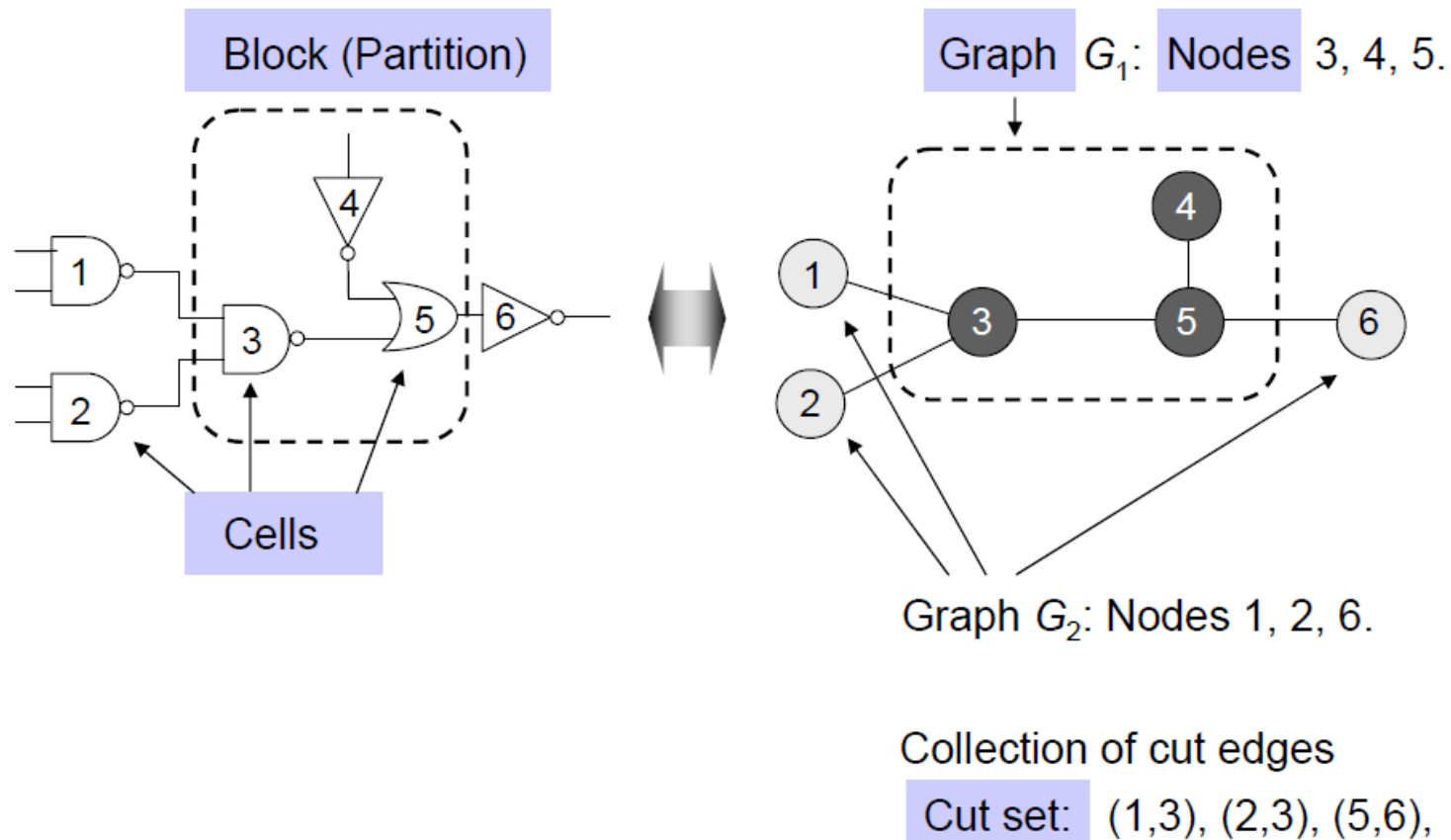
- 也可以降低區塊間的連線數量



分割時

- 要降低區塊的连接線數量

Partitioning Terminology



Kernighan-Lin(KL) 演算法

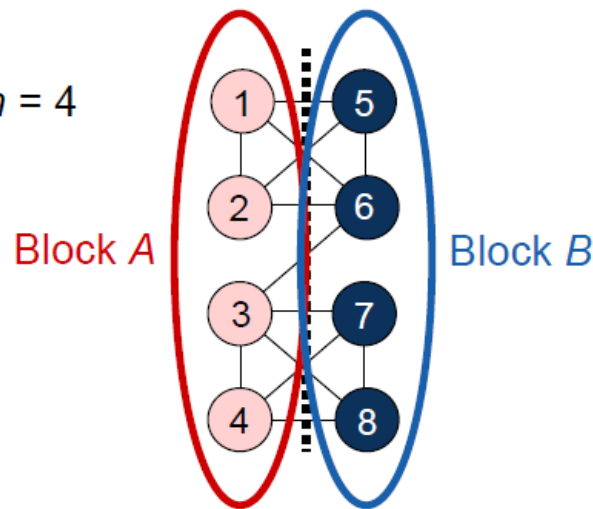
- 可以平均分割並降低區塊間的連接成本

Kernighan-Lin (KL) Algorithm

Given: A graph with $2n$ nodes where each node has the same weight.

Goal: A partition (division) of the graph into two disjoint subsets A and B with minimum cut cost and $|A| = |B| = n$.

Example: $n = 4$



於是可以透過 KL 算法

- 不斷分割成兩區，然後就 1, 2, 4, 8, 16...

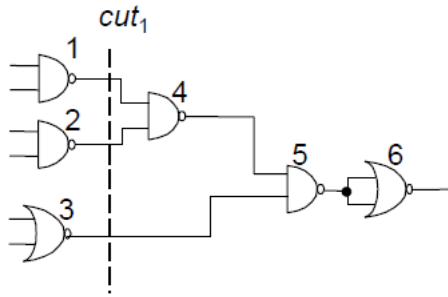
Min-Cut Placement

- ▶ Use partitioning algorithm (such as KL) to divide netlist into two regions
- ▶ Use partitioning algorithm to recursively divide the two partitions into two smaller regions (four regions total)
- ▶ Each cut heuristically minimizes the number of cut nets

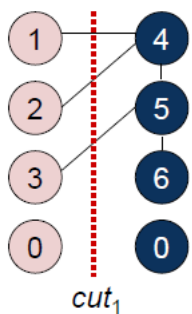
以下是 KL 算法進行不斷分割的範例

- 這樣就能將邏輯電路分進一格一個的區塊當中

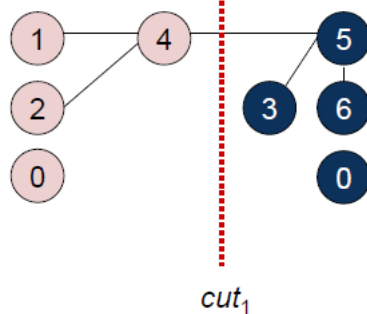
Min-Cut Example



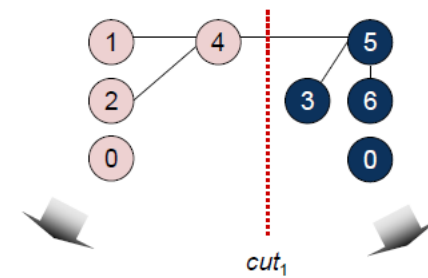
Vertical cut cut_1 : $L=\{1,2,3\}$, $R=\{4,5,6\}$



KL Algorithms

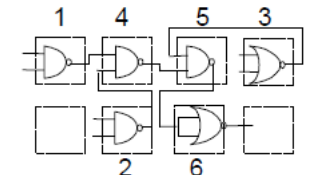
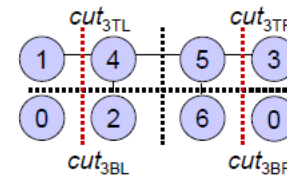
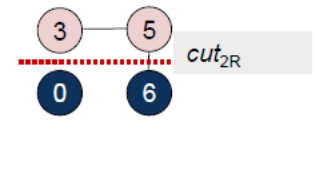
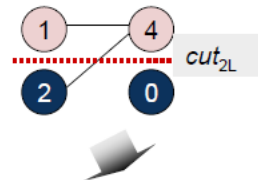


Min-Cut Example



Horizontal cut cut_{2L} : $T=\{1,4\}$, $B=\{2,0\}$

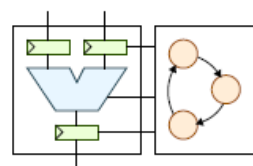
Horizontal cut cut_{2R} : $T=\{3,5\}$, $B=\{6,0\}$



接著看 Routing 的算法

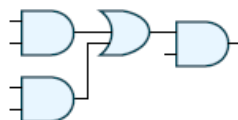
- 先從 Global Routing 開始

Part 3: CAD Algorithms



$$x = a'bc + a'bc'$$
$$y = b'c' + ab' + ac$$

$$x = a'b$$
$$y = b'c' + ac$$

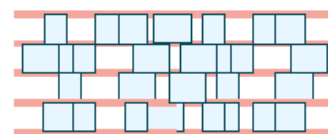


RTL to Logic
Synthesis

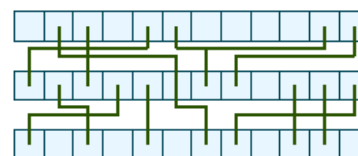
Technology
Independent
Synthesis

Technology
Dependent
Synthesis

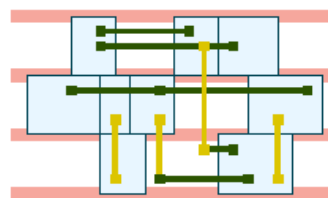
Placement



**Global
Routing**



Detailed
Routing



Routing 的範例

- 那些點之間要連接起來，用集合 N_i 表示

General Routing Problem

Netlist:

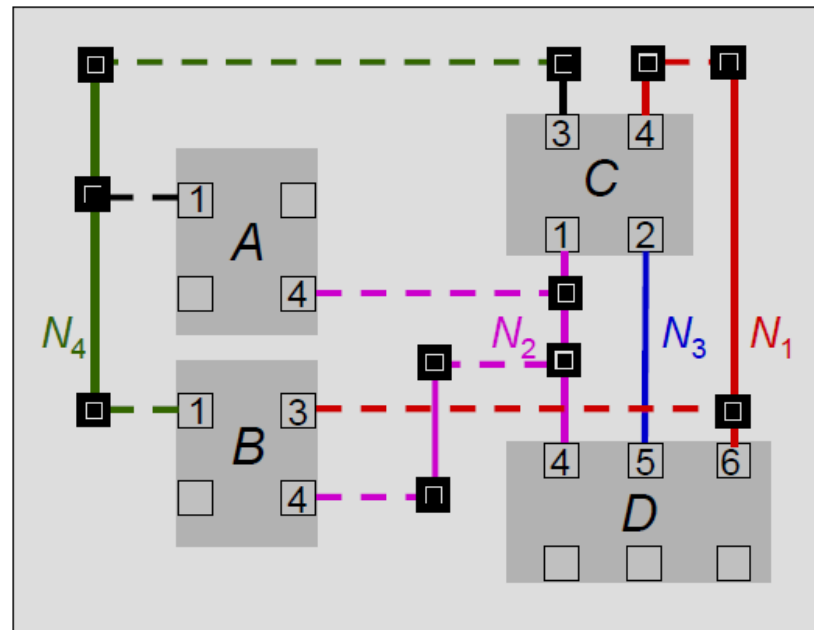
$N_1 = \{C_4, D_6, B_3\}$

$N_2 = \{D_4, B_4, C_1, A_4\}$

$N_3 = \{C_2, D_5\}$

$N_4 = \{B_1, A_1, C_3\}$

Technology Information
(Design Rules)



對於上層的 Global Routing

- 可用 Heuristic Sequential Steiner Tree Algorithm

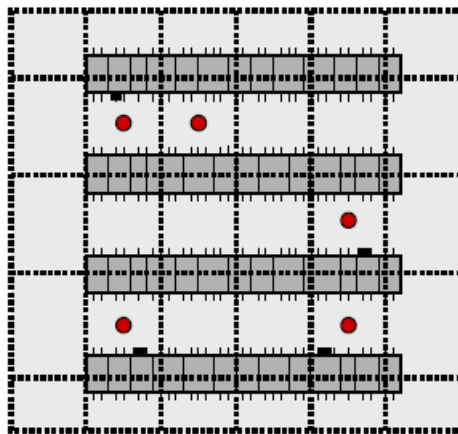
Heuristic Sequential Steiner Tree Algorithm

1. Find the closest (in terms of rectilinear distance) pin pair, construct their minimum bounding box (MBB)
2. Find the closest point pair (p_{MBB}, p_C) between any point p_{MBB} on the MBB and p_C from the set of pins to consider
3. Construct the MBB of p_{MBB} and p_C
4. Add the *L*-shape that p_{MBB} lies on to T (deleting the other *L*-shape). If p_{MBB} is a pin, then add any *L*-shape of the MBB to T .
5. Goto step 2 until the set of pins to consider is empty

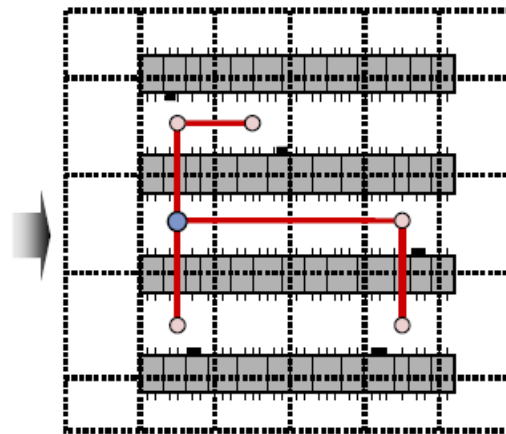


進行 Rectangular Routing

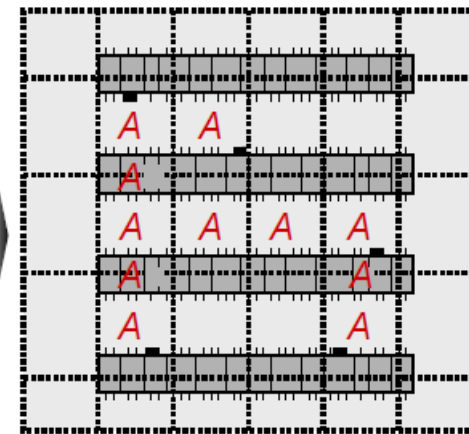
Rectilinear Routing



Pins assigned to grid cells



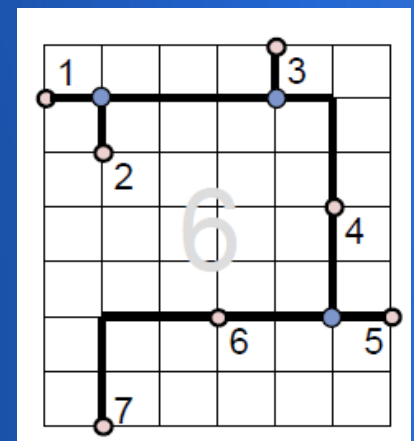
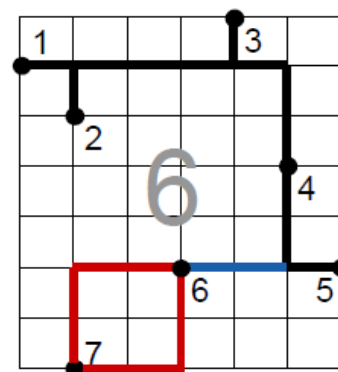
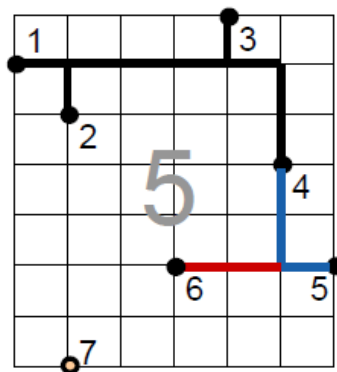
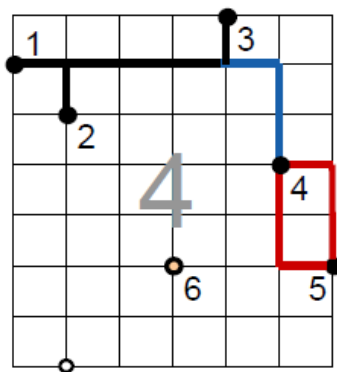
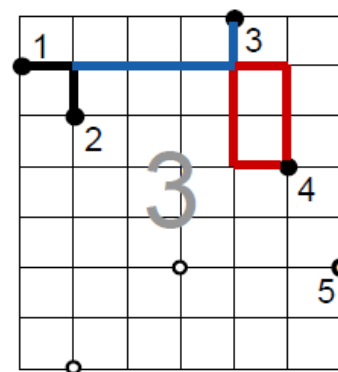
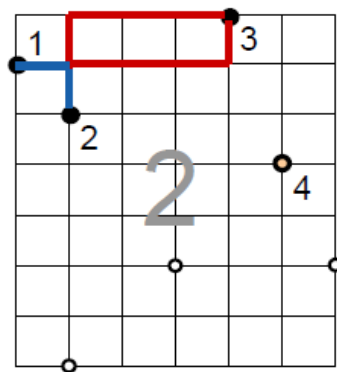
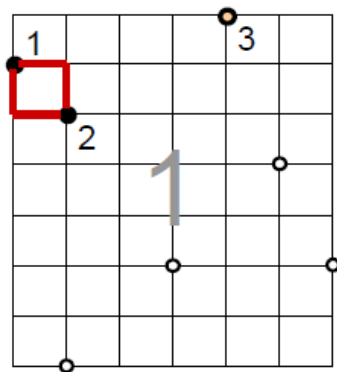
Rectilinear Steiner minimum tree (RSMT)



Assigned routing regions and feedthrough cells

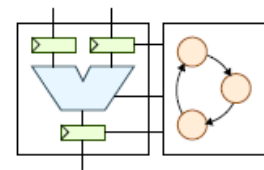
以下是該算法的一個範例

Heuristic Sequential Steiner Tree Example



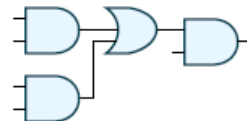
然後還要做 Detail Routing

Part 3: CAD Algorithms



$$x = a'bc + a'bc'$$
$$y = b'c' + ab' + ac$$

$$x = a'b$$
$$y = b'c' + ac$$

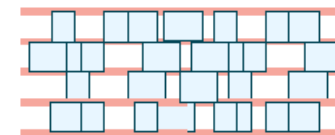


RTL to Logic
Synthesis

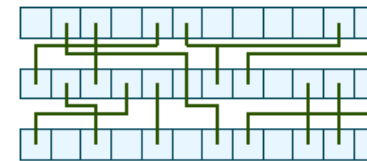
Technology
Independent
Synthesis

Technology
Dependent
Synthesis

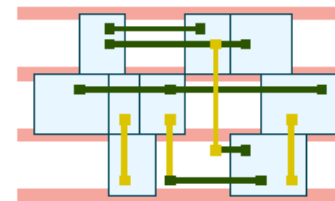
Placement



Global
Routing



**Detailed
Routing**

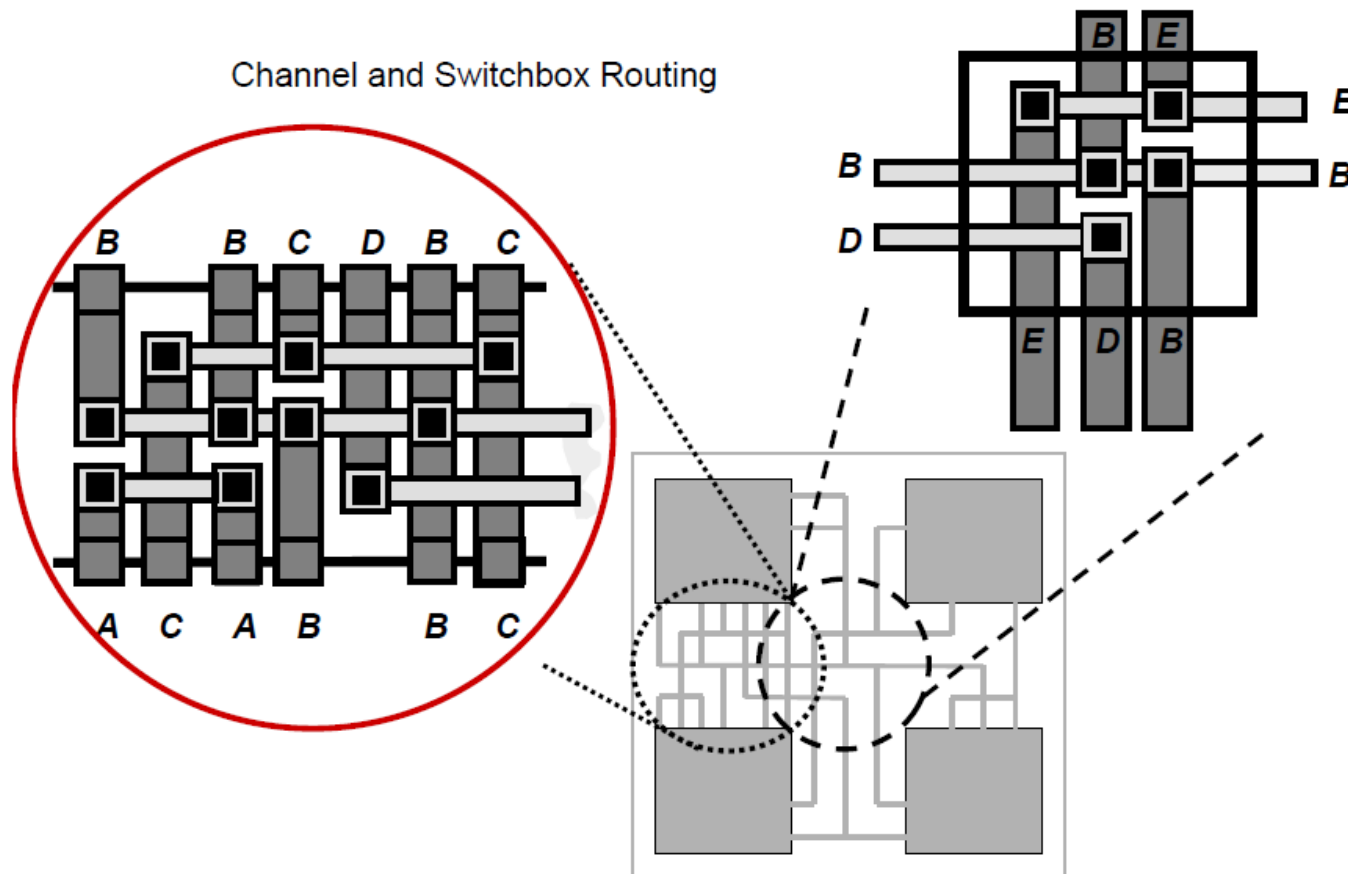


Detail Routing

- 問題是要做區塊間的轉接線路

Detailed Routing

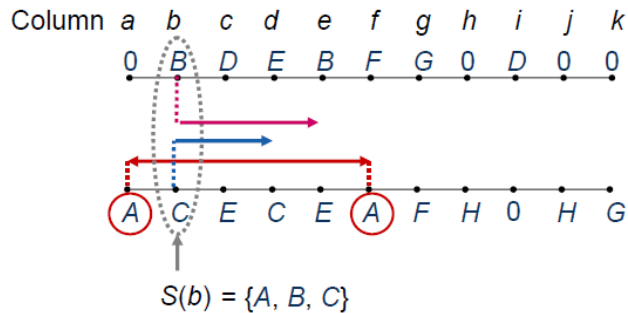
Channel and Switchbox Routing



但是這些轉接線路

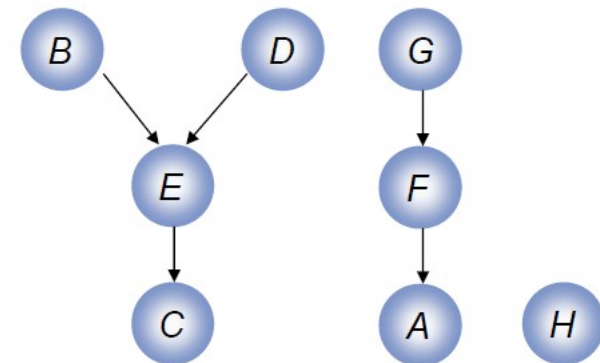
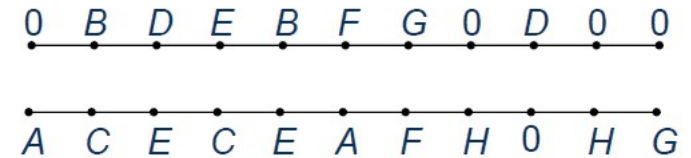
- 有水平與垂直兩種限制條件

Horizontal Constraint Graphs



- Let $S(col)$ denote the set of nets that pass through column col
- $S(col)$ contains all nets that either (1) are connected to a pin in column col or (2) have pin connections to both the left and right of col
- Since horizontal segments cannot overlap, each net in $S(col)$ must be assigned to a different track in column col
- $S(col)$ represents the lower bound on the number of tracks in column col ; lower bound of the channel height is given by maximum cardinality of any $S(col)$

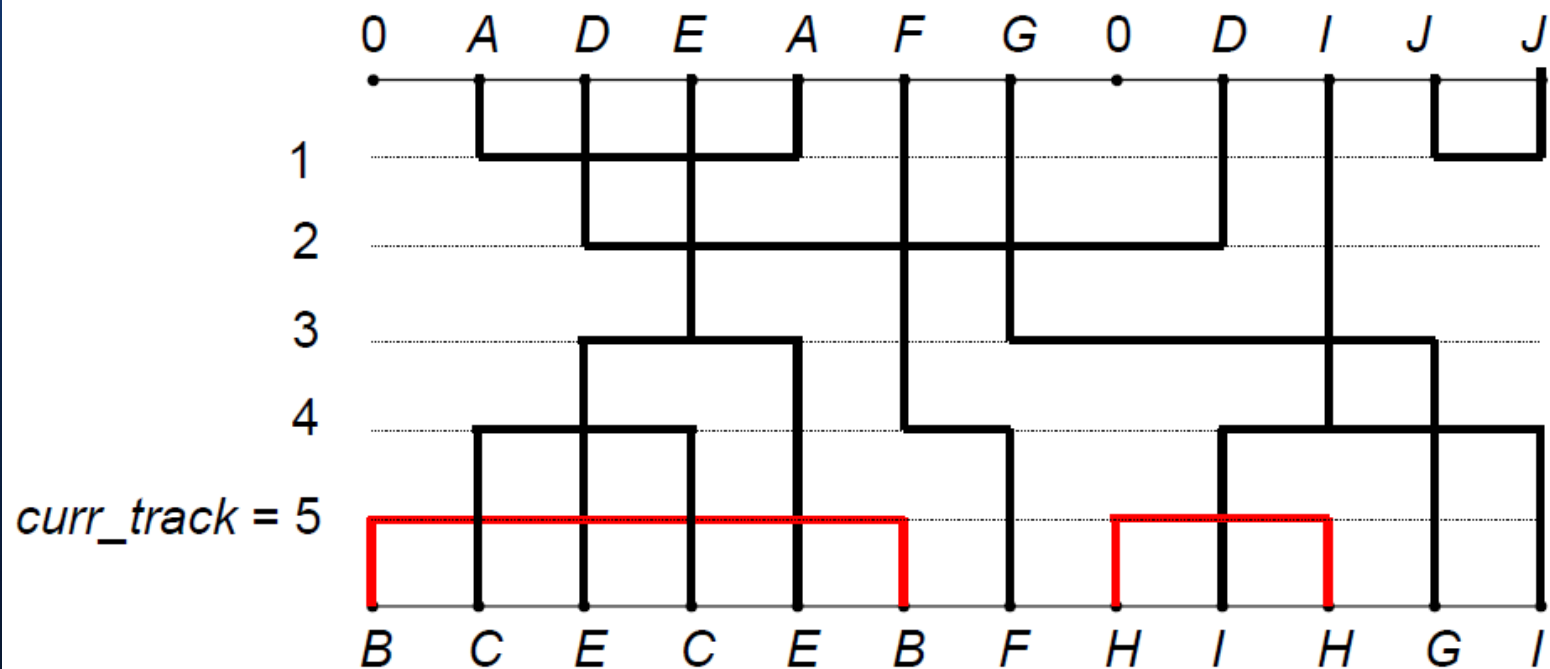
Vertical Constraint Graphs



我們可以用 Left-Edge Algorithm

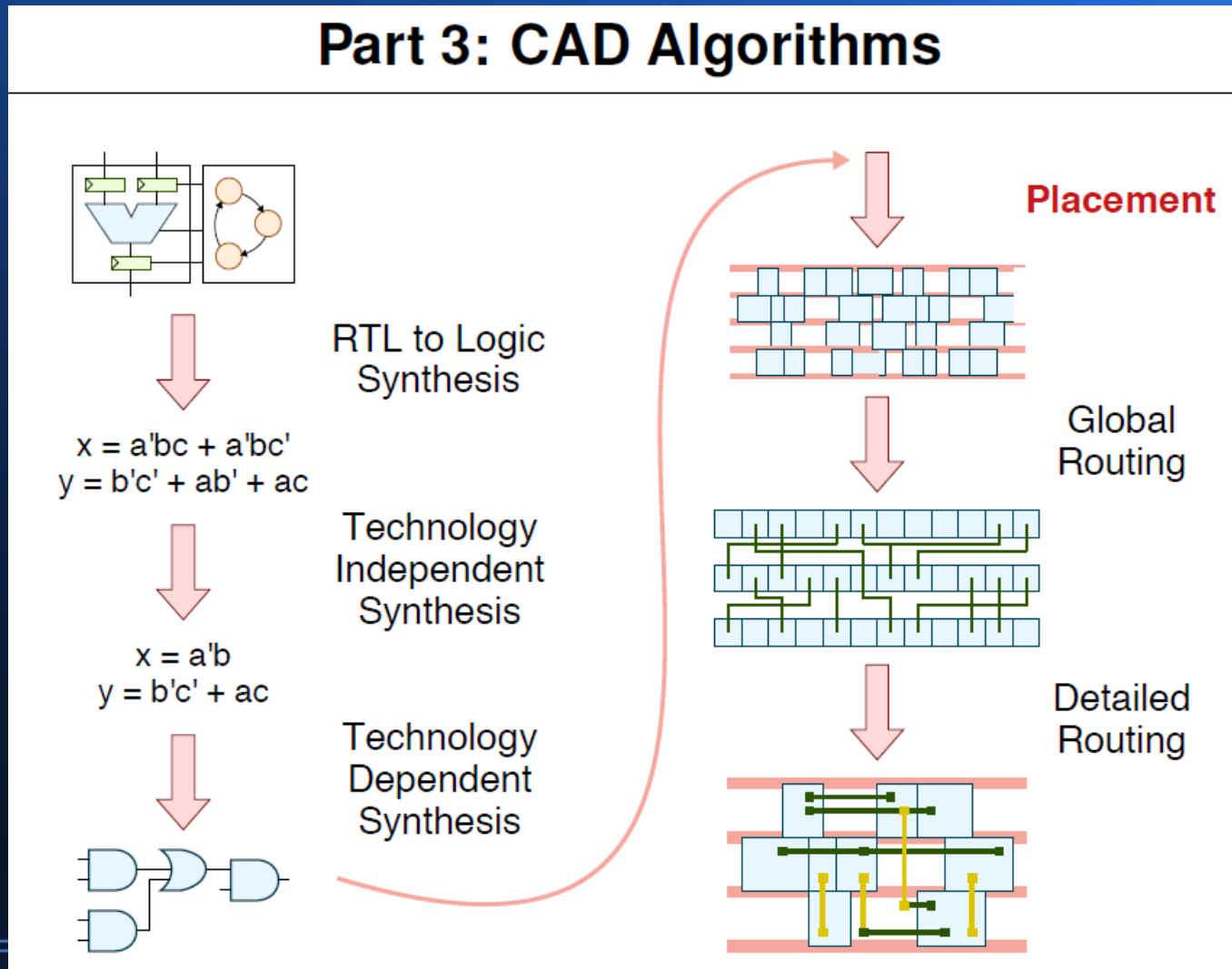
- 完成 Detail Routing 的優化工作

Left-Edge Algorithm Example

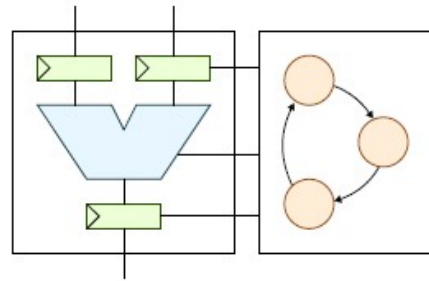


到目前為止

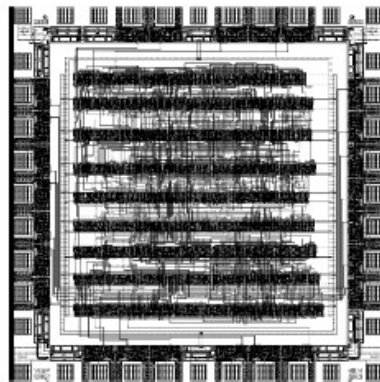
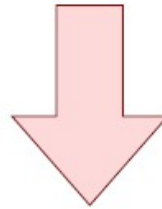
- 我們已經看完了優化算法在 IC 設計上的用途



整個 IC 技術的結構

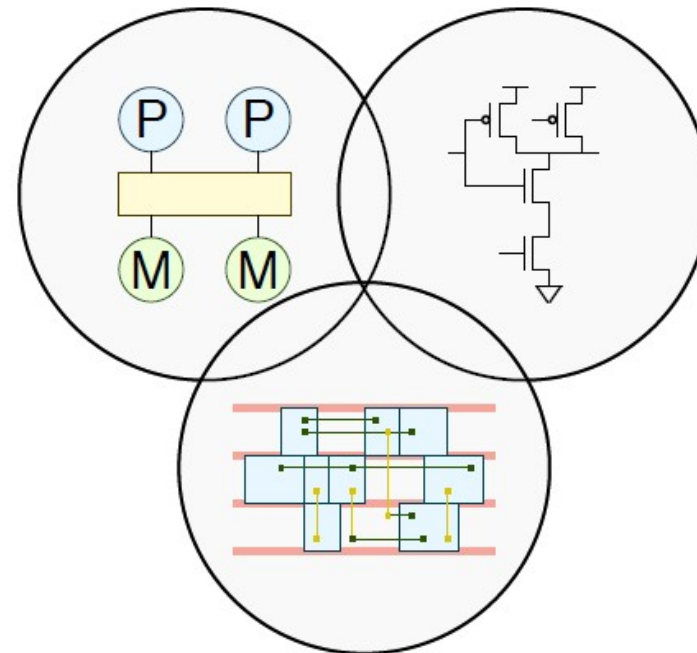


Part 1
ASIC Design
Overview



Prereq
Computer
Architecture

Part 2
Digital CMOS
Circuits

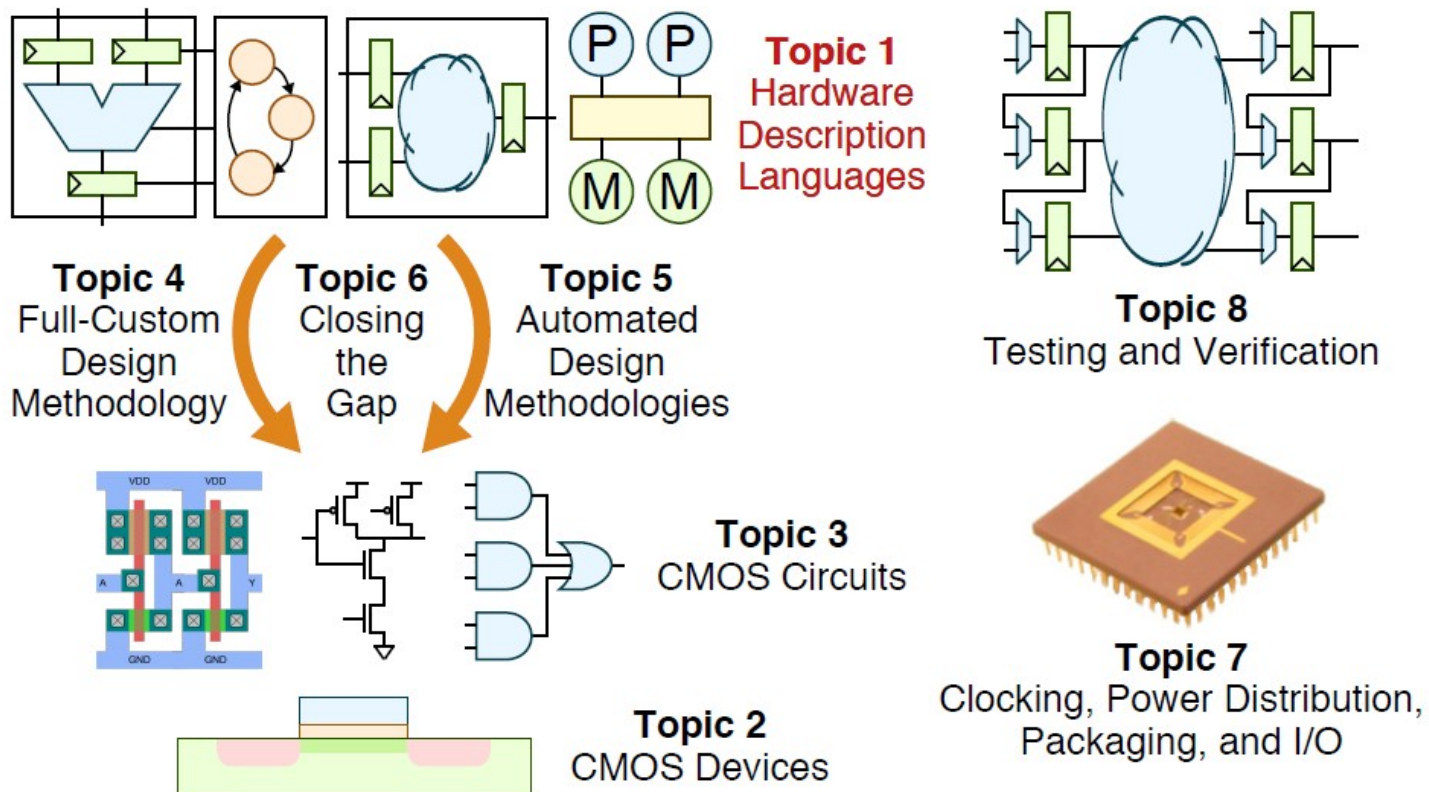


Part 3
CAD Algorithms

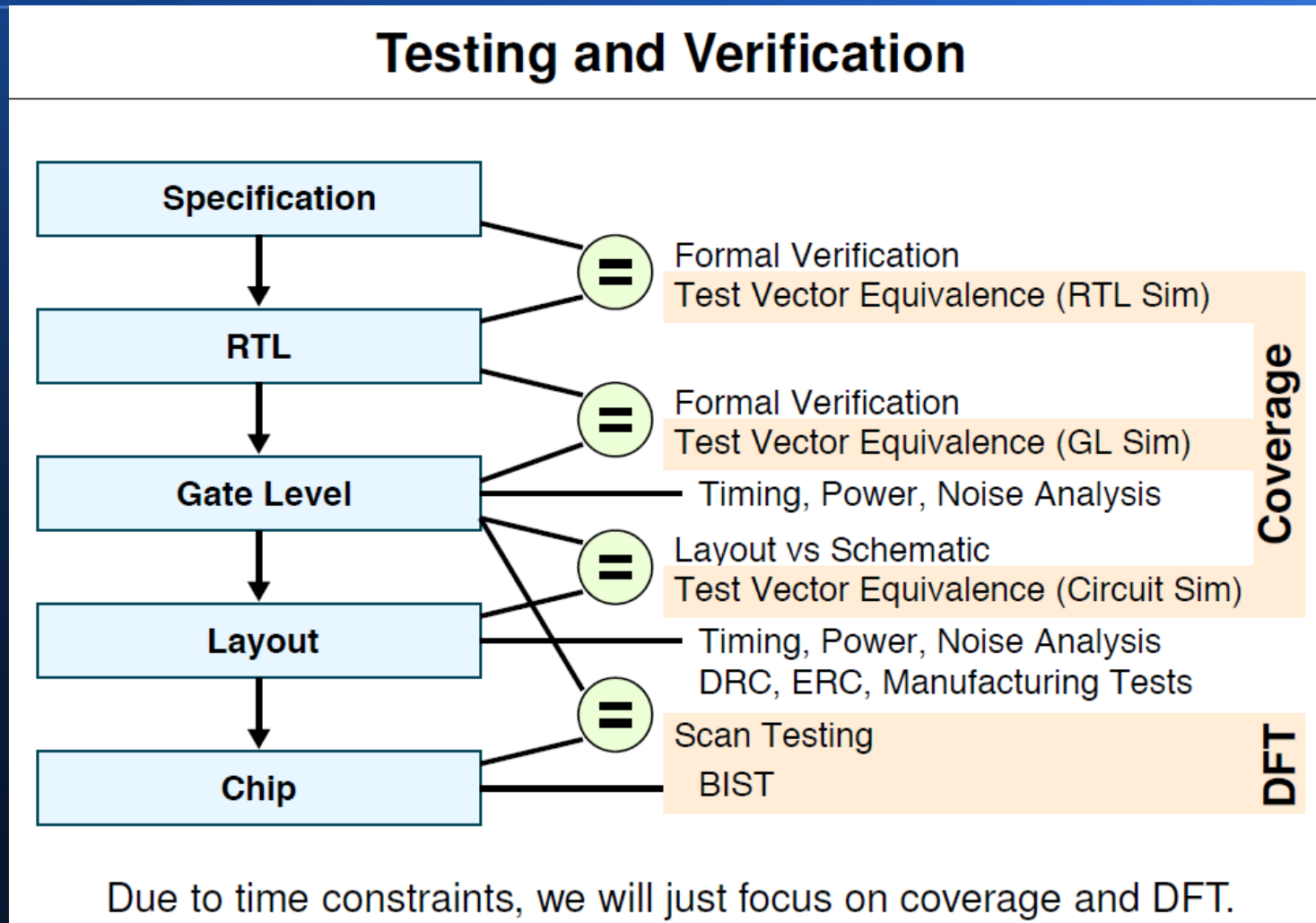
從硬體描述語言開始

- 經由《數位邏輯》到《CMOS 布局》

Part 1: ASIC Design Overview

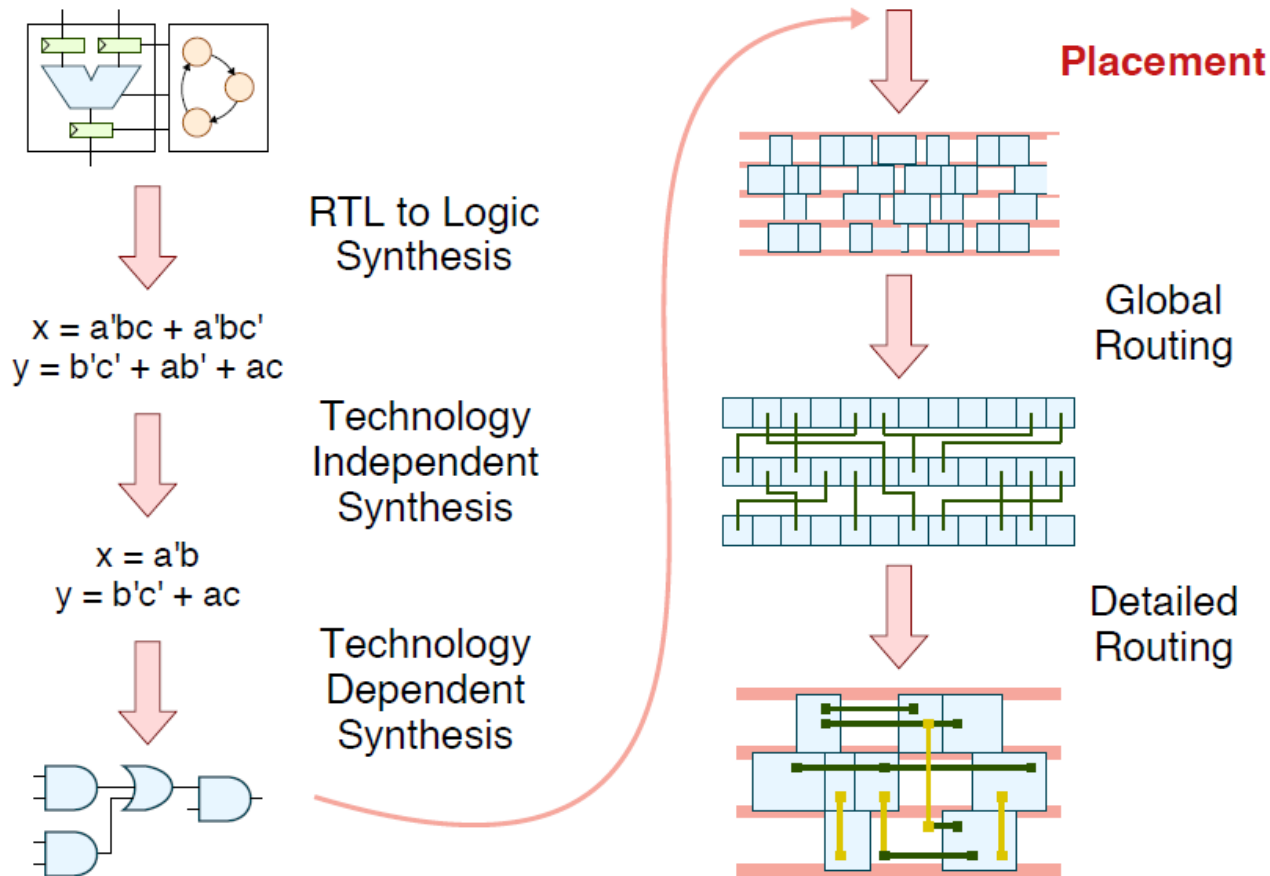


經過一關又一關的測試與驗證



然後用優化軟體進行布局

Part 3: CAD Algorithms



如果你對這些 EDA 軟體有興趣

- 想親自動手體驗這個過程，除了那些商用的合成軟體之外，開放原始碼也逐漸有了這類軟體實作（例如 Yosys / IceStorm / NextPnr），但是成熟度還比不上 A 那些公司的 FPGA Altera/Xilinx EDA 工具，當然更無法和那種 ASIC 的 EDA 工具，像是 Cadence/Synopsys 相比
- 不過如果只是要跑 Verilog 的程式模擬，Icarus Verilog 則比商用的工具更快更好用
- SystemVerilog 則可以用 Verilator 來跑模擬

Icarus Verilog 官網



Icarus Verilog

Search this site

Home

Quick Links

- Add-ons
- Notable Links

Support

- Bugs
- Support Providers

Donations

Media

Welcome to the home page for Icarus Verilog. This is the source for your favorite free implementation of Verilog!

What Is Icarus Verilog?

Icarus Verilog is a Verilog simulation and synthesis tool. It operates as a compiler, compiling source code written in Verilog (IEEE-1364) into some target format. For batch simulation, the compiler can generate an intermediate form called *vvp assembly*. This intermediate form is executed by the ```vvp` command. For synthesis, the compiler generates netlists in the desired format.

The compiler proper is intended to parse and elaborate design descriptions written to the IEEE standard *IEEE Std 1364-2005*. This is a fairly large and complex standard, so it will take some time to fill all the dark alleys of the standard, but that's the goal.

Icarus Verilog is a work in progress, and since the language standard is not standing still either, it probably always will be. That is as it should be. However, I will make stable releases from time to time, and will endeavor to not retract any features that appear in these stable releases. The quick links above will show the current stable release.

The main porting target is Linux, although it works well on many similar operating systems. Various people have contributed precompiled binaries of stable releases for a variety of targets. These releases are ported by volunteers, so what binaries are available depends on who takes the time to do the packaging. *Icarus Verilog* has been ported to That Other Operating System, as a command line tool, and there are installers for users without compilers. You can compile it entirely with free tools, too, although there are precompiled binaries of stable releases.

Icarus 的用法

不安全 | ccckmit.wikidot.com/ve:icarushello

site-name .wikidot.com Share on      

Icarus Verilog 編譯器

文章 留言 授權

下載

1. Icarus Verilog for Windows — <http://bleyer.org/icarus/>
 - iverilog : 編譯
 - vvp : 執行
 - iverilog-vpi : Verilog 與 C 的連結方法
 - 參考 : http://iverilog.wikia.com/wiki/Using_VPI
2. Icarus Verilog — <http://iverilog.icarus.com/>
 - 相關套件 : <http://ivi.sourceforge.net/screenshots.html> (波形模擬繪圖)

Icarus 的執行範例

不安全 | ccckmit.wikidot.com/ve:xor

site-name .wikidot.com

Share on      

Icarus Verilog 執行結果

```
D:\ccc101\icarus\ccc>iverilog -o xorTest xorTest.v
```

```
D:\ccc101\icarus\ccc>vvp xorTest
```

```
a=0 b=0 c=0
```

```
a=1 b=0 c=1
```

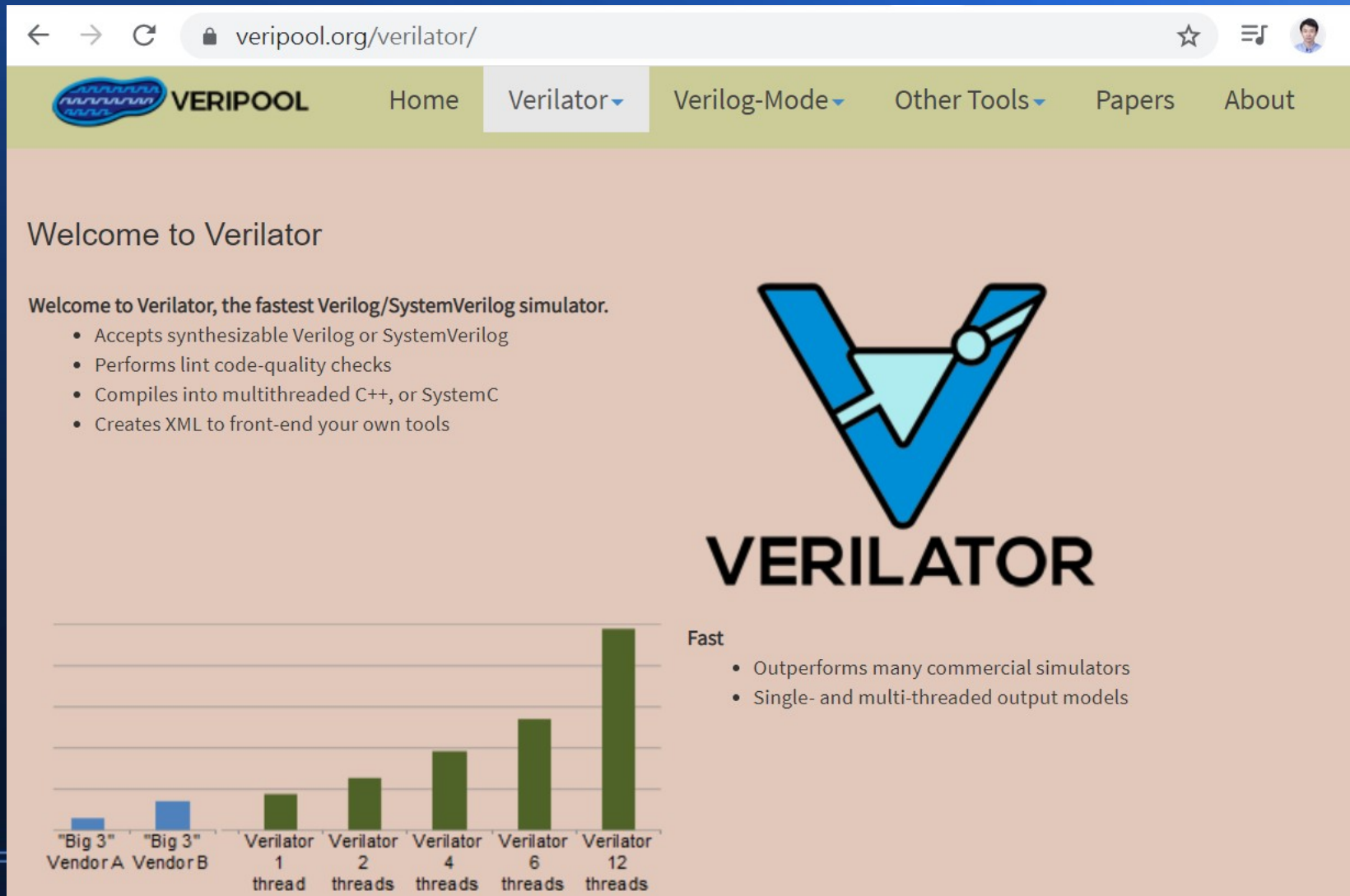
```
a=0 b=1 c=1
```

```
a=1 b=1 c=0
```

```
a=0 b=0 c=0
```

```
a=1 b=0 c=1
```


Veriator 官網



The screenshot shows the Veriator website homepage. The browser address bar displays `veripool.org/veriator/`. The navigation bar includes the VeriPool logo, a 'Home' link, and a 'Veriator' dropdown menu. Other links in the bar are 'Verilog-Mode', 'Other Tools', 'Papers', and 'About'. The main content area features a 'Welcome to Veriator' heading, a description of Veriator as the fastest Verilog/SystemVerilog simulator, a list of features, the Veriator logo, a performance bar chart, and a 'Fast' section with additional features.

← → ↻ `veripool.org/veriator/` ☆ 🎵 👤

VERIPOOL Home Veriator Verilog-Mode Other Tools Papers About

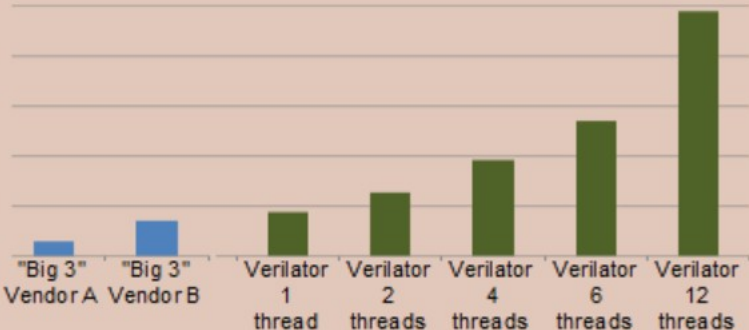
Welcome to Veriator

Welcome to Veriator, the fastest Verilog/SystemVerilog simulator.

- Accepts synthesizable Verilog or SystemVerilog
- Performs lint code-quality checks
- Compiles into multithreaded C++, or SystemC
- Creates XML to front-end your own tools



VERILATOR

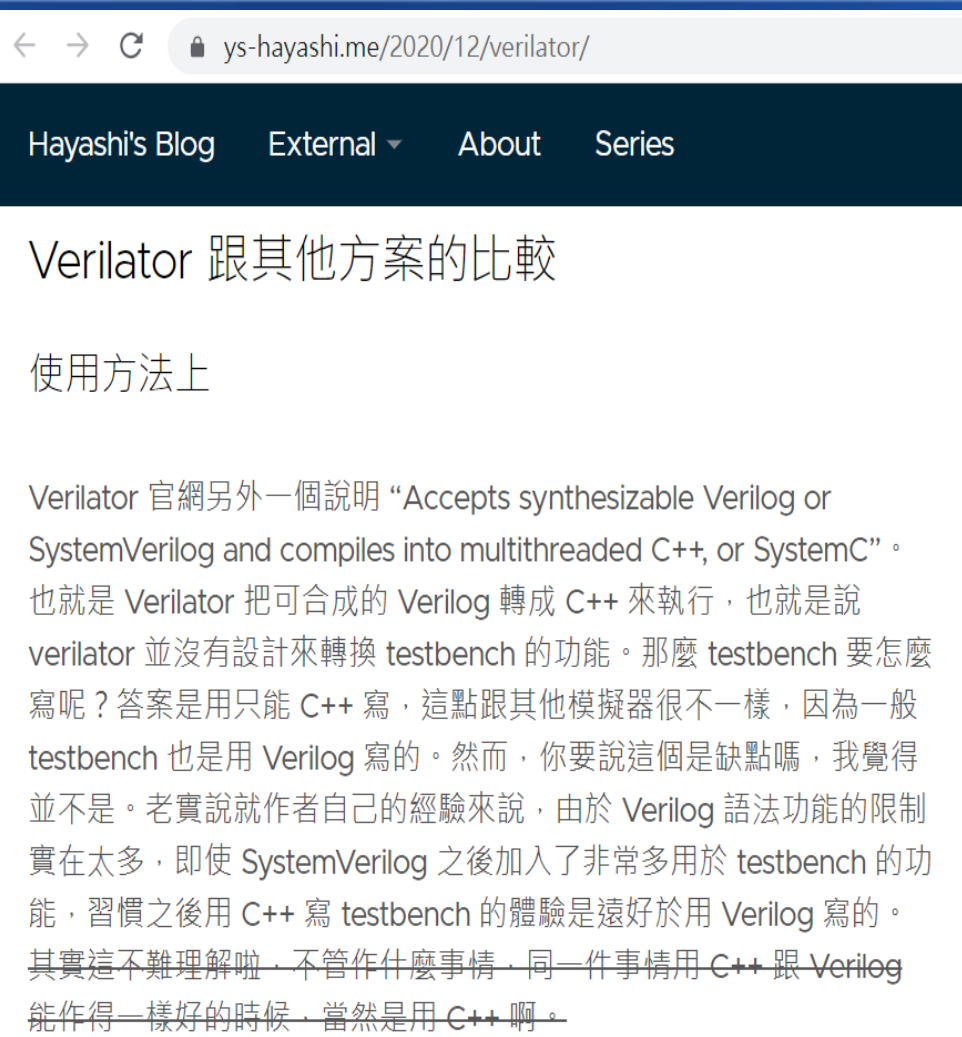


Configuration	Performance (Relative)
"Big 3" Vendor A	Low
"Big 3" Vendor B	Low
Veriator 1 thread	Low
Veriator 2 threads	Medium-Low
Veriator 4 threads	Medium
Veriator 6 threads	High
Veriator 12 threads	Very High

Fast

- Outperforms many commercial simulators
- Single- and multi-threaded output models

Verilator 的執行範例



← → ↻ ys-hayashi.me/2020/12/verilator/

Hayashi's Blog External ▾ About Series

Verilator 跟其他方案的比較

使用方法上

Verilator 官網另外一個說明 “Accepts synthesizable Verilog or SystemVerilog and compiles into multithreaded C++, or SystemC”。

也就是 Verilator 把可合成的 Verilog 轉成 C++ 來執行，也就是說 verilator 並沒有設計來轉換 testbench 的功能。那麼 testbench 要怎麼寫呢？答案是用只能 C++ 寫，這點跟其他模擬器很不一樣，因為一般 testbench 也是用 Verilog 寫的。然而，你要說這個是缺點嗎，我覺得並不是。老實說就作者自己的經驗來說，由於 Verilog 語法功能的限制實在太多，即使 SystemVerilog 之後加入了非常多用於 testbench 的功能，習慣之後用 C++ 寫 testbench 的體驗是遠好於用 Verilog 寫的。其實這不難理解啦，不管作什麼事情，同一件事情用 C++ 跟 Verilog 能作得一樣好的時候，當然是用 C++ 啊。

使用 verilator 模擬：

1. 撰寫 testbench `testbench.cpp`。
2. 準備可合成之待測模組 `design_under_test.v`。
3. 把待測試模組轉換成 C++ (或是 SystemC)，並指名要使用的 testbench `verilator design_under_test.v --exe testbench.cpp -cc`。
4. Verilator 產生的檔案都在 `obj_dir` 資料夾 (這個是預設名稱)。
5. 在該資料夾下面編譯產生 binary `make -C obj_dir -f Vdesign_under_test.mk`。
6. 執行模擬 `./obj_dir/Vdesign_under_test`。

上面的步驟跟 vcs 的手動模式的複雜程度差不多，但是 verilator 沒有一鍵完成步驟 3 之後的自動化指令，所以用起來相對麻煩。

Yosys 官網

不安全 | clifford.at/yosys/

Yosys Open SYnthesis Suite

About

Documentation

F.A.Q.

Screenshots

Download

Links

Donate

Reddit

IRC (Freenode)

Stack Overflow

About

Yosys is a framework for Verilog RTL synthesis. It currently has extensive Verilog-2005 support and provides a basic set of synthesis algorithms for various application domains. Selected features and typical applications:

- Process almost any synthesizable Verilog-2005 design
- Converting Verilog to BLIF / EDIF/ BTOR / SMT-LIB / simple RTL Verilog / etc.
- Built-in formal methods for checking properties and equivalence
- Mapping to ASIC standard cell libraries (in Liberty File Format)
- Mapping to Xilinx 7-Series and Lattice iCE40 FPGAs
- Foundation and/or front-end for custom flows

Yosys can be adapted to perform any synthesis job by combining the existing passes (algorithms) using synthesis scripts and adding additional passes as needed by extending the Yosys C++ code base.

Yosys is free software licensed under the [ISC license](#) (a GPL compatible license that is similar in terms to the MIT license or the 2-clause BSD license).

Yosys 是甚麼？

eet-china.com/mp/a59391.html

Yosys是verilogrtl综合的一个框架。它目前拥有广泛的Verilog-2005支持，并为各种应用领域提供了一套基本的综合算法。其特点如下：

- 1) 能处理所有可合成的Verilog-2005设计
- 2) 将Verilog转换为BLIF/EDIF/BTOR/SMT-LIB/simple RTL Verilog等格式
- 3) 可以做属性和等价性的检查，做逻辑化简、优化
- 4) 映射到ASIC标准单元库（Liberty文件格式）
- 5) 映射到Xilinx 7系列和Lattice iCE40、ECP5 FPGA, GOWin fpga上等
- 6) 自定义综合流程、扩展自定义的综合算法、扩展针对其他FPGA的综合等

Yosys可以通过使用综合脚本组合现有的算法，并通过扩展Yosys C++代码来实现新的pass,通过在综合流程中添加新的pass来执行自定义的任何综合工作。

Yosys 是2012年开始的项目，至今还在更新（cliffordwolf/yosys · GitHub），其自带一个cmos的标准库，类似于Synopsys GTECH，它的内部库映射算法使用的是 Berkeley ABC （A Simple System for Sequential Synthesis and Verification）。

透過 Yosys 和 Icarus

- 我們可以用 Verilog 來開發電路並測試
- 也可以研究上述 EDA 那些算法，究竟是如何實作的。
- 還能將 Verilog 編譯合成後，放到 Xilinx7 系列的 FPGA 開發板上形成系統

在此，我們不得不介紹一位奇人 Clifford Wolf

- 不過現在他 / 她改名叫 Claire Xen

2019年末，他发了一条Twitter: I've made some changes to my twitter profile and my twitter avatar. You are smart, you'll figure it out. ;-). 决定从他变成“她”，名字也从Clifford Wolf 改为Claire Xen，个人主页 (clairexen.net/home)，github ([clairexen - Overview](https://github.com/clairexen)) 等，全部更改。图3左右是他和她。



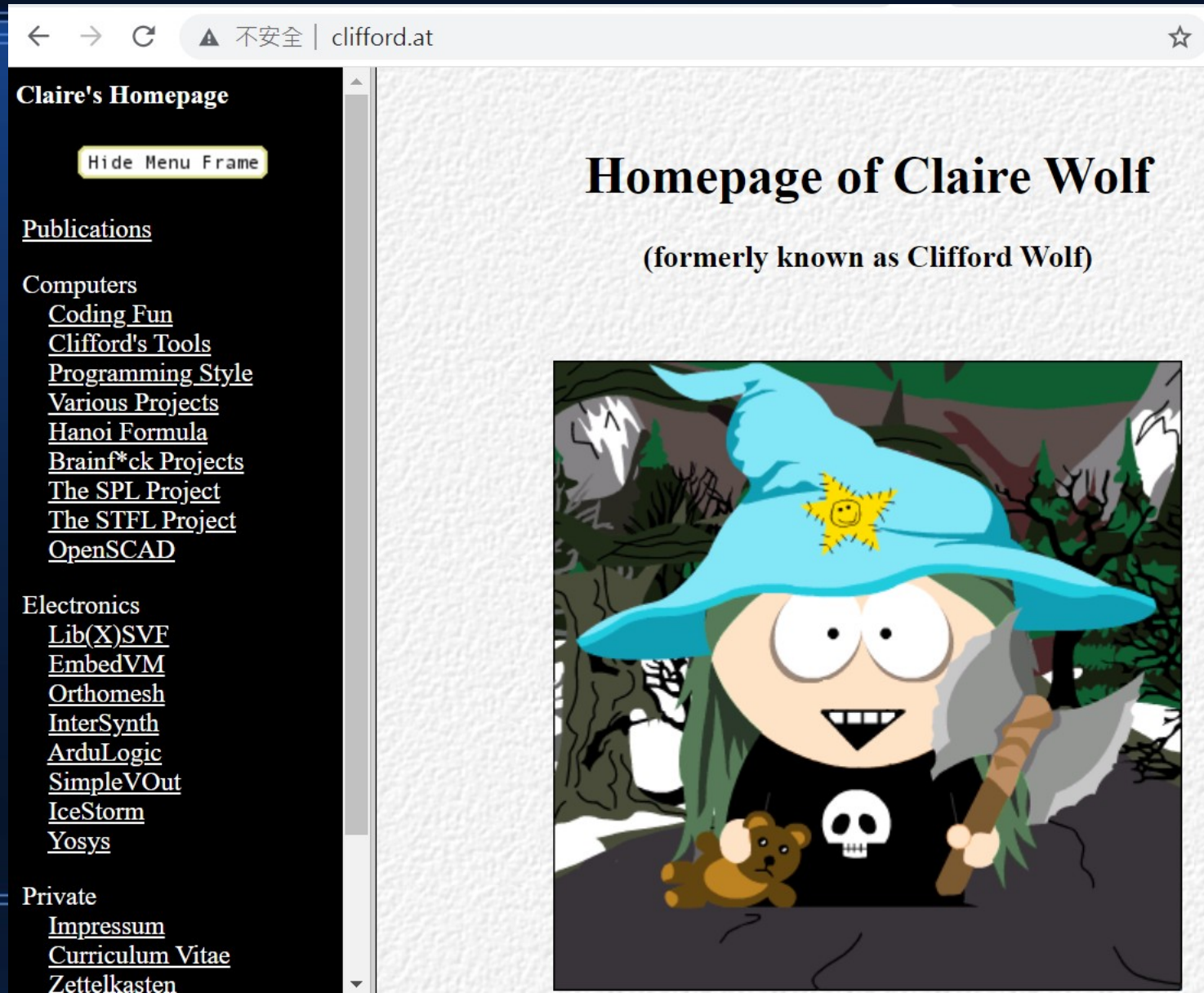
yosys/nextpnr/icestorm 都是她的作品

从2020年开始Claire Xen在她的新github主页中开始重构前面提到的picorv32，新的项目名字叫picorv ([clairexen/picorv](#)) : Clean rewrite of PicoRV32, with improved flexibility and extensibility, support for RV64, and more.

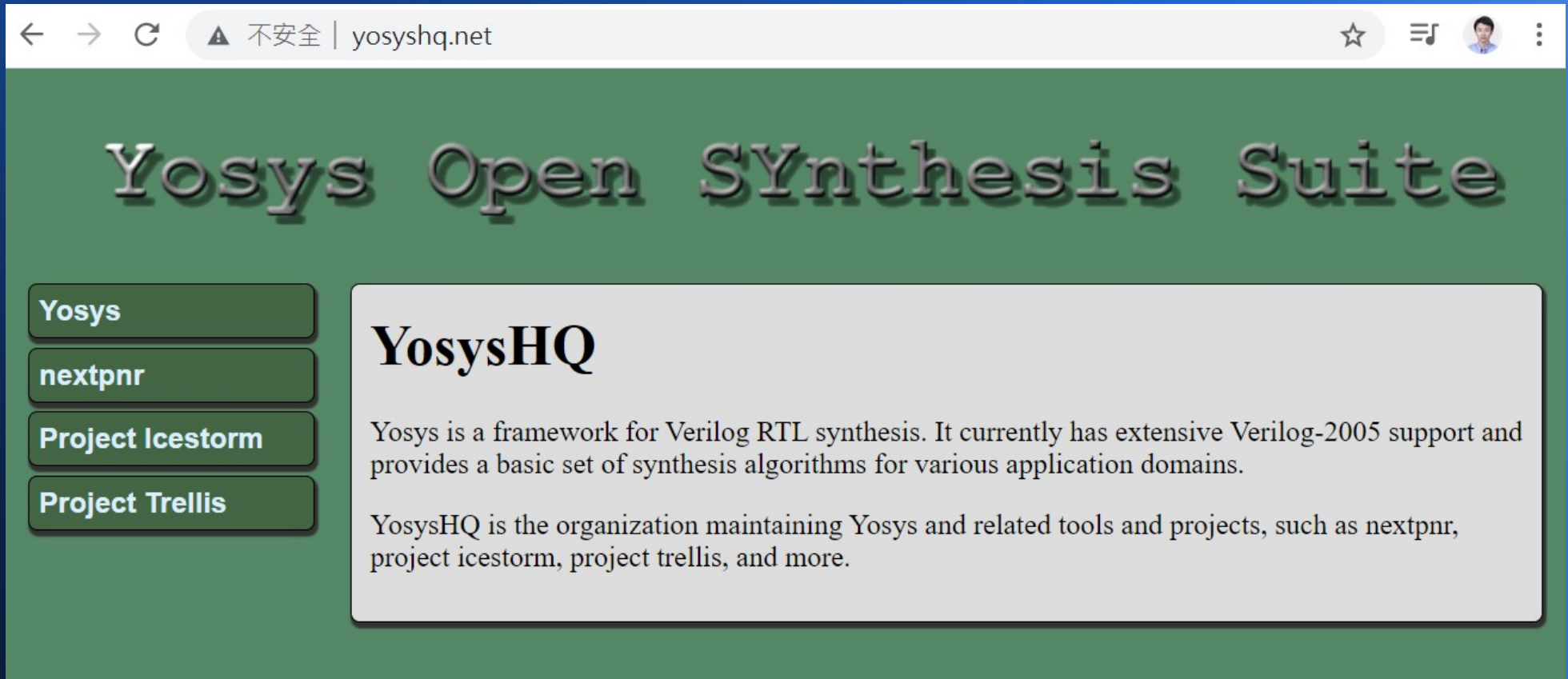
从2021年开始，**Co-Founder & CTO at YosysHQ** ([YosysHQ](#))，YosysHQ联合创始人和CTO，看起来离开了之前的Symbiotic EDA。这最后的这段信息，是她新主页上给出的。看公司介绍和之前的Symbiotic EDA一模一样。但是看业务范围的话，Symbiotic EDA主要提供FPGA培训、设计和验证商业服务。而YosysHQ目前看，主要涵盖四大FPGA开源项目：

1. yosys (FPGA综合工具)
2. nextpnr (FPGA布局布线工具)
3. icestorm (Lattice iCE40 FPGA架构信息库和完整生成bitstream工具链组合)
4. trellis (Lattice ECP5 FPGA架构信息库和完整生成bitstream工具链组合)

這是她的網站



這是她目前公司的網站



她似乎想以一己之力

- 補完整個的開源 EDA 工具鏈

或許有一天

- 整個 IC 產業的所有 EDA 工具
- 都可以真的變成 Open Source 也說不定

希望那天

- 早日到來！

從 IC 設計到晶圓代工領域

- 特別是從 EDA 優化的觀點
- 差不多就是用這些技術整合起來的

當然

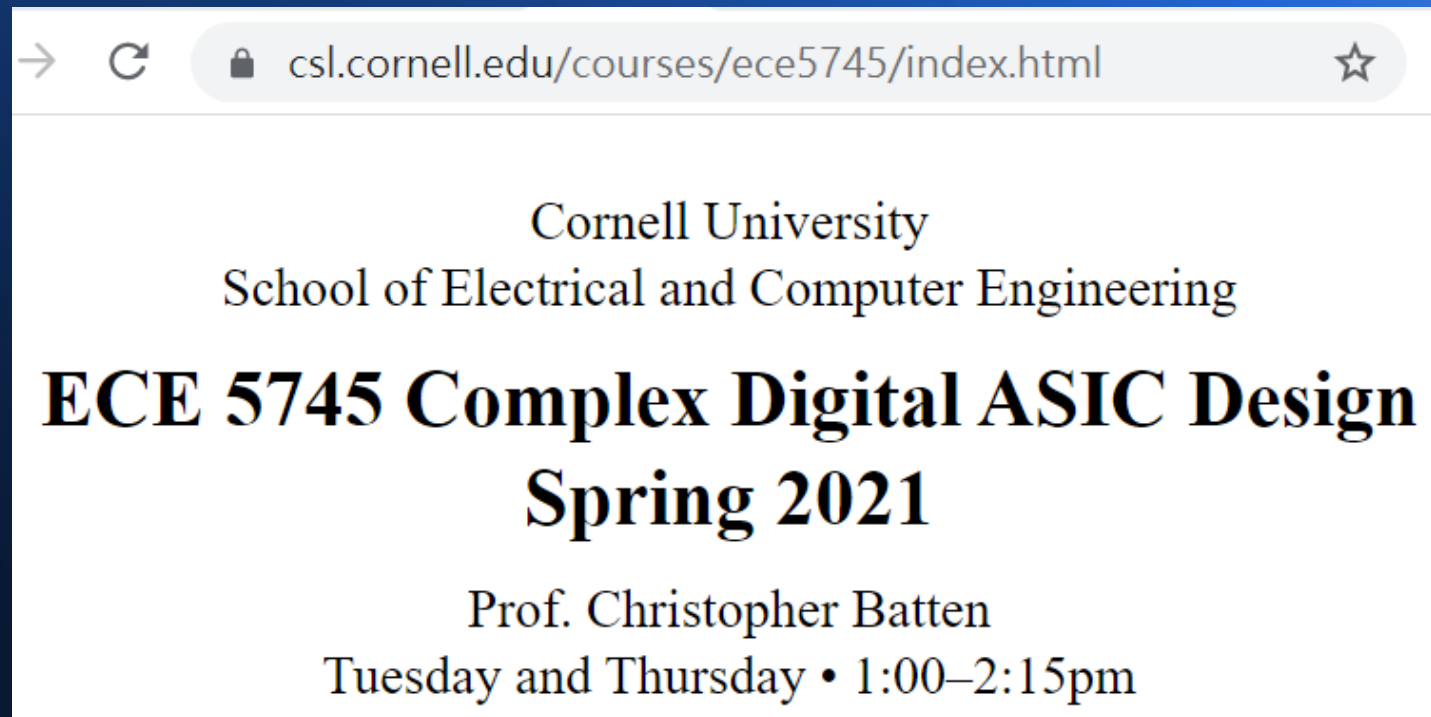
- 這是不完整的

不過應該足以讓有興趣的人

- 開始理解 IC 產業的運作原理了

感謝

- 康乃爾大學的 ASIC 課程



讓我對 IC 產業

- 有了更進一步的理解！